

Technical Report Documentation Page

PDF generated on: 2026 11⁄05 04:07:19 AM

1. Report No. PS-N/PF-1032 R.1	2. Report Date 2026/11/04		
3. Title V31NP NETWORK STANDARD			
4. Author(s) P.K. — 619851078911877 , kokain@v31null.de			
a. Performing Organization Name(s) and Address(es) Bureau for Onion & Over of Þ Services *of* Þe General populace Ministeria custodiae filaciorum (Secura) , aßervatio — fileprtejlf5mpnrgs5isoknpg6zmidwvc2jtpotepkfeocx4zltm2ad.onion , psnpf1032@v31null.de	b. Sponsoring/Monitoring Agency Name(s) and Address(es) Mbrlí úír Entípatùíţù fùţbíz — v31null.de , mef@v31null.de Services for Þe populace General et Private Services for serving of images from Lokal disk — imgsw.v31null.de Kokain organization — v31n.de , kokain@v31null.de Services for Þe populace General et Private Musik applikation of Per.-Gen.-portal nullpunkts as ver. 2 for General use — musik.v31null.de Þe General populace Ministeria custodiae filaciorum (Secura) , aßervatio — file.v31null.de		
6. Abstract This document sets the V31NP NETWORK STANDARD: the exchange of .v31np files between a server and a client in as few requests as possible, the request by which a client states the files it holds, the codes of the answers, the answer by which a client patches its copy, the order in which an archive is rebuilt, the index entry by which a client checks every file and keeps the header fields that the server sends with it, the list and the levels in which a client takes the files it needs, and the byte ranges of an archive and of each file in it.			
7. Key Words V31NP NETWORK STANDARD, .v31np, network, server, client, stub, sync check, V31NPIELALAIMIT, 267 Re-sync, 268 List, CRC-32, index, level, byte range, V31NBAIMITIF, SHA-256, Ed25519, header field, Tor	8. Distribution Statement		
	a. Report A; Approved for public release: distribution is unlimited, no restrictions.	b. Abstract A; Approved for public release: distribution is unlimited, no restrictions.	ú. This page A; Approved for public release: distribution is unlimited, no restrictions.
9. Security Classif. (of this document) **UNCLASSIFIED**	10. №ⁱ OF PP. 20		

(A) INTRODUCTION

(a) Table of Contents

(A) INTRODUCTION....ii
(b) List of Tables....ii
(ú) Notice....iii
(u) Foreword....iii
(ù) Tasked With....iii
(g) Scope and Purpose....v
(d) Notation....v
(B) THE EXCHANGE....1
(a) Architecture....1
(b) Procedures....1
(ú) Status Codes....2
(u) The Index Entry....3
(ù) Integrity and Authenticity....4
(g) Levels....4
(d) Byte Ranges....5
(Ú) THE REQUEST....6
(U) THE 267 ANSWER....7
(Ù) ORDER OF THE REBUILT ARCHIVE....9
(G) ANSWERS PER QUERY....10
(a) The Answer....10
(b) Sync of a Query....10
(ú) The Index....10
(u) The Drop-in — INFORMATIVE....11
(D) FULFILMENT....13
(E) GLOSSARY....14
(Ż) REFERENCES....15

(b) List of Tables

Table A-1 Tasks....iii
Table A-2 Requests of Non-Governmental Organisations....iv
Table B-1 Procedures....1
Table B-2 Status Codes....2
Table B-3 Lines of the Index Entry....3
Table B-4 Levels....5
Table Ú-1 Layout of a Request....6
Table U-1 Layout of a 267 Answer....7
Table U-2 Entries of a 267 File....7
Table Ù-1 Order of Folders and Files....9
Table Ù-2 Order of Names in a Tie....9

Table G-1 Lines of the Index....10
Table D-1 Receipt....13

(ú) Notice

The contents of this document reflect the views of the author, who is responsible for the facts and the accuracy of the data presented herein. This document sets the V31NP NETWORK STANDARD. However, v31n does not act as a confirmation agency. The descriptions of programmes herein are provided for informational purposes, and developers are tasked with the conformance of their own programmes with this standard. V31null does not endorse products or manufacturers. Trade or manufacturer's names appear herein only because they are considered essential to the objective of this document. v31n is not related to non official platforms, services, people, & ‘oc genus omne in any way or form. Textual, programmatic, and media content published across these services is made available under the terms of the v31n license. No statutory copyright, proprietary monopoly, or reservation of rights subsists in these works. Any person is granted full, unconditional, and irrevocable permission to access, copy, modify, adapt, distribute, or incorporate all materials for any purpose, without formality, fee, or prior authorization. The constitutive Latin charter and legal terms of this declaration are recorded in the system registry and resp. places.

(u) Foreword

This document has been published to define the V31NP NETWORK STANDARD, the exchange of .v31np files between a server and a client. It applies to every person dealing with Prinzian systems and with native systems of v31n.

The exchange is defined in § (B). Its part (a) sets out the architecture, its part (b) the procedures between a client and a server, its part (ú) the status codes of the answers, its part (u) the index entry that every .v31np file of the exchange holds, its part (ù) the integrity and authenticity of the files, its part (g) the levels in which a client takes the files, and its part (d) the byte ranges. § (Ú) sets out the request by which a client states the files it holds and those it wants, and § (U) the answer by which a server sends only what differs. § (Ù) sets out the order in which an archive is rebuilt. § (G) sets out the answers per query: the answer, the sync of a query, the index and, informatively, the drop-in. § (D) sets out the fulfilment of the tasks and requests of § (A/(ù)). § (E) gives the terms used in this document, and § (Ż) the documents to which it refers.

P.K.

(ù) Tasked With

This document is tasked with the requests of the agencies in Table A-1, and takes up the requests of non-governmental organisations in Table A-2. Their fulfilment is set out in § (D).

Table A-1 Tasks

Requester	Task
Services for þe populace General et Private Services for serving of images from Lokal disk — imgsw.v31null.de	Delivery of many images in the answer to a single request,

Requester	Task
	also where that request is made to a page that the server generates, as with PHP
Mbrlí úir Entípatùíțù fùțbíz — v31null.de	Security of the exchange: the integrity and authenticity of every file exchanged, and the protections that the server sets for its pages, in force for every file answered from an archive
þe General populace Ministeria custodiae filaciorum (Secura) , aßervatio — fileprtejlf5mpnrgs5isoknpg6zmidwvc2jtpotepkfeocx4zltm2ad.onion	Availability of the exchange over Tor, through onion services, also for clients that keep no storage between sessions
Services for þe populace General et Private Musik applikation of Per.-Gen.-portal nullpunkts as ver. 2 for General use	Serving of static files, as audio, video, images, fonts, styles and scripts, from one archive kept by the client, also in parts by byte range

Table A-2 Requests of Non-Governmental Organisations

Requester	Request
Kokain organization — v31n.de , kokain@v31null.de	Serving of the exchange by a Node.js server; references written as usual, whether relative, from the root of the site or to other sites, working without change; and a ready-to-use function of the Bureau, dropped into a site and used by a single call

(g) Scope and Purpose

The V31NP NETWORK STANDARD, as defined by v31n, applies to every exchange of .v31np files between a server and a client over a network. A site is loaded in as few requests as possible.

The .v31np file is defined in PS-N/PF-1031, and every request and answer of this document that carries files is a .v31np file as defined there. This document defines the formats and the meaning of the codes. How a server answers, and which further checks it runs, is chosen by its administrator.

(d) Notation

Bytes are written in hexadecimal, two digits to a byte, as in 00 and 01. π and φ denote the marks of PS-N/PF-1031, Table B-2. Numbers are stored in little-endian byte order, the least significant byte first.

A reference to a part of this document gives the sign § followed by the division and the part, as in § (B/(a)), which denotes (a) of (B). Tables are numbered by division, as in Table B-1 and Table U-2.

The requirements of this document are stated with shall, and each binds unless this document states otherwise. Should denotes a recommendation, which does not bind, and may denotes a permission. A part marked INFORMATIVE gives guidance and binds no one.

(B) THE EXCHANGE

(a) Architecture

The page that a client opens first is a stub. The stub shall carry the name and the URL of its archive, and may carry the CRC-32 of that archive. It carries them as the meta elements v31np-archive, v31np-url and v31np-crc, or as the attributes data-archive, data-url and data-crc of its script element, and a client reads whichever is present. The stub performs the handshake before the content of the site is loaded. After the handshake, every file of the site comes out of one .v31np file.

Every key of the exchange begins with /, except the empty key and the key v31np index.txt.

The client keeps its copy of the archive locally. A browser keeps it in its Indexed Database (W3C, Indexed Database API), each file in parts of 4 194 304 bytes, so that a byte range reads only its parts; it computes the CRC-32 and the SHA-256 of a file while it stores it. A programme that is not a browser, as one that serves a local host to its own view, keeps its copy as one .v31np file. From the list of § (B/(b)) it writes that file at its full size, with the header built from the rows of the index entry and the footer with the CRC-32 of the archive, writes every file that it receives at its address, and notes beside the file which entries are present. It reads a file at its address. Once every entry is present, the file is a complete .v31np file (PS-N/PF-1031).

Pages that a server generates, as with PHP, are served by plain HTTP, or as an answer per query of § (G), which holds the executed page together with the files to which it refers. References to images and other files are written as usual. A browser that runs JavaScript should answer them by a Service Worker (W3C, Service Workers) from the stored archive by key, with the header fields of § (B/(u)); otherwise they are requested from the server.

Every .v31np file of the exchange is sent with the content type application/v31np. No .v31np file of the exchange, and no file answered by its key, shall be sent with the field Content-Encoding.

This document applies to the exchange over a network. Programmes that present .v31np files on a system, as set out in PS-N/PF-1031, § (Ú), are not bound by it.

(b) Procedures

Table B-1 presents the procedures between a client and a server.

Table B-1 Procedures

Procedure	Request	Answer
Handshake	The client opens the stub	The stub
Sync check	The client sends its request of § (Ú) by POST to the URL of the archive	One of the codes of Table B-2
Re-sync	Part of the sync check	A 267 answer of § (U), from which the client patches its copy
List	Part of the sync check	268 with the index entry of the archive

Procedure	Request	Answer
Wanted files	The client names the keys that it wants in its request of § (Ú)	A 267 answer of § (U) that holds those files
Byte ranges	The client requests the URL of the archive, or of a file in it, by GET with the field Range	206 with the ranges, as set out in § (B/(d))
Full file	The client requests the URL of the archive by GET	200 with the full .v31np file

The client sends its request on every visit, also where the CRC-32 in the stub equals that of its copy. The server computes the CRC-32 of its archive and of each file in it once, when the archive is built, and afterwards only reads them.

The procedures run as follows:

1. On a first visit, the client holds no files and sends an empty request. The server answers with the full file (200), points to a pre-built file (303), or gives the list (268).
2. On a return visit where the copy is in sync, the server answers 304, and nothing follows.
3. On a return visit where the copy is out of sync, the server answers with only what differs (267), with the full file (200), or with the new list (268), as the server decides.
4. A server may point every client to a pre-built file (303), which it writes once.
5. A client may request the full .v31np file. A server should give the list, and then the files by wanted keys (§ (Ú)) and by byte ranges (§ (B/(d))).

A client shall send its request for a copy only to the origin of the archive of that copy. A client may send an empty request in place of the files that it holds. Where a full file breaks off, the client may request the rest by the field Range with the field If-Range, or request the full file again.

The list is the index entry of the archive (§ (B/(u))). A server that gives the list compares only the CRC-32 of the index entry in the request with that of its own: where they are equal it answers 304, and otherwise 268 with its index entry. From the rows of the index entry the client computes the address of every file: the header of the archive is built from the keys and the sizes in the order of § (Ù), and the files follow in that order. The CRC-32 of the archive computed from the rows equals the field ETag of the 268 answer.

(ú) Status Codes

Table B-2 presents the status codes of the answers and their meaning in this document. The codes other than 267 and 268 are those of RFC 9110. 267 and 268 are sent over HTTP/1.1 with the reason phrases Re-sync and List, in ASCII; HTTP/2 and HTTP/3 carry the number alone.

Table B-2 Status Codes

Code	Meaning
200	The full file follows
206	The byte ranges asked follow, as set out in § (B/(d))

Code	Meaning
267 Re-sync	Only what differs, or the files wanted, follow, as a 267 answer of § (U)
268 List	Only the index entry of the archive follows, as a .v31np file that holds that single entry
303	The pre-built file named in the field Location answers the request
304	The copy is in sync, and nothing follows
400	The field Range is malformed
404	The archive, or the file named by the query key=, does not exist
405	The method is neither GET, HEAD nor POST, or, for a file requested by its key, neither GET nor HEAD
413	The request is larger than the server accepts
414	The URL of a file requested by its key is longer than the server accepts
415	A request by POST to the URL of the archive does not carry the content type application/v31np
416	No range of the field Range can be satisfied
422	The request is broken, as by a wrong magic or a wrong checksum

The field ETag shall carry the CRC-32 in 8 hexadecimal digits in upper case within quotation marks, as a strong validator (RFC 9110). A server should send the field Cache-Control with no-store with 267, with 268, and with every answer to a sync check. On 401, 403, 404, a code of the class 5xx, or a code that it does not know, a client shall keep its copy and show from it. A server may answer 413 and 414 by limits of its own, and a client may refuse an answer larger than it can store.

A client in a browser sends its sync check with the cache mode no-store of the Fetch Standard. A 303 is followed by the browser itself; the client reads the final answer, whose URL names the pre-built file.

(u) The Index Entry

Every archive of the exchange, and every answer per query of § (G), holds one entry under the key v31np index.txt, the index entry. The server builds it from its index of § (G/(ú)) when it builds the archive or the answer, for the files of that archive or answer only. The index entry begins with the magic V31NBAIMITIF followed by 3 bytes 00, 15 bytes in all, and is otherwise written as the index: UTF-8 text, lines ending with a carriage return and a line feed, fields separated by a tab, text fields escaped as in PS-N/PF-1031, Table G-3, and numbers written in decimal. Table B-3 presents the lines of the index entry.

Table B-3 Lines of the Index Entry

Line	Fields	Contents
Magic	V31NBAIMITIF, 00	The first line

Line	Fields	Contents
	00 00	
Row	I, key, size, CRC-32, SHA-256, level	One file of the archive or answer: its key, its size in bytes, its CRC-32 in 8 hexadecimal digits in upper case, where the server signs its SHA-256 in 64 hexadecimal digits in upper case, and its level of § (B/(g)) where one is written. The executed page of an answer per query has its row under the empty key
Header field	H, key, name, value	One header field (RFC 9110) that the server sends with the file of the key when that file is requested by itself
Signature	S, signature	Where the server signs: the Ed25519 signature (RFC 8032) over every byte of the index entry before this line, in 128 hexadecimal digits in upper case. The last line

The rows are written in the order of § (Ù), each key once, and the header fields follow the rows, in the order of their keys. The index entry has no row of its own.

A programme recognises the index entry by its magic. It should not care about it, and may not show it.

Where a client answers a request for a file from its copy, it answers with the header fields of its key, beside the content type, the length and the byte ranges of the file.

(ù) Integrity and Authenticity

The transport gives the integrity and authenticity of the exchange: HTTPS, or the onion address of an onion service of Tor. CRC-32 serves the sync check.

A server may add to its index entries the SHA-256 (FIPS 180-4) of every file and an Ed25519 signature (RFC 8032), as set out in Table B-3. Its public key reaches the client by other means than the archive and the stub, as the client chooses.

A client that holds the public key of the server checks its copy before it stores it, whether received whole or patched by a 267 answer: the signature of the index entry, and for every other file its size, its CRC-32 and its SHA-256 against its row, and that every file has a row. Where a check fails, or the index entry or its signature is missing, the client requests the full file once; where that fails too, it stores nothing of the answer.

A client shall drop every key that does not begin with /, and every key with a segment . or .., except the empty key and the key v31np index.txt, and store the rest of the answer.

(g) Levels

Every file of an archive belongs to one of six levels, which a client takes in their order. The level of a file is written as the last field of its row in the index (§ (G/(ú))) and in the index entry (§ (B/(u))). A file whose row has no level belongs to level Ù. Table B-4 presents the levels.

Table B-4 Levels

Level	Contents
S	Essentials: the files that a page needs to show itself
A	Additional
B, Ú, U, Ù	The rest, in this order

A page that answers per query (§ (G)) answers with the executed page and the files of level S, and with the files of further levels where the server chooses. The client then takes the files of level A, by one request of wanted keys. It takes the files of the levels B to Ù when it requests them, or when the server sends them.

(d) Byte Ranges

A server answers byte ranges (RFC 9110) of an archive at the URL of the archive, and of a file in it at the URL of the archive with the query key=, followed by the key of the file. The key in the query shall be written in UTF-8 and percent-encoded (RFC 3986), and the server decodes it once. A file requested by its key is answered as a file of its own: with its content type, the header fields of its key (§ (B/(u))), its CRC-32 in the field ETag, and its own byte ranges. Audio and video are played from it.

1. A request of one range is answered 206 with the field Content-Range.
2. A request of several ranges is answered by one 206 answer of the content type multipart/byteranges, whose boundary is v31np- followed by the CRC-32 of the archive or of the file; each part carries its own fields Content-Type and Content-Range.
3. A range that cannot be satisfied is dropped; where none is left, the answer is 416.
4. A malformed field Range is answered 400.
5. Where the field If-Range does not match the field ETag, the whole archive or file is answered (200).

A client that answers byte ranges from its copy answers them in the same way.

(Ú) THE REQUEST

The request is a .v31np file that holds a single entry, whose key is empty. That entry is the request file, which begins with the magic V31NPIELALAIMIT, 15 bytes of ASCII. Table Ú-1 presents the layout of a request.

Table Ú-1 Layout of a Request

Part	Length (bytes)	Contents
Header of the .v31np file		
Magic	15	V31NSPACKFORMAT (ASCII)
π, T1, π, T2, π	variable	One row, for the empty key
Request file		
Magic	15	V31NPIELALAIMIT (ASCII)
π	12	Mark
T1	variable	One row per file of the copy: key, address
π	12	Mark
T2	variable	One row per file of the copy: key, size 8, a, b, c
π	12	Mark
CRC-32 of the files	8 per file	One CRC-32 per file, in the order of T1, in hexadecimal digits (ASCII)
π	12	Mark
CRC32	4	Checksum of the request file
Footer of the .v31np file		
π	12	Mark
CRC32	4	Checksum

The header of the request file is binary and is built as the header of a .v31np file (PS-N/PF-1031, § (B/(a))). It lists the keys of the files of the copy, and the keys that the client wants, in the order of § (Ù). Each entry has the size 8, and the addresses chain through the CRC-32 text of the request file.

Each CRC-32 is written as the number, in 8 hexadecimal digits in upper case, the most significant digit first, without separators. The checksum of the request file is computed as the checksum of a .v31np file (PS-N/PF-1031, § (B/(a.v.))), over every byte of the request file before it.

A key that the client wants and does not hold is written with the CRC-32 text 00000000.

An empty request lists no files: its T1, its T2 and its CRC-32 text are empty.

(U) THE 267 ANSWER

The 267 answer is a .v31np file that holds a single entry, whose key is empty. That entry is the 267 file, which begins with the magic V31N267 followed by 8 bytes 00, 15 bytes in all; the bytes 00 are reserved. Table U-1 presents the layout of a 267 answer.

Table U-1 Layout of a 267 Answer

Part	Length (bytes)	Contents
Header of the .v31np file		
Magic	15	V31NSPACKFORMAT (ASCII)
π, T1, π, T2, π	variable	One row, for the empty key
267 file		
Magic	15	V31N267 (ASCII), 00 00 00 00 00 00 00 00
π	12	Mark
T1	variable	One row per changed key: key, address
π	12	Mark
T2	variable	One row per changed key: key, size, a, b, c
π	12	Mark
Entries	variable	One entry per changed key, in the order of T1, as in Table U-2
π	12	Mark
CRC32	4	Checksum of the 267 file
Footer of the .v31np file		
π	12	Mark
CRC32	4	Checksum

Every key of the 267 file is in its T1 and in its T2. A key of the copy that is not in the 267 file stays as it is. The checksum of the 267 file is computed as that of the request file (§ (Ú)).

Every entry ends in one flag byte. Table U-2 presents the entries and what the client does with each.

Table U-2 Entries of a 267 File

Entry	Data	Size	The client
Deleted	01	1	Deletes the key
Changed or new	The bytes of the file, then 00	Size of the file + 1	Replaces the file with the bytes before the flag

Entry	Data	Size	The client
Now empty	00	1	Replaces the file with an empty file

A server that gives the list (§ (B/(b))) answers a request of wanted keys with one 267 answer that holds every wanted file as changed or new. A wanted key that is not in the archive comes as deleted. Where the index entry of the request is not the current one, the current index entry comes in the same answer.

The server sends the CRC-32 of its whole archive in the field ETag (RFC 9110). After patching, the client may compare the CRC-32 of its rebuilt archive with it, and on a mismatch it requests the full file.

(Ù) ORDER OF THE REBUILT ARCHIVE

The client rebuilds its archive in the order in which the server builds its own, as follows:

1. Keys are split at / and compared part by part, and at each level folders come before files.
2. Within a level, names are compared in alphabetical order, and a tie is broken, as set out in PS/G-1104, § (B/(b)).

The lower case of the folding of PS/G-1104 is the simple lowercase mapping of Unicode 18.0, field 13 of UnicodeData.txt.

Table Ù-1 presents keys in this order, and Table Ù-2 names that are equal after folding.

Table Ù-1 Order of Folders and Files

Place	Key
1	/m/img/a.png
2	/m/img/b.png
3	/m/a.png
4	/m/b.png
5	/z.png

Table Ù-2 Order of Names in a Tie

Place	Name
1	A.png
2	a.png
3	AA.png
4	AAa.png
5	aaaa.png
6	ú.png
7	V.png
8	v.png

(G) ANSWERS PER QUERY

(a) The Answer

A page that a server generates may answer with a .v31np file that holds the page as executed together with the files to which it refers. The entry with the empty key holds the executed page, and every other entry holds a file to which the page refers, under the reference of the page, resolved against the URL of the page and written from the root of the host. The server builds the answer when it is requested, from the files as they are stored, through the index of § (G/(ú)), with its entries in the order of § (Ù). The server sends the CRC-32 of the whole answer in the field ETag (RFC 9110). The answer holds the files of level S, and of further levels where the server chooses (§ (B/(g))).

The page that a browser opens is a shell that holds the script. The shell carries the name and the URL of the archive as the meta elements v31np-archive and v31np-url, as a stub does. The script requests the answer and shows the executed page, every reference of the page and of its scripts taken from the stored files, so that every request of the page goes to the URL of the archive. A file that the client does not hold is taken by its key (§ (B/(d))). What the shell shows before the answer has arrived is the choice of the administrator.

The page answers itself, at its own URL. A request by POST with the content type application/v31np, or by GET or HEAD with application/v31np in the field Accept, is answered with the answer per query; every other request is answered with the shell. The requests of the client carry these fields. Other requests of the page, as forms, reach it unchanged. Such a page should send the field Vary with Accept.

(b) Sync of a Query

A client keeps the answer of each query as a copy of its own, named by the URL of the query. It sends the request of § (Ú) for that copy by POST to the URL of the query, an empty request where it holds no copy, and the server answers with the codes of Table B-2: 200 with the full answer, 304 where the copy is in sync, and 267 with what differs, as set out in § (U). A key of the copy that the answer no longer holds is deleted. A GET of the URL of the query with application/v31np in the field Accept answers with the full answer.

(ú) The Index

The server finds each file of an answer per query, and of an archive that it builds, through its index, the file v31np index.txt, without searching the folders in which the files are stored. The index of a site lies in the root folder of the site. A file that is not in the index is not served. The index is UTF-8 text. Its lines end with a carriage return and a line feed, a reader also accepts a line feed alone, and empty lines are ignored. The fields of a line are separated by a tab, text fields are escaped as in PS-N/PF-1031, Table G-3, and numbers are written in decimal. Table G-1 presents the lines of the index.

Table G-1 Lines of the Index

Line	Fields	Contents
Magic	V31NBAIMITIF, 00 00 00	The first line that is not empty, as in the index entry of § (B/(u))
Root	root	The next line that is not empty: the folder of the files, relative to the

Line	Fields	Contents
		folder of the index
Row	I, key, size, CRC-32, path, time, SHA-256, level	One file: its key, its size in bytes, its CRC-32 in 8 hexadecimal digits in upper case, its path relative to the root, the time of its last write as in the journal of PS-N/PF-1031, § (G/(b.vi.)), where the server signs its SHA-256 in 64 hexadecimal digits in upper case, and its level of § (B/(g)) where one is written
Header field	H, key, name, value	One header field (RFC 9110) that the server sends with the file of the key when that file is requested by itself

The rows are written in the order of § (Ù), each key once, and the header fields follow the rows. No line divides the rows from the header fields; the first field of a line names its kind. Where the size of a file, or the time of its last write counted in seconds, differs from its row, the server computes the CRC-32 of the file again, and its SHA-256 where it signs, and writes the row anew. The index is written whole, to a temporary file in the same folder that then replaces it.

(u) The Drop-in — INFORMATIVE

The Bureau gives the exchange as one folder, v31np, which a site places in the root folder of its host, and which serves a PHP server and a Node.js server alike. The archive covers the whole host, so that references written relative to the page, from the root of the site, or to other sites work without change. The site names its archive. One command, of PHP or of Node.js, builds the archive, the index of § (G/(ú)) and the script of the folder; both build the same bytes. The command takes the files of the types of pages, styles, scripts, images, fonts, audio and video, and further types that the administrator names, and of those only the files that the rules of the server do not deny, and none in a folder named node_modules.

A site loads its pages in one of two forms:

1. A page carries in its head a script element whose source is /v31np/v31np.js and whose attributes data-archive and data-url name the archive and its URL; the page is otherwise unchanged. The script stops the reading of the page by the browser, takes the list (§ (B/(b))), takes by one request of wanted keys (§ (Ú)) every file named by the markup of the page that the client does not hold, then the files named by its styles, and shows the page with every such reference taken from the stored files. Audio and video are taken by their keys (§ (B/(d))). Links, forms and frames keep their URLs. A meta element with the attribute http-equiv Content-Security-Policy (W3C, Content Security Policy Level 3) and the content default-src * data: blob: 'unsafe-inline' 'unsafe-eval', placed before the script, keeps the browser from fetching the files named by the markup before the script runs. Where the script comes from another origin, as from the Bureau, the URL of data-url is read against the URL of the page. Where the server signs, the attribute data-key of the script element gives its public key in 64 hexadecimal digits. The attribute data-query names, separated by spaces, the URLs of the site whose answers come per query (§ (G/(a))). The client asks them with the request of § (Ú), gives the executed answer to the page, and keeps every other file of the answer under its key, which is the URL, query included, by which the page asks for that file, written from the root of the host.

2. A page answers per query (§ (G/(a))) by a single call at its top: v31np with the name of the archive in PHP, and v31np.page with the request, the answer, the name of the archive and the page in Node.js.

A Node.js server serves the exchange by the module of the folder, called with the request and the answer as a handler, or used as middleware of Express. It answers the URLs of the archive, of its pre-built file and of the script, and refuses every other URL of the folder.

A URL named in data-query answers per query by the call v31np_query at its top, with the index of its files and a function that gives the keys of the files from its output. Every other request receives its usual answer.

An existing site takes the drop-in as follows:

1. The script element goes into the head of the page as the browser receives it. Where one file of the site includes another to build the page, it goes into the file that writes the head.
2. Files outside the root folder of the host, and files that a script of the site serves under a query, come with the answer of the URL that lists them, each under the URL by which the page asks for it. The index of such files lies beside them, and its rows keep the key apart from the path of the file.
3. Settings of the server that apply to every script of the site, as a file prepended to each, apply to the scripts of the folder v31np too. A setting that names a file by a relative path names it there by its full path.
4. Links, forms, the requests of the page and its scripts work as before; only the transfer of the files changes. A list of the site comes as one answer, and the files it names are then taken from the stored files.

(D) FULFILMENT

Table D-1 presents the state of the tasks of Table A-1 and of the requests of Table A-2, and the parts of this document that fulfil them.

Table D-1 Receipt

Requester	Task	State	See
imgsw	Many images, one request	Done	§§ (B/(a)) & (G)
MEF	Security of the exchange	Done	§§ (B/(u))∽(B/(ù)) & (G/(ú))
Secura	Availability over Tor	Done	Format itself
m2	Static files, byte ranges	Done	§§ (B/(a))∽(B/(u)) , (B/(g))∽(B/(ɗ)) , (Ú)∽(U) , (G/(a)) , & (G/(ú))
Kokain	Node.js, references, drop-in	Done	§§ (G/(a)) & (G/(u))

(E) GLOSSARY

267 answer	The answer by which a server sends only what differs; see § (U).
Answer per query	A .v31np file that holds a page as executed together with the files to which it refers; see § (G).
Byte range	A part of an archive or of a file in it, named in the field Range (RFC 9110); see § (B/(d)).
Copy	The archive that a client keeps locally.
Executed page	The output of a page that a server generates, as sent to a client.
Handshake	The opening of the stub, before the content of the site is loaded.
Header field	A field of the header of an HTTP answer (RFC 9110), as CSP, HSTS, X-Frame-Options, X-Content-Type-Options and Referrer-Policy.
Index	The file through which a server finds the files of its answers per query and of the archives it builds; see § (G/(ú)).
Index entry	The entry v31np index.txt of an archive or an answer per query, which lists its files, their header fields and the signature; see § (B/(u)).
Level	One of the six levels S, A, B, Ú, U and Ù, in which a client takes the files of an archive; see § (B/(g)).
List	The index entry of an archive, sent alone with 268; see § (B/(b)).
Pre-built file	A file that a server writes once, and to which it points clients with 303.
Request	The .v31np file by which a client states the files it holds and those it wants; see § (Ú).
Shell	The page that a browser opens for a page that answers per query, holding the script that requests the answer; see § (G/(a)).
Stub	The page that a client opens first, carrying the name and the URL of the archive.
Sync check	The request of a client and the answer of the server by which the copy is compared with the archive of the server; see § (B/(b)).
Wanted key	A key that a client names in its request with the CRC-32 text 00000000; see § (Ú).

(Ż) REFERENCES

Internet Engineering Task Force (IETF) (2005) "RFC 3986: Uniform Resource Identifier (URI): Generic Syntax".

Internet Engineering Task Force (IETF) (2022) "RFC 9110: HTTP Semantics".

Internet Research Task Force (IRTF) (2017) "RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)".

National Institute of Standards and Technology (NIST) (2015) "FIPS 180-4: Secure Hash Standard (SHS)".

Unicode Consortium (2026) "The Unicode Standard, Version 18.0".

W3C (n.d.) "Content Security Policy Level 3".

W3C (n.d.) "Indexed Database API".

W3C (n.d.) "Service Workers".

WHATWG (n.d.) "Fetch Standard".

PS-N/PF-1031

PS/G-1104



V31NP : Drop-in

Document: PS-N/PF-1032

PDF generated on: 2026 11/05 04:07:16 AM

1 - File: lib/answer.php

Size

6222

SHA-256

06A97203A5B9CB064A7622D0FD4B567FA16FF3081E121C300CA1A21F9BD8BDB9

Code

```
<?php
require_once __DIR__ . '/pack.php';
require_once __DIR__ . '/index.php';

final class V31nAnswer
{
    public static function start(): void
    {
        ob_start();
    }

    private static function clear(): void
    {
        while (ob_get_level()) {
            ob_end_clean();
        }
    }

    private static function ask(string $body): ?array
    {
        $outer = V31nPack::parse(V31nPack::MAGIC, $body);
        if ($outer === null || count($outer['entries']) !== 1 || $outer['entries'][0][0] !== '') {
            return null;
        }
        [, $at, $size] = $outer['entries'][0];
        $inner = substr($body, $at, $size);
        $ask = V31nPack::parse(V31nPack::ASK, $inner);
        if ($ask === null) {
            return null;
        }
        $client = [];
        foreach ($ask['entries'] as [$key, $at, $size]) {
            $text = substr($inner, $at, $size);
            if ($size !== 8 || !preg_match('/^[0-9A-F]{8}$/', $text)) {
                return null;
            }
            $client[(string)$key] = $text;
        }
        return $client;
    }

    public static function send(V31nIndex $index, array $keys): void
    {
        $html = ob_get_clean();
        $rows = [];
        $secret = V31nIndex::secret();
        foreach ($keys as $key) {
            $key = (string)$key;
            if ($key === '' || $key === V31nIndex::KEY || isset($rows[$key])) {
                continue;
            }
            $row = $index->fresh($key, $secret !== null);
            if ($row !== null) {
                $rows[$key] = $row;
            }
        }
        $index->save();
        $page = ['size' => strlen($html), 'crc' => V31nPack::hex(crc32($html)), 'sha' => strtoupper(hash('sha256', $html))];
        $mem = ['' => $html, V31nIndex::KEY => $index->entry(['' => $page] + $rows, $secret)];
        $all = array_merge(array_map('strval', array_keys($mem)), array_map('strval', array_keys($rows)));
        usort($all, [V31nOrder::class, 'key']);
        $server = [];
        $items = [];
        foreach ($all as $key) {
            $size = isset($mem[$key]) ? strlen($mem[$key]) : $rows[$key]['size'];
            $crc = isset($mem[$key]) ? V31nPack::hex(crc32($mem[$key])) : $rows[$key]['crc'];
            $server[$key] = ['size' => $size, 'crc' => $crc];
            $items[] = [$key, $size];
        }
        $head = V31nPack::head(V31nPack::MAGIC, $items);
        $sum = crc32($head);
        $length = strlen($head) + V31nPack::FOOT;
        foreach ($all as $key) {
            $sum = V31nPack::combine($sum, hexdec($server[$key]['crc']), $server[$key]['size']);
            $length += $server[$key]['size'];
        }
        $sum = V31nPack::combine($sum, crc32(V31nPack::PI), V31nPack::MARK);
        self::clear();
        header('Cache-Control: no-store');
        header('ETag: "' . V31nPack::hex($sum) . '"');

        $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
        if ($method === 'POST') {
            $client = self::ask(file_get_contents('php://input'));
            if ($client === null) {
                http_response_code(422);
                exit;
            }
            if ($client) {
                $changed = [];
                foreach ($all as $key) {
                    if (($client[$key] ?? null) !== $server[$key]['crc']) {
                        $changed[] = $key;
                    }
                }
                $deleted = array_values(array_diff(array_keys($client), $all));
                if (!$changed && !$deleted) {
                    http_response_code(304);
                    exit;
                }
                $keys = array_merge($changed, array_map('strval', $deleted));
                usort($keys, [V31nOrder::class, 'key']);
                $entries = [];
                foreach ($keys as $key) {
                    if (!isset($server[$key])) {
                        $entries[] = [$key, "\x01"];
                    } elseif (isset($mem[$key])) {
                        $entries[] = [$key, $mem[$key] . "\x00"];
                    } else {
                        $bytes = $server[$key]['size'] > 0 ? file_get_contents($index->path($key), false, null, 0, $server[$key]['size']) : '';
                        if (strlen($bytes) !== $server[$key]['size']) {
                            http_response_code(500);
                            exit;
                        }
                        $entries[] = [$key, $bytes . "\x00"];
                    }
                }
                $answer = V31nPack::write(V31nPack::MAGIC, [['', V31nPack::write(V31nPack::PATCH, $entries)]]);
                header('HTTP/1.1 267 Re-sync');
                header('Content-Type: application/v31np');
                header('Content-Length: ' . strlen($answer));
                echo $answer;
                exit;
            }
        } elseif ($method !== 'GET' && $method !== 'HEAD') {
            http_response_code(405);
            header('Allow: GET, HEAD, POST');
            exit;
        }

        header('Content-Type: application/v31np');
        header('Content-Length: ' . $length);
        if ($method === 'HEAD') {
            exit;
        }
        $ctx = hash_init('crc32b');
        echo $head;
        hash_update($ctx, $head);
        foreach ($all as $key) {
            if (isset($mem[$key])) {
                echo $mem[$key];
                hash_update($ctx, $mem[$key]);
```

```
            continue;
        }
        $left = $rows[$key]['size'];
        $h = fopen($index->path($key), 'rb');
        while ($left > 0 && $h !== false && !feof($h)) {
            $chunk = fread($h, min(1048576, $left));
            if ($chunk === false || $chunk === '') {
                break;
            }
            echo $chunk;
            hash_update($ctx, $chunk);
            $left -= strlen($chunk);
            flush();
        }
        if ($h !== false) {
            fclose($h);
        }
        if ($left !== 0) {
            exit;
        }
    }
    echo V31nPack::PI;
    hash_update($ctx, V31nPack::PI);
    echo pack('V', hexdec(hash_final($ctx)));
    exit;
  }
}
```

2 - File: lib/index.php

Size

11603

SHA-256

599C55D08E8B2898213B30CD214C56376DFF1E216ACD9DC53183DACD9C80FA19

Code

```
<?php
require_once __DIR__ . '/order.php';

final class V31nIndex
{
    const MAGIC = "V31NBAIMITIF\0\0\0";
    const KEY = 'v31np index.txt';
    const SHIFT = 11644473600;
    const LEVELS = ['S', 'A', 'B', 'Ú', 'U', 'Ů'];

    private string $file;
    private string $root;
    private string $rootText;
    private array $rows = [];
    private array $heads = [];
    private bool $dirty = false;

    private function __construct(string $file, string $rootText)
    {
        $this->file = $file;
        $this->rootText = $rootText;
        $this->root = dirname($file) . '/' . $rootText;
    }

    public static function escape(string $s): string
    {
        return strtr($s, ["\\" => "\\\\", "\t" => "\\t", "\n" => "\\n", "\r" => "\\r"]);
    }

    public static function unescape(string $s): ?string
    {
        $out = '';
        $n = strlen($s);
        for ($i = 0; $i < $n; $i++) {
            $c = $s[$i];
            if ($c !== "\\") {
                $out .= $c;
                continue;
            }
            $e = $s[++$i] ?? '';
            $map = ["\\" => "\\", 't' => "\t", 'n' => "\n", 'r' => "\r"];
            if (!isset($map[$e])) {
                return null;
            }
            $out .= $map[$e];
        }
        return mb_check_encoding($out, 'UTF-8') ? $out : null;
    }

    public static function fileTime(string $path): int
    {
        return (filemtime($path) + self::SHIFT) * 10000000;
    }

    public static function crcOf(string $path): string
    {
        return strtoupper(hash_file('crc32b', $path));
    }

    public static function shaOf(string $path): string
    {
        return strtoupper(hash_file('sha256', $path));
    }

    public static function secret(): ?string
    {
        $settings = json_decode((string)@file_get_contents(__DIR__ . '/../settings.json'), true);
        $file = is_array($settings) ? (string)($settings['sign'] ?? '') : '';
        if ($file === '') {
            return null;
        }
        $seed = trim((string)@file_get_contents(__DIR__ . '/../' . $file));
        if (!preg_match('/^[0-9A-F]{64}$/', $seed)) {
            throw new RuntimeException('sign key');
        }
        return sodium_crypto_sign_secretkey(sodium_crypto_sign_seed_keypair(hex2bin($seed)));
    }

    public static function load(string $file): ?self
    {
        $text = @file_get_contents($file);
        if ($text === false) {
            return null;
        }
        $index = null;
        $magic = false;
        foreach (preg_split('/\r?\n/', $text) as $line) {
            if ($line === '') {
                continue;
            }
            if (!$magic) {
                if ($line !== self::MAGIC) {
                    return null;
                }
                $magic = true;
                continue;
            }
            if ($index === null) {
                $root = self::unescape($line);
                if ($root === null || $root === '') {
                    return null;
                }
                $index = new self($file, $root);
                continue;
            }
            $f = explode("\t", $line);
            if ($f[0] === 'I') {
                if ($index->heads || count($f) < 6 || count($f) > 8) {
                    return null;
                }
                $key = self::unescape($f[1]);
                $path = self::unescape($f[4]);
                if ($key === null || $key === '' || $path === null || $path === '' || isset($index->rows[$key])
                    || !preg_match('/^\d+$/', $f[2]) || !preg_match('/^[0-9A-F]{8}$/', $f[3]) || !preg_match('/^\d+$/', $f[5])) {
                    return null;
                }
                $sha = null;
                $level = null;
                foreach (array_slice($f, 6) as $extra) {
                    if ($sha === null && $level === null && preg_match('/^[0-9A-F]{64}$/', $extra)) {
                        $sha = $extra;
                    } elseif ($level === null && in_array($extra, self::LEVELS, true)) {
                        $level = $extra;
                    } else {
                        return null;
                    }
                }
                $index->rows[$key] = ['size' => (int)$f[2], 'crc' => $f[3], 'path' => $path, 'time' => (int)$f[5], 'sha' => $sha, 'level' => $level];
                continue;
            }
            if ($f[0] === 'H' && count($f) === 4) {
                $key = self::unescape($f[1]);
                $name = self::unescape($f[2]);
                $value = self::unescape($f[3]);
                if ($key === null || !isset($index->rows[$key]) || $name === null || !preg_match('/^[!#$%&\'*+.^_`|~0-9A-Za-z-]+$/', $name)
                    || $value === null || preg_match('/[\x00\r\n]/', $value)) {
                    return null;
                }
                $index->heads[] = [$key, $name, $value];
                continue;
            }
            return null;
```

```
    }
    return $index;
}

public static function create(string $file, string $rootText): self
{
    $index = new self($file, $rootText);
    $index->dirty = true;
    return $index;
}

public function scan(callable $take, bool $sha): void
{
    $base = rtrim(str_replace('\\', '/', realpath($this->root)), '/') . '/';
    $found = [];
    $it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($this->root, FilesystemIterator::SKIP_DOTS));
    foreach ($it as $info) {
        if (!$info->isFile()) {
            continue;
        }
        $rel = substr(str_replace('\\', '/', realpath($info->getPathname())), strlen($base));
        if (!$take($rel)) {
            continue;
        }
        $key = '/' . $rel;
        $found[$key] = true;
        if (!isset($this->rows[$key])) {
            $this->rows[$key] = ['size' => -1, 'crc' => '00000000', 'path' => $rel, 'time' => 0, 'sha' => null, 'level' => null];
            $this->dirty = true;
        }
    }
    foreach (array_keys($this->rows) as $key) {
        $key = (string)$key;
        if (!isset($found[$key])) {
            unset($this->rows[$key]);
            $this->dirty = true;
        } else {
            $this->fresh($key, $sha);
        }
    }
    $kept = array_values(array_filter($this->heads, fn($h) => isset($this->rows[$h[0]])));
    if (count($kept) !== count($this->heads)) {
        $this->heads = $kept;
        $this->dirty = true;
    }
}

public static function rooted(string $key): string
{
    return str_starts_with($key, '/') ? $key : '/' . $key;
}

public function put(string $key, string $path): bool
{
    if ($key === '' || $path === '' || !is_file($this->root . '/' . $path)) {
        return false;
    }
    $key = self::rooted($key);
    if (!isset($this->rows[$key]) || $this->rows[$key]['path'] !== $path) {
        $this->rows[$key] = ['size' => -1, 'crc' => '00000000', 'path' => $path, 'time' => 0, 'sha' => null, 'level' => null];
        $this->dirty = true;
    }
    return true;
}

public function has(string $key): bool
{
    return isset($this->rows[$key]);
}

public function keys(): array
{
    return array_map('strval', array_keys($this->rows));
}

public function path(string $key): string
{
    return $this->root . '/' . $this->rows[$key]['path'];
}

public function fresh(string $key, bool $sha = false): ?array
{
    if (!isset($this->rows[$key])) {
        return null;
    }
    $row = $this->rows[$key];
    $path = $this->path($key);
    clearstatcache(true, $path);
    if (!is_file($path)) {
        return null;
    }
    $size = filesize($path);
    $time = self::fileTime($path);
    $changed = $size !== $row['size'] || intdiv($time, 10000000) !== intdiv($row['time'], 10000000);
    if ($changed) {
        $row['size'] = $size;
        $row['time'] = $time;
        $row['crc'] = self::crcOf($path);
    }
    if ($sha && ($changed || $row['sha'] === null)) {
        $row['sha'] = self::shaOf($path);
    }
    if ($row !== $this->rows[$key]) {
        $this->rows[$key] = $row;
        $this->dirty = true;
    }
    return $row;
}

public function level(string $key): string
{
    return $this->rows[$key]['level'] ?? 'Ù';
}

public function setLevel(string $key, ?string $level): void
{
    if (isset($this->rows[$key]) && ($level === null || in_array($level, self::LEVELS, true)) && $this->rows[$key]['level'] !== $level) {
        $this->rows[$key]['level'] = $level;
        $this->dirty = true;
    }
}

public function heads(string $key): array
{
    return array_values(array_filter($this->heads, fn($h) => $h[0] === $key));
}

public function setHeads(string $key, array $pairs): void
{
    $others = array_values(array_filter($this->heads, fn($h) => $h[0] !== $key));
    $mine = array_map(fn($p) => [$key, $p[0], $p[1]], $pairs);
    if ($mine !== $this->heads($key)) {
        $this->heads = array_merge($others, $mine);
        $this->dirty = true;
    }
}

private static function order(array $keys): array
{
    $keys = array_map('strval', $keys);
    usort($keys, [V31nOrder::class, 'key']);
    return $keys;
}

private function headLines(array $keys): string
{
    $out = '';
    foreach ($keys as $key) {
        foreach ($this->heads($key) as [, $name, $value]) {
            $out .= implode("\t", ['H', self::escape($key), self::escape($name), self::escape($value)]) . "\r\n";
        }
    }
    return $out;
}

public function save(): bool
{
    if (!$this->dirty) {
        return true;
    }
    $keys = self::order($this->keys());
    $out = self::MAGIC . "\r\n" . self::escape($this->rootText) . "\r\n";
    foreach ($keys as $key) {
```

```
            $r = $this->rows[$key];
            $f = ['I', self::escape($key), $r['size'], $r['crc'], self::escape($r['path']), $r['time']];
            if ($r['sha'] !== null) {
                $f[] = $r['sha'];
            }
            if (($r['level'] ?? null) !== null) {
                $f[] = $r['level'];
            }
            $out .= implode("\t", $f) . "\r\n";
        }
        $out .= $this->headLines($keys);
        $temp = $this->file . '.' . bin2hex(random_bytes(16)) . '.tmp';
        if (file_put_contents($temp, $out) !== strlen($out) || !rename($temp, $this->file)) {
            @unlink($temp);
            return false;
        }
        $this->dirty = false;
        return true;
    }

    public function entry(array $rows, ?string $secret): string
    {
        $keys = self::order(array_keys($rows));
        $out = self::MAGIC . "\r\n";
        foreach ($keys as $key) {
            $r = $rows[$key];
            $f = ['I', self::escape($key), $r['size'], $r['crc']];
            if ($secret !== null) {
                $f[] = $r['sha'];
            }
            if (($r['level'] ?? null) !== null) {
                $f[] = $r['level'];
            }
            $out .= implode("\t", $f) . "\r\n";
        }
        $out .= $this->headLines($keys);
        if ($secret !== null) {
            $out .= "S\t" . strtoupper(bin2hex(sodium_crypto_sign_detached($out, $secret))) . "\r\n";
        }
        return $out;
    }
}
```

3 - File: lib/order.js

Size

5369

SHA-256

1CBB1DF63C5CE1FE1CBEEB2E29768DD336373880B59C92D5F457EC519F016179

Code

```
const V31nOrder = (tables => {
    const ranges = tables.ranges;
    const slots = new Map(tables.slots.map(([c, rank, place]) => [c, [rank, place]]));

    const range = c => {
        let lo = 0, hi = ranges.length - 1, hit = -1;
        while (lo <= hi) {
            const mid = (lo + hi) >> 1;
            if (ranges[mid][0] <= c) { hit = mid; lo = mid + 1; } else hi = mid - 1;
        }
        return hit >= 0 && c <= ranges[hit][1] ? ranges[hit] : null;
    };
    const space = c => { const r = range(c); return r !== null && r[2]; };
    const loose = c => range(c) !== null;
    const points = s => Array.from(s, ch => ch.codePointAt(0));
    const text = p => String.fromCodePoint(...p);
    const shifts = tables.lower;
    const lowerCase = c => {
        let lo = 0, hi = shifts.length - 1, hit = -1;
        while (lo <= hi) {
            const mid = (lo + hi) >> 1;
            if (shifts[mid][0] <= c) { hit = mid; lo = mid + 1; } else hi = mid - 1;
        }
        if (hit < 0) return c;
        const [first, last, step, delta] = shifts[hit];
        return c > last || (c - first) % step !== 0 ? c : c + delta;
    };

    const fold = name => {
        let t = points(name);
        let i = 0;
        while (i < t.length && space(t[i])) i++;
        if (i < t.length && t[i] === 0x5B) {
            const close = t.indexOf(0x5D, i + 1);
            if (close !== -1 && close > i + 1) t = t.slice(i + 1, close);
        }
        i = 0;
        while (i < t.length && loose(t[i])) i++;
        t = t.slice(i);

        const d = points(text(t).normalize('NFD'));
        const doubled = [];
        for (let k = 0; k < d.length; k++) {
            if (d[k] !== 0x0A && k + 1 < d.length && d[k + 1] === 0x308) {
                doubled.push(d[k], d[k]);
                k++;
            } else doubled.push(d[k]);
        }
        t = points(text(doubled).normalize('NFC'));

        const numbered = [];
        for (let k = 0; k < t.length;) {
            if (t[k] === 0x2116) {
                let j = k + 1;
                while (j < t.length && space(t[j])) j++;
                const start = j;
                while (j < t.length && t[j] >= 0x30 && t[j] <= 0x39) j++;
                if (j > start) {
                    let lead = start;
                    while (lead + 1 < j && t[lead] === 0x30) lead++;
                    const digits = t.slice(lead, j);
                    numbered.push(0x2116);
                    for (let p = digits.length; p < 5; p++) numbered.push(0x30);
                    numbered.push(...digits);
                    k = j;
                    continue;
                }
            }
            numbered.push(t[k]);
            k++;
        }
        return numbered.map(lowerCase);
    };

    const stem = name => {
        const dot = name.lastIndexOf('.');
        return dot === -1 || dot === 0 ? name : name.slice(0, dot);
    };

    const rank = (s1, s2) => {
        const min = Math.min(s1.length, s2.length);
        for (let i = 0; i < min; i++) {
            const c1 = s1[i], c2 = s2[i];
            if (c1 === c2) continue;
            const r1 = slots.get(c1), r2 = slots.get(c2);
            if (r1 && r2) {
                if (r1[0] !== r2[0]) return r1[0] < r2[0] ? -1 : 1;
            } else if (r1) return -1;
            else if (r2) return 1;
            else return c1 < c2 ? -1 : 1;
        }
        return s1.length === s2.length ? 0 : s1.length < s2.length ? -1 : 1;
    };

    const place = (s1, s2) => {
        const min = Math.min(s1.length, s2.length);
        for (let i = 0; i < min; i++) {
            if (s1[i] === s2[i]) continue;
            const r1 = slots.get(s1[i]), r2 = slots.get(s2[i]);
            if (r1 && r2 && r1[1] !== r2[1]) return r1[1] < r2[1] ? -1 : 1;
            return 0;
        }
        return 0;
    };

    const capital = c => lowerCase(c) !== c;

    const cases = (s1, s2) => {
        const min = Math.min(s1.length, s2.length);
        for (let i = 0; i < min; i++) {
            const c1 = capital(s1[i]), c2 = capital(s2[i]);
            if (c1 !== c2) return c1 ? -1 : 1;
        }
        return 0;
    };

    const name = (a, b) => {
        const stemA = fold(stem(a)), stemB = fold(stem(b));
        let c = rank(stemA, stemB);
        if (c !== 0) return c;
        const foldA = fold(a), foldB = fold(b);
        c = rank(foldA, foldB);
        if (c !== 0) return c;
        c = place(stemA, stemB);
        if (c !== 0) return c;
        c = place(foldA, foldB);
        if (c !== 0) return c;
        const pa = points(a), pb = points(b);
        c = cases(pa, pb);
        if (c !== 0) return c;
        if (pa.length !== pb.length) return pa.length < pb.length ? -1 : 1;
        for (let i = 0; i < pa.length; i++) if (pa[i] !== pb[i]) return pa[i] < pb[i] ? -1 : 1;
        return 0;
    };

    const key = (a, b) => {
        const pa = a.split('/'), pb = b.split('/');
        const n = Math.min(pa.length, pb.length);
        for (let i = 0; i < n; i++) {
            const folderA = i < pa.length - 1, folderB = i < pb.length - 1;
            if (folderA !== folderB) return folderA ? -1 : 1;
            const c = name(pa[i], pb[i]);
```

```
        if (c !== 0) return c;
      }
      return pa.length === pb.length ? 0 : pa.length < pb.length ? -1 : 1;
    };

    return { name, key };
  })(V31N_TABLES);
```

4 - File: lib/order.php

Size *8038*

SHA-256 *9FEB5DBE2F534E5A810C64374854500B7654DB7A46411091D5AEE68B1AD3EDD2*

Code

```
<?php
final class V31nOrder
{
    private static ?array $ranges = null;
    private static ?array $slots = null;
    private static ?array $shifts = null;

    private static function load(): void
    {
        if (self::$slots !== null) {
            return;
        }
        $t = json_decode(file_get_contents(__DIR__ . '/tables.json'), true);
        self::$ranges = $t['ranges'];
        self::$shifts = $t['lower'];
        self::$slots = [];
        foreach ($t['slots'] as [$c, $rank, $place]) {
            self::$slots[$c] = [$rank, $place];
        }
    }

    private static function range(int $c): ?array
    {
        $lo = 0;
        $hi = count(self::$ranges) - 1;
        $hit = null;
        while ($lo <= $hi) {
            $mid = intdiv($lo + $hi, 2);
            if (self::$ranges[$mid][0] <= $c) {
                $hit = $mid;
                $lo = $mid + 1;
            } else {
                $hi = $mid - 1;
            }
        }
        return $hit !== null && $c <= self::$ranges[$hit][1] ? self::$ranges[$hit] : null;
    }

    private static function space(int $c): bool
    {
        $r = self::range($c);
        return $r !== null && $r[2];
    }

    private static function loose(int $c): bool
    {
        return self::range($c) !== null;
    }

    private static function points(string $s): array
    {
        return $s === '' ? [] : array_map('mb_ord', mb_str_split($s, 1, 'UTF-8'));
    }

    private static function text(array $p): string
    {
        return implode('', array_map('mb_chr', $p));
    }

    private static function normal(string $s, int $form): string
    {
        if ($s === '') {
            return $s;
        }
        $n = Normalizer::normalize($s, $form);
        return $n === false ? $s : $n;
    }

    private static function lowerCase(int $c): int
    {
        $lo = 0;
        $hi = count(self::$shifts) - 1;
        $hit = null;
        while ($lo <= $hi) {
            $mid = intdiv($lo + $hi, 2);
            if (self::$shifts[$mid][0] <= $c) {
                $hit = $mid;
                $lo = $mid + 1;
            } else {
                $hi = $mid - 1;
            }
        }
        if ($hit === null) {
            return $c;
        }
        [$first, $last, $step, $delta] = self::$shifts[$hit];
        return $c > $last || ($c - $first) % $step !== 0 ? $c : $c + $delta;
    }

    private static function fold(string $name): array
    {
        $t = self::points($name);
        $i = 0;
        while ($i < count($t) && self::space($t[$i])) {
            $i++;
        }
        if ($i < count($t) && $t[$i] === 0x5B) {
            $close = array_search(0x5D, array_slice($t, $i + 1), true);
            if ($close !== false && $close > 0) {
                $t = array_slice($t, $i + 1, $close);
            }
        }
        $i = 0;
        while ($i < count($t) && self::loose($t[$i])) {
            $i++;
        }
        $t = array_slice($t, $i);

        $d = self::points(self::normal(self::text($t), Normalizer::FORM_D));
        $doubled = [];
        for ($k = 0; $k < count($d); $k++) {
            if ($d[$k] !== 0x0A && $k + 1 < count($d) && $d[$k + 1] === 0x308) {
                $doubled[] = $d[$k];
                $doubled[] = $d[$k];
                $k++;
            } else {
                $doubled[] = $d[$k];
            }
        }
        $t = self::points(self::normal(self::text($doubled), Normalizer::FORM_C));

        $numbered = [];
        for ($k = 0; $k < count($t);) {
            if ($t[$k] === 0x2116) {
                $j = $k + 1;
                while ($j < count($t) && self::space($t[$j])) {
                    $j++;
                }
                $start = $j;
                while ($j < count($t) && $t[$j] >= 0x30 && $t[$j] <= 0x39) {
                    $j++;
                }
                if ($j > $start) {
                    $lead = $start;
                    while ($lead + 1 < $j && $t[$lead] === 0x30) {
                        $lead++;
                    }
                    $digits = array_slice($t, $lead, $j - $lead);
                    $numbered[] = 0x2116;
                    for ($p = count($digits); $p < 5; $p++) {
                        $numbered[] = 0x30;
                    }
```

```
                array_push($numbered, ...$digits);
                $k = $j;
                continue;
            }
        }
        $numbered[] = $t[$k];
        $k++;
    }

    return array_map([self::class, 'lowerCase'], $numbered);
}

private static function stem(string $name): string
{
    $dot = strrpos($name, '.');
    return $dot === false || $dot === 0 ? $name : substr($name, 0, $dot);
}

private static function rank(array $s1, array $s2): int
{
    $min = min(count($s1), count($s2));
    for ($i = 0; $i < $min; $i++) {
        $c1 = $s1[$i];
        $c2 = $s2[$i];
        if ($c1 === $c2) {
            continue;
        }
        $r1 = self::$slots[$c1] ?? null;
        $r2 = self::$slots[$c2] ?? null;
        if ($r1 !== null && $r2 !== null) {
            if ($r1[0] !== $r2[0]) {
                return $r1[0] < $r2[0] ? -1 : 1;
            }
        } elseif ($r1 !== null) {
            return -1;
        } elseif ($r2 !== null) {
            return 1;
        } else {
            return $c1 < $c2 ? -1 : 1;
        }
    }
    return count($s1) <=> count($s2);
}

private static function place(array $s1, array $s2): int
{
    $min = min(count($s1), count($s2));
    for ($i = 0; $i < $min; $i++) {
        if ($s1[$i] === $s2[$i]) {
            continue;
        }
        $r1 = self::$slots[$s1[$i]] ?? null;
        $r2 = self::$slots[$s2[$i]] ?? null;
        if ($r1 !== null && $r2 !== null && $r1[1] !== $r2[1]) {
            return $r1[1] < $r2[1] ? -1 : 1;
        }
        return 0;
    }
    return 0;
}

private static function capital(int $c): bool
{
    return self::lowerCase($c) !== $c;
}

private static function cases(array $s1, array $s2): int
{
    $min = min(count($s1), count($s2));
    for ($i = 0; $i < $min; $i++) {
        $c1 = self::capital($s1[$i]);
        $c2 = self::capital($s2[$i]);
        if ($c1 !== $c2) {
            return $c1 ? -1 : 1;
        }
    }
    return 0;
}

public static function name(string $a, string $b): int
{
    self::load();
    $stemA = self::fold(self::stem($a));
    $stemB = self::fold(self::stem($b));
    $c = self::rank($stemA, $stemB);
    if ($c !== 0) {
        return $c;
    }
    $foldA = self::fold($a);
    $foldB = self::fold($b);
    $c = self::rank($foldA, $foldB);
    if ($c !== 0) {
        return $c;
    }
    $c = self::place($stemA, $stemB);
    if ($c !== 0) {
        return $c;
    }
    $c = self::place($foldA, $foldB);
    if ($c !== 0) {
        return $c;
    }
    $pointsA = self::points($a);
    $pointsB = self::points($b);
    $c = self::cases($pointsA, $pointsB);
    if ($c !== 0) {
        return $c;
    }
    if (count($pointsA) !== count($pointsB)) {
        return count($pointsA) < count($pointsB) ? -1 : 1;
    }
    return $pointsA <=> $pointsB;
}

public static function key(string $a, string $b): int
{
    $pa = explode('/', $a);
    $pb = explode('/', $b);
    $n = min(count($pa), count($pb));
    for ($i = 0; $i < $n; $i++) {
        $folderA = $i < count($pa) - 1;
        $folderB = $i < count($pb) - 1;
        if ($folderA !== $folderB) {
            return $folderA ? -1 : 1;
        }
        $c = self::name($pa[$i], $pb[$i]);
        if ($c !== 0) {
            return $c;
        }
    }
    return count($pa) <=> count($pb);
}
}
```

5 - File: lib/pack.js

Size

8730

SHA-256

05217873AD44B7601A8B947E2473E200858FC220BCFC8A7E643957DF6EA05000

Code

```
const V31nPack = (() => {
    const enc = new TextEncoder();
    const dec = new TextDecoder('utf-8', { fatal: true });
    const bytes = s => enc.encode(s);
    const MAGIC = bytes('V31NSPACKFORMAT');
    const ASK = bytes('V31NPIELALAIMIT');
    const PATCH = new Uint8Array([0x56, 0x33, 0x31, 0x4E, 0x32, 0x36, 0x37, 0, 0, 0, 0, 0, 0, 0]);
    const PI = new Uint8Array([0x24, 0x3F, 0x6A, 0x88, 0x85, 0xA3, 0x08, 0xD3, 0x13, 0x19, 0x8A, 0x2E]);
    const PHI = new Uint8Array([0x9E, 0x37, 0x79, 0xB9, 0x7F, 0x4A, 0x7C, 0x15, 0xF3, 0x9C, 0xC0, 0x60]);
    const MARK = 12, LEAD = 27, FOOT = 16, LIMIT = 4503599627370496;

    const table = new Uint32Array(256);
    for (let n = 0; n < 256; n++) {
        let c = n;
        for (let k = 0; k < 8; k++) c = (c & 1) ? (0xEDB88320 ^ (c >>> 1)) : (c >>> 1);
        table[n] = c >>> 0;
    }
    const crc = (data, prev = 0) => {
        let c = (prev ^ 0xFFFFFFFF) >>> 0;
        for (let i = 0; i < data.length; i++) c = table[(c ^ data[i]) & 0xFF] ^ (c >>> 8);
        return (c ^ 0xFFFFFFFF) >>> 0;
    };
    const hex = v => v.toString(16).toUpperCase().padStart(8, '0');
    const times = (mat, vec) => {
        let sum = 0;
        for (let i = 0; vec !== 0; i++, vec >>>= 1) if (vec & 1) sum = (sum ^ mat[i]) >>> 0;
        return sum;
    };
    const square = mat => mat.map(v => times(mat, v));
    const combine = (crc1, crc2, len2) => {
        if (len2 <= 0) return crc1;
        let odd = [0xEDB88320];
        for (let n = 1, row = 1; n < 32; n++, row = (row << 1) >>> 0) odd[n] = row;
        let even = square(odd);
        odd = square(even);
        for (;;) {
            even = square(odd);
            if (len2 % 2) crc1 = times(even, crc1);
            len2 = Math.floor(len2 / 2);
            if (len2 === 0) break;
            odd = square(even);
            if (len2 % 2) crc1 = times(odd, crc1);
            len2 = Math.floor(len2 / 2);
            if (len2 === 0) break;
        }
        return (crc1 ^ crc2) >>> 0;
    };

    const join = parts => {
        let n = 0;
        for (const p of parts) n += p.length;
        const out = new Uint8Array(n);
        let at = 0;
        for (const p of parts) { out.set(p, at); at += p.length; }
        return out;
    };
    const num = v => {
        const out = [];
        do { out.push(v % 256); v = Math.floor(v / 256); } while (v > 0);
        return new Uint8Array(out);
    };
    const sep = marker => join([new Uint8Array([marker]), PHI]);
    const split = key => {
        const slash = key.lastIndexOf('/');
        const base = slash === -1 ? 0 : slash + 1;
        const dot = key.lastIndexOf('.');
        const ext = dot === -1 || dot < base ? key.length : dot;
        return [key.slice(0, base), key.slice(base, ext), key.slice(ext)];
    };

    const head = (magic, items) => {
        const t2 = [];
        for (const { key, size } of items) {
            t2.push(sep(0), bytes(key), sep(1), num(size));
            for (const part of split(key)) t2.push(sep(1), bytes(part));
        }
        const t2b = join(t2);
        let start = 0, previous, header;
        do {
            previous = start;
            const parts = [magic, PI];
            let address = previous;
            for (const { key, size } of items) {
                parts.push(sep(0), bytes(key), sep(1), num(address));
                address += size;
            }
            parts.push(PI, t2b, PI);
            header = join(parts);
            start = header.length;
        } while (start !== previous);
        return header;
    };

    const archiveCrc = (magic, files) => {
        const header = head(magic, files.map(f => ({ key: f.key, size: f.bytes.length })));
        let sum = crc(header);
        for (const f of files) sum = crc(f.bytes, sum);
        return { header, crc: crc(PI, sum) };
    };

    const write = (magic, files) => {
        const { header, crc: c } = archiveCrc(magic, files);
        const tail = new Uint8Array(FOOT);
        tail.set(PI);
        new DataView(tail.buffer).setUint32(MARK, c, true);
        return { buf: join([header, ...files.map(f => f.bytes), tail]), crc: c };
    };

    const find = (buf, pat, from, to) => {
        if (to > buf.length) to = buf.length;
        outer: for (let i = from; i + pat.length <= to; i++) {
            for (let k = 0; k < pat.length; k++) if (buf[i + k] !== pat[k]) continue outer;
            return i;
        }
        return -1;
    };
    const same = (buf, at, pat) => {
        if (at + pat.length > buf.length) return false;
        for (let k = 0; k < pat.length; k++) if (buf[at + k] !== pat[k]) return false;
        return true;
    };

    const rows = (buf, from, to) => {
        const out = [];
        let open = false, at = from;
        for (;;) {
            const hit = find(buf, PHI, at, to);
            if (hit >= 0 && hit <= at) return null;
            const end = hit < 0 ? to : hit - 1;
            if (open) out[out.length - 1].push(buf.subarray(at, end));
            else if (end !== from) return null;
            if (hit < 0) return out;
            const marker = buf[hit - 1];
            if (marker === 0) { out.push([]); open = true; }
            else if (marker !== 1 || !open) return null;
            at = hit + MARK;
        }
    };
    const value = b => {
        if (b.length === 0 || b.length > 7) return null;
        let v = 0;
        for (let i = b.length - 1; i >= 0; i--) v = v * 256 + b[i];
```

```
    return v < LIMIT ? v : null;
  };
  const text = b => { try { return dec.decode(b); } catch { return null; } };

  const parse = (magic, buf) => {
    const total = buf.length;
    if (total < LEAD + 2 * MARK + FOOT) return null;
    if (!same(buf, 0, magic) || !same(buf, 15, PI)) return null;
    const body = total - FOOT;
    const b = find(buf, PI, LEAD, body);
    if (b < 0) return null;
    const c = find(buf, PI, b + MARK, body);
    if (c < 0) return null;
    const t1 = rows(buf, LEAD, b), t2 = rows(buf, b + MARK, c);
    if (!t1 || !t2 || t1.length !== t2.length) return null;
    const sizes = new Map();
    for (const r of t2) {
      if (r.length !== 5) return null;
      const key = text(r[0]), size = value(r[1]);
      if (key === null || size === null || sizes.has(key)) return null;
      const parts = split(key);
      for (let k = 0; k < 3; k++) if (text(r[k + 2]) !== parts[k]) return null;
      sizes.set(key, size);
    }
    let next = c + MARK;
    const entries = [];
    for (const r of t1) {
      if (r.length !== 2) return null;
      const key = text(r[0]), at = value(r[1]);
      if (key === null || at === null || at !== next || !sizes.has(key) || sizes.get(key) > body - next) return null;
      entries.push({ key, at, size: sizes.get(key) });
      next += sizes.get(key);
      sizes.delete(key);
    }
    if (next !== body || !same(buf, body, PI)) return null;
    const want = new DataView(buf.buffer, buf.byteOffset + body + MARK, 4).getUint32(0, true);
    if (crc(buf.subarray(0, body + MARK)) !== want) return null;
    return { entries, crc: want };
  };

  const headOf = (magic, buf) => {
    if (buf.length < LEAD) return 'more';
    if (!same(buf, 0, magic) || !same(buf, 15, PI)) return null;
    const b = find(buf, PI, LEAD, buf.length);
    if (b < 0) return 'more';
    const c = find(buf, PI, b + MARK, buf.length);
    if (c < 0) return 'more';
    const t1 = rows(buf, LEAD, b), t2 = rows(buf, b + MARK, c);
    if (!t1 || !t2 || t1.length !== t2.length) return null;
    const sizes = new Map();
    for (const r of t2) {
      if (r.length !== 5) return null;
      const key = text(r[0]), size = value(r[1]);
      if (key === null || size === null || sizes.has(key)) return null;
      const parts = split(key);
      for (let k = 0; k < 3; k++) if (text(r[k + 2]) !== parts[k]) return null;
      sizes.set(key, size);
    }
    let next = c + MARK;
    const entries = [];
    for (const r of t1) {
      if (r.length !== 2) return null;
      const key = text(r[0]), at = value(r[1]);
      if (key === null || at === null || at !== next || !sizes.has(key)) return null;
      entries.push({ key, at, size: sizes.get(key) });
      next += sizes.get(key);
      sizes.delete(key);
    }
    return { entries, length: c + MARK, end: next };
  };

  return { MAGIC, ASK, PATCH, PI, FOOT, MARK, crc, hex, combine, bytes, join, head, write, archiveCrc, parse, headOf };
})();
```

6 - File: lib/pack.php

Size

7203

SHA-256

633BF8E585FE8D861BC446E460A2F8BFEDE82BE9E9EDF2CBBBEDEC378231BEFC

Code

```
<?php
require_once __DIR__ . '/order.php';

final class V31nPack
{
    const MAGIC = 'V31NSPACKFORMAT';
    const ASK = 'V31NPIELALAIMIT';
    const PATCH = "V31N267\0\0\0\0\0\0\0\0";
    const PI = "\x24\x3F\x6A\x88\x85\xA3\x08\xD3\x13\x19\x8A\x2E";
    const PHI = "\x9E\x37\x79\xB9\x7F\x4A\x7C\x15\xF3\x9C\xC0\x60";
    const MARK = 12;
    const LEAD = 27;
    const FOOT = 16;
    const LIMIT = 4503599627370496;

    public static function hex(int $crc): string
    {
        return sprintf('%08X', $crc);
    }

    private static function times(array $mat, int $vec): int
    {
        $sum = 0;
        for ($i = 0; $vec !== 0; $i++, $vec >>= 1) {
            if ($vec & 1) {
                $sum ^= $mat[$i];
            }
        }
        return $sum;
    }

    private static function square(array $mat): array
    {
        $out = [];
        for ($n = 0; $n < 32; $n++) {
            $out[$n] = self::times($mat, $mat[$n]);
        }
        return $out;
    }

    public static function combine(int $crc1, int $crc2, int $len2): int
    {
        if ($len2 <= 0) {
            return $crc1;
        }
        $odd = [0xEDB88320];
        for ($n = 1, $row = 1; $n < 32; $n++, $row <<= 1) {
            $odd[$n] = $row;
        }
        $even = self::square($odd);
        $odd = self::square($even);
        do {
            $even = self::square($odd);
            if ($len2 & 1) {
                $crc1 = self::times($even, $crc1);
            }
            $len2 >>= 1;
            if ($len2 === 0) {
                break;
            }
            $odd = self::square($even);
            if ($len2 & 1) {
                $crc1 = self::times($odd, $crc1);
            }
            $len2 >>= 1;
        } while ($len2 !== 0);
        return ($crc1 ^ $crc2) & 0xFFFFFFFF;
    }

    private static function num(int $v): string
    {
        $out = '';
        do {
            $out .= chr($v & 0xFF);
            $v >>= 8;
        } while ($v > 0);
        return $out;
    }

    private static function sep(int $marker): string
    {
        return chr($marker) . self::PHI;
    }

    private static function split(string $key): array
    {
        $slash = strrpos($key, '/');
        $base = $slash === false ? 0 : $slash + 1;
        $dot = strrpos($key, '.');
        $ext = $dot === false || $dot < $base ? strlen($key) : $dot;
        return [substr($key, 0, $base), substr($key, $base, $ext - $base), substr($key, $ext)];
    }

    public static function head(string $magic, array $items): string
    {
        $t2 = '';
        foreach ($items as [$key, $size]) {
            $t2 .= self::sep(0) . $key . self::sep(1) . self::num($size);
            foreach (self::split($key) as $part) {
                $t2 .= self::sep(1) . $part;
            }
        }
        $start = 0;
        do {
            $previous = $start;
            $header = $magic . self::PI;
            $address = $previous;
            foreach ($items as [$key, $size]) {
                $header .= self::sep(0) . $key . self::sep(1) . self::num($address);
                $address += $size;
            }
            $header .= self::PI . $t2 . self::PI;
            $start = strlen($header);
        } while ($start !== $previous);
        return $header;
    }

    public static function write(string $magic, array $files): string
    {
        $items = [];
        foreach ($files as [$key, $bytes]) {
            $items[] = [$key, strlen($bytes)];
        }
        $out = self::head($magic, $items);
        foreach ($files as [, $bytes]) {
            $out .= $bytes;
        }
        $out .= self::PI;
        return $out . pack('V', crc32($out));
    }

    private static function rows(string $buf, int $from, int $to): ?array
    {
        $out = [];
        $open = false;
        $at = $from;
        for (;;) {
            $hit = $at < $to ? strpos($buf, self::PHI, $at) : false;
            if ($hit !== false && $hit + self::MARK > $to) {
                $hit = false;
            }
            if ($hit !== false && $hit <= $at) {
```

```
            return null;
        }
        $end = $hit === false ? $to : $hit - 1;
        if ($open) {
            $out[count($out) - 1][] = substr($buf, $at, $end - $at);
        } elseif ($end !== $from) {
            return null;
        }
        if ($hit === false) {
            return $out;
        }
        $marker = ord($buf[$hit - 1]);
        if ($marker === 0) {
            $out[] = [];
            $open = true;
        } elseif ($marker !== 1 || !$open) {
            return null;
        }
        $at = $hit + self::MARK;
    }
}

private static function value(string $b): ?int
{
    $n = strlen($b);
    if ($n === 0 || $n > 7) {
        return null;
    }
    $v = 0;
    for ($i = $n - 1; $i >= 0; $i--) {
        $v = ($v << 8) | ord($b[$i]);
    }
    return $v < self::LIMIT ? $v : null;
}

public static function parse(string $magic, string $buf): ?array
{
    $total = strlen($buf);
    if ($total < self::LEAD + 2 * self::MARK + self::FOOT) {
        return null;
    }
    if (substr($buf, 0, 15) !== $magic || substr($buf, 15, self::MARK) !== self::PI) {
        return null;
    }
    $body = $total - self::FOOT;
    $b = strpos($buf, self::PI, self::LEAD);
    if ($b === false || $b + self::MARK > $body) {
        return null;
    }
    $c = strpos($buf, self::PI, $b + self::MARK);
    if ($c === false || $c + self::MARK > $body) {
        return null;
    }
    $t1 = self::rows($buf, self::LEAD, $b);
    $t2 = self::rows($buf, $b + self::MARK, $c);
    if ($t1 === null || $t2 === null || count($t1) !== count($t2)) {
        return null;
    }
    $sizes = [];
    foreach ($t2 as $r) {
        if (count($r) !== 5) {
            return null;
        }
        $key = $r[0];
        $size = self::value($r[1]);
        if (!mb_check_encoding($key, 'UTF-8') || $size === null || array_key_exists($key, $sizes)) {
            return null;
        }
        if ([$r[2], $r[3], $r[4]] !== self::split($key)) {
            return null;
        }
        $sizes[$key] = $size;
    }
    $next = $c + self::MARK;
    $entries = [];
    foreach ($t1 as $r) {
        if (count($r) !== 2) {
            return null;
        }
        $at = self::value($r[1]);
        if ($at === null || $at !== $next || !array_key_exists($r[0], $sizes) || $sizes[$r[0]] > $body - $next) {
            return null;
        }
        $entries[] = [$r[0], $at, $sizes[$r[0]]];
        $next += $sizes[$r[0]];
        unset($sizes[$r[0]]);
    }
    if ($next !== $body || substr($buf, $body, self::MARK) !== self::PI) {
        return null;
    }
    $crc = unpack('V', substr($buf, $body + self::MARK, 4))[1];
    if (crc32(substr($buf, 0, $body + self::MARK)) !== $crc) {
        return null;
    }
    return ['entries' => $entries, 'crc' => $crc];
}
}
```

7 - File: lib/ranges.php

Size

3150

SHA-256

2E60576BD05B5B8455D4CE2399F3A8196BC5949BF41A072ABDB60108D6CBB631

Code

```
<?php

function v31np_ranges(string $file, int $base, int $total, string $type, string $etag, array $heads = []): void
{
    $method = $_SERVER['REQUEST_METHOD'];
    header('ETag: "' . $etag . '"');
    foreach ($heads as [$n, $v]) {
        header($n . ': ' . $v, false);
    }
    $send = function (int $start, int $end) use ($file, $base): void {
        $h = fopen($file, 'rb');
        fseek($h, $base + $start);
        $left = $end - $start + 1;
        while ($left > 0) {
            $chunk = fread($h, min(1048576, $left));
            if ($chunk === false || $chunk === '') {
                break;
            }
            echo $chunk;
            $left -= strlen($chunk);
            flush();
        }
        fclose($h);
    };
    $range = trim($_SERVER['HTTP_RANGE'] ?? '');
    $ifRange = trim($_SERVER['HTTP_IF_RANGE'] ?? '');
    if ($range === '' || ($ifRange !== '' && $ifRange !== '"' . $etag . '"')) {
        header('Content-Type: ' . $type);
        header('Accept-Ranges: bytes');
        header('Content-Length: ' . $total);
        if ($method === 'GET' && $total > 0) {
            $send(0, $total - 1);
        }
        exit;
    }
    if (!str_starts_with($range, 'bytes=')) {
        http_response_code(400);
        exit;
    }
    $parts = [];
    foreach (explode(',', substr($range, 6)) as $spec) {
        if (!preg_match('/^(\d*)-(\d*)$/', trim($spec), $m) || ($m[1] === '' && $m[2] === '')) {
            http_response_code(400);
            exit;
        }
        if ($m[1] === '') {
            $start = max(0, $total - (int)$m[2]);
            $end = $total - 1;
        } else {
            $start = (int)$m[1];
            $end = $m[2] === '' ? $total - 1 : min((int)$m[2], $total - 1);
        }
        if ($start < $total && $start <= $end && ($m[1] !== '' || (int)$m[2] > 0)) {
            $parts[] = [$start, $end];
        }
    }
    if (!$parts) {
        http_response_code(416);
        header('Content-Range: bytes */' . $total);
        exit;
    }
    http_response_code(206);
    header('Accept-Ranges: bytes');
    if (count($parts) === 1) {
        [$start, $end] = $parts[0];
        header('Content-Type: ' . $type);
        header('Content-Range: bytes ' . $start . '-' . $end . '/' . $total);
        header('Content-Length: ' . ($end - $start + 1));
        if ($method === 'GET') {
            $send($start, $end);
        }
        exit;
    }
    $boundary = 'v31np-' . $etag;
    $lines = [];
    $length = 0;
    foreach ($parts as $i => [$start, $end]) {
        $lines[$i] = "--" . $boundary . "\r\nContent-Type: " . $type . "\r\nContent-Range: bytes " . $start . '-' . $end . '/' . $total . "\r\n\r\n";
        $length += strlen($lines[$i]) + ($end - $start + 1) + 2;
    }
    $closing = "--" . $boundary . "--\r\n";
    header('Content-Type: multipart/byteranges; boundary=' . $boundary);
    header('Content-Length: ' . ($length + strlen($closing)));
    if ($method === 'GET') {
        foreach ($parts as $i => [$start, $end]) {
            echo $lines[$i];
            $send($start, $end);
            echo "\r\n";
        }
        echo $closing;
    }
    exit;
}
```

8 - File: lib/settings.php

Size *399*

SHA-256 *E61DEE96BE4BFE65553AA48B5AB3A1215338C42B4CA281F574154E45434563A9*

Code

```
<?php

function v31np_settings(string $name): array
{
    $all = json_decode((string)@file_get_contents(dirname(__DIR__) . '/settings.json'), true);
    $all = is_array($all) ? $all : [];
    $own = is_array($all['archives'][$name] ?? null) ? $all['archives'][$name] : [];
    unset($all['archives']);
    return $own + $all + ['empty' => '268', 'behind' => '268', 'send' => ['S'], 'sign' => ''];
}
```

9 - File: lib/sync.js

Size

24304

SHA-256

0AE45D8900B36759F636A1726996FE580D53CF11EA294DB82D88840DE469027C

Code

```
const V31nSync = (() => {
  const CHUNK = 4194304;
  const INDEX = 'V31NBAIMITIF\0\0\0';
  const INDEX_KEY = 'v31np index.txt';
  const LEVELS = ['S', 'A', 'B', '\u00DA', 'U', '\u00D9'];
  const request = r => new Promise((ok, fail) => { r.onsuccess = () => ok(r.result); r.onerror = () => fail(r.error); });
  const done = t => new Promise((ok, fail) => { t.oncomplete = () => ok(); t.onerror = () => fail(t.error); t.onabort = () => fail(t.error); });
  let opened = null;
  const db = () => opened || (opened = new Promise((ok, fail) => {
    const r = indexedDB.open('v31np', 3);
    r.onupgradeneeded = e => {
      const d = r.result;
      if (e.oldVersion < 3) {
        for (const name of ['files', 'archives']) if (d.objectStoreNames.contains(name)) d.deleteObjectStore(name);
      }
      for (const [name, keyPath] of [['files', ['name', 'key']], ['parts', ['id', 'n']], ['archives', 'name'], ['queries', ['name', 'key']], ['log', 'id']]) {
        if (!d.objectStoreNames.contains(name)) d.createObjectStore(name, name === 'log' ? { keyPath, autoIncrement: true } : { keyPath });
      }
    };
    r.onsuccess = () => ok(r.result);
    r.onerror = () => fail(r.error);
  }));
  const range = name => IDBKeyRange.bound([name], [name, []]);
  const idRange = id => IDBKeyRange.bound([id], [id, []]);
  const byKey = (a, b) => V31nOrder.key(a.key, b.key);
  const tx = async (stores, mode) => (await db()).transaction(stores, mode);
  const want = () => !!V31N_KEY;

  const held = async name => (await request((await tx('files', 'readonly')).objectStore('files').getAll(range(name)))).sort(byKey);
  const archive = async name => request((await tx('archives', 'readonly')).objectStore('archives').get(name));
  const file = async (name, key) => request((await tx('files', 'readonly')).objectStore('files').get([name, key]));
  const heads = async (name, key) => {
    const a = await archive(name);
    return a && a.heads && a.heads[key] ? a.heads[key] : [];
  };

  const newId = () => Date.now().toString(36) + Math.random().toString(36).slice(2);
  const putPart = async (id, n, blob) => {
    const t = await tx('parts', 'readwrite');
    t.objectStore('parts').put({ id, n, blob });
    await done(t);
  };
  const dropParts = async ids => {
    if (!ids.length) return;
    const t = await tx('parts', 'readwrite');
    const store = t.objectStore('parts');
    for (const id of ids) store.delete(idRange(id));
    await done(t);
  };
  const partsOf = async meta => {
    if (!meta.parts) return [];
    const all = await request((await tx('parts', 'readonly')).objectStore('parts').getAll(idRange(meta.id)));
    return all.sort((a, b) => a.n - b.n).map(p => p.blob);
  };
  const blob = async (meta, type) => new Blob(await partsOf(meta), type ? { type } : undefined);
  const slice = async (meta, start, end, type) => {
    const first = Math.floor(start / CHUNK), last = Math.floor(end / CHUNK);
    const store = (await tx('parts', 'readonly')).objectStore('parts');
    const asks = [];
    for (let n = first; n <= last; n++) asks.push(request(store.get([meta.id, n])));
    const got = await Promise.all(asks);
    const out = got.map((p, i) => {
      const base = (first + i) * CHUNK;
      return p.blob.slice(Math.max(0, start - base), Math.min(p.blob.size, end + 1 - base));
    });
    return new Blob(out, type ? { type } : undefined);
  };
  const bytesOf = async meta => new Uint8Array(await (await blob(meta)).arrayBuffer());

  const sink = () => {
    const id = newId();
    const sha = want() ? V31nSha256.create() : null;
    let n = 0, size = 0, crc = 0, buf = [], fill = 0;
    const flush = async () => {
      if (!fill) return;
      await putPart(id, n++, new Blob(buf));
      buf = [];
      fill = 0;
    };
    return {
      id,
      push: async data => {
        crc = V31nPack.crc(data, crc);
        if (sha) sha.update(data);
        size += data.length;
        let off = 0;
        while (off < data.length) {
          const take = Math.min(CHUNK - fill, data.length - off);
          buf.push(data.slice(off, off + take));
          fill += take;
          off += take;
          if (fill === CHUNK) await flush();
        }
      },
      end: async () => {
        await flush();
        return { id, parts: n, size, crc: V31nPack.hex(crc), sha: sha ? sha.hex() : null };
      }
    };
  };
  const store = async (key, bytes) => {
    const s = sink();
    await s.push(bytes);
    return { key, ...(await s.end()) };
  };

  const strict = new TextDecoder('utf-8', { fatal: true });
  const isIndex = b => b.length >= 15 && [...INDEX].every((c, i) => b[i] === c.charCodeAt(0));
  const unescape = s => {
    let out = '';
    for (let i = 0; i < s.length; i++) {
      if (s[i] !== '\\') { out += s[i]; continue; }
      const e = { '\\': '\\', t: '\t', n: '\n', r: '\r' }[s[++i]];
      if (e === undefined) return null;
      out += e;
    }
    return out;
  };
  const readIndex = bytes => {
    if (!bytes || !isIndex(bytes)) return null;
    let text;
    try { text = strict.decode(bytes); } catch { return null; }
    const rows = new Map(), heads = {};
    let sig = null, signedTo = 0, at = 0, first = true;
    for (const raw of text.split('\n')) {
      const start = at;
      at += raw.length + 1;
      const line = raw.endsWith('\r') ? raw.slice(0, -1) : raw;
      if (line === '') continue;
      if (first) { if (line !== INDEX) return null; first = false; continue; }
      if (sig !== null) return null;
      const f = line.split('\t');
      if (f[0] === 'I' && f.length >= 4 && f.length <= 6 && !Object.keys(heads).length) {
        const key = unescape(f[1]);
        if (key === null || rows.has(key) || !/^\d+$/.test(f[2]) || !/^[0-9A-F]{8}$/.test(f[3])) return null;
        let sha = null, level = null;
        for (const extra of f.slice(4)) {
          if (sha === null && level === null && /^[0-9A-F]{64}$/.test(extra)) sha = extra;
          else if (level === null && LEVELS.includes(extra)) level = extra;
          else return null;
        }
        rows.set(key, { size: Number(f[2]), crc: f[3], sha, level: level || '\u00D9' });
```

```
    } else if (f[0] === 'H' && f.length === 4) {
      const key = unescape(f[1]), n = unescape(f[2]), v = unescape(f[3]);
      if (key === null || !rows.has(key) || !n || !/^[!#$%&'*+.^_`|~0-9A-Za-z-]+$/.test(n) || v === null || /[\0\r\n]/.test(v)) return null;
      (heads[key] = heads[key] || []).push([n, v]);
    } else if (f[0] === 'S' && f.length === 2 && /^[0-9A-F]{128}$/.test(f[1])) {
      sig = f[1];
      signedTo = start;
    } else return null;
  }
  return { rows, heads, sig, signed: sig ? new TextEncoder().encode(text.slice(0, signedTo)) : null };
};
const unhex = s => new Uint8Array(s.match(/../g).map(h => parseInt(h, 16)));
const signedBy = async ix => {
  if (!V31N_KEY) return true;
  if (!ix || !ix.sig) return false;
  try {
    const pub = await crypto.subtle.importKey('raw', unhex(V31N_KEY), { name: 'Ed25519' }, false, ['verify']);
    return await crypto.subtle.verify({ name: 'Ed25519' }, pub, unhex(ix.sig), ix.signed);
  } catch {
    return false;
  }
};
const fit = key => key === '' || key === INDEX_KEY || (key.startsWith('/') && !key.split('/').some(s => s === '.' || s === '..'));
const matches = (row, m) => !!row && row.size === m.size && row.crc === m.crc && (!V31N_KEY || (!!row.sha && row.sha === m.sha));
const checkWhole = async (metas, ix) => {
  if (!V31N_KEY) return true;
  if (!await signedBy(ix)) return false;
  const others = metas.filter(m => m.key !== INDEX_KEY);
  return others.length === [...ix.rows.keys()].filter(fit).length && others.every(m => matches(ix.rows.get(m.key), m));
};
const archiveSum = metas => {
  const header = V31nPack.head(V31nPack.MAGIC, metas.map(m => ({ key: m.key, size: m.size })));
  let sum = V31nPack.crc(header);
  for (const m of metas) sum = V31nPack.combine(sum, parseInt(m.crc, 16), m.size);
  return V31nPack.hex(V31nPack.combine(sum, V31nPack.crc(V31nPack.PI), V31nPack.MARK));
};

const streamFull = async response => {
  const reader = response.body.getReader();
  let pending = new Uint8Array(0);
  const pull = async () => {
    const r = await reader.read();
    if (r.done) return false;
    pending = pending.length ? V31nPack.join([pending, r.value]) : r.value;
    return true;
  };
  const made = [];
  const fail = async error => {
    try { await reader.cancel(); } catch {}
    await dropParts(made);
    return { error };
  };
  let head;
  for (;;) {
    head = V31nPack.headOf(V31nPack.MAGIC, pending);
    if (head !== 'more') break;
    if (!await pull()) return fail('short header');
  }
  if (!head) return fail('broken header');
  let sum = V31nPack.crc(pending.subarray(0, head.length));
  pending = pending.subarray(head.length);
  const metas = [];
  let indexBytes = null;
  for (const e of head.entries) {
    const s = fit(e.key) ? sink() : null;
    if (s) made.push(s.id);
    const keep = e.key === INDEX_KEY ? [] : null;
    let left = e.size;
    while (left > 0) {
      if (!pending.length && !await pull()) return fail('short');
      const take = pending.subarray(0, Math.min(left, pending.length));
      if (s) await s.push(take);
      sum = V31nPack.crc(take, sum);
      if (keep) keep.push(take.slice());
      pending = pending.subarray(take.length);
      left -= take.length;
    }
    if (s) metas.push({ key: e.key, ...(await s.end()) });
    if (keep) indexBytes = V31nPack.join(keep);
  }
  while (pending.length < V31nPack.FOOT) if (!await pull()) return fail('short footer');
  for (let k = 0; k < V31nPack.MARK; k++) if (pending[k] !== V31nPack.PI[k]) return fail('broken footer');
  sum = V31nPack.crc(pending.subarray(0, V31nPack.MARK), sum);
  const stated = new DataView(pending.buffer, pending.byteOffset + V31nPack.MARK, 4).getUint32(0, true);
  if (pending.length > V31nPack.FOOT || await pull()) return fail('trailing bytes');
  if (sum !== stated) return fail('crc');
  return { metas, ix: readIndex(indexBytes), crc: V31nPack.hex(sum), ids: made };
};

const commit = async (name, put, remove, record) => {
  const t = await tx(['files', 'archives'], 'readwrite');
  const files = t.objectStore('files');
  for (const key of remove) files.delete([name, key]);
  for (const m of put) files.put({ name, ...m });
  if (record) t.objectStore('archives').put({ name, ...record });
  await done(t);
};

const listOf = (ix, indexSize) => {
  const keys = [...ix.rows.keys(), INDEX_KEY].sort(V31nOrder.key);
  const items = keys.map(key => ({ key, size: key === INDEX_KEY ? indexSize : ix.rows.get(key).size }));
  let at = V31nPack.head(V31nPack.MAGIC, items).length;
  const list = {};
  for (const it of items) {
    list[it.key] = [at, it.size];
    at += it.size;
  }
  return list;
};

const etag = r => (r.headers.get('etag') || '').replace(/"/g, '') || null;

const run = async (name, why, url = '/' + name + '.v31np', wanted = []) => {
  const log = { name, why, steps: [] };
  const step = (what, r, bytes) => log.steps.push({ what, status: r.status, redirected: r.redirected, etag: etag(r), bytes });
  const old = await held(name);

  const whole = async (r, again) => {
    const got = await streamFull(r);
    step(again ? 'full after failed check' : 'full', r, got.error ? null : got.metas.reduce((n, m) => n + m.size, 0));
    if (got.error) { log.result = 'broken full file: ' + got.error; return; }
    const tag = etag(r);
    if (tag && !r.redirected && tag !== got.crc) {
      await dropParts(got.ids);
      log.result = 'etag differs from full file';
      return;
    }
    if (!await checkWhole(got.metas, got.ix)) {
      await dropParts(got.ids);
      if (again) { log.result = 'refused, check failed'; return; }
      return whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
    }
    await commit(name, got.metas, old.map(m => m.key), { crc: got.crc, heads: got.ix ? got.ix.heads : {}, mode: 'full', url });
    await dropParts(old.map(m => m.id));
    log.result = 'stored ' + got.metas.length + ' files, crc ' + got.crc;
  };

  const listed = async (bytes, tag) => {
    const ix = readIndex(bytes);
    if (!ix) return null;
    const rows = [...ix.rows.keys(), INDEX_KEY].sort(V31nOrder.key).map(key => key === INDEX_KEY
      ? { key, size: bytes.length, crc: V31nPack.hex(V31nPack.crc(bytes)) }
      : { key, size: ix.rows.get(key).size, crc: ix.rows.get(key).crc });
    if (tag !== archiveSum(rows) || !await signedBy(ix)) return null;
    return { ix, list: listOf(ix, bytes.length) };
  };

  const asked = new Map(old.map(f => [f.key, f.crc]));
  for (const key of wanted) if (!asked.has(key)) asked.set(key, '00000000');
  const ask = V31nPack.write(V31nPack.ASK, [...asked].sort((x, y) => V31nOrder.key(x[0], y[0])).map(([key, crc]) => ({ key, bytes: V31nPack.bytes(crc) })));
  const body = V31nPack.write(V31nPack.MAGIC, [{ key: '', bytes: ask.buf }]);
  log.sent = body.buf.length;
  if (wanted.length) log.wanted = wanted.length;
  const before = await archive(name);
  const r = await fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/v31np' }, body: body.buf, cache: 'no-store' });

  if (r.status === 200) await whole(r, false);
  else if (r.status === 304) { step('sync', r, 0); log.result = 'in sync'; }
  else if (r.status === 268) {
    const buf = new Uint8Array(await r.arrayBuffer());
    step('sync', r, buf.length);
    const outer = V31nPack.parse(V31nPack.MAGIC, buf);
    const c = outer && outer.entries.length === 1 && outer.entries[0].key === INDEX_KEY ? outer.entries[0] : null;
```

```
      const bytes = e ? buf.slice(e.at, e.at + e.size) : null;
      const got = bytes ? await listed(bytes, etag(r)) : null;
      if (!got) await whole(await fetch(url, { headers: { 'Accept': 'application/v31np' }, cache: 'no-store' }), true);
      else {
        const im = await store(INDEX_KEY, bytes);
        const stale = old.filter(m => m.key === INDEX_KEY || !matches(got.ix.rows.get(m.key), m));
        await commit(name, [im], stale.map(m => m.key), { crc: etag(r), heads: got.ix.heads, mode: 'list', url, list: got.list });
        await dropParts(stale.map(m => m.id));
        log.result = 'list of ' + got.ix.rows.size + ' files, ' + (old.length - stale.length) + ' kept, crc ' + etag(r);
      }
    } else if (r.status === 267) {
      const buf = new Uint8Array(await r.arrayBuffer());
      step('sync', r, buf.length);
      const outer = V31nPack.parse(V31nPack.MAGIC, buf);
      const e = outer && outer.entries.length === 1 && outer.entries[0].key === '' ? outer.entries[0] : null;
      const inner = e ? buf.slice(e.at, e.at + e.size) : null;
      const patch = inner ? V31nPack.parse(V31nPack.PATCH, inner) : null;
      if (!patch) log.result = 'broken 267 answer';
      else if (before && before.mode === 'list') {
        const files = new Map(old.map(m => [m.key, m]));
        const added = [], removed = [], absent = [];
        let bad = false;
        for (const p of patch.entries) {
          if (!fit(p.key)) continue;
          const flag = p.size > 0 ? inner[p.at + p.size - 1] : -1;
          if (flag === 1 && p.size === 1) {
            if (files.has(p.key)) { files.delete(p.key); removed.push(p.key); }
            else absent.push(p.key);
          } else if (flag === 0) {
            const m = await store(p.key, inner.slice(p.at, p.at + p.size - 1));
            files.set(p.key, m);
            added.push(m);
          } else bad = true;
        }
        const im = files.get(INDEX_KEY);
        const indexBytes = im ? await bytesOf(im) : null;
        let got = null;
        if (added.some(m => m.key === INDEX_KEY)) got = await listed(indexBytes, etag(r));
        else if (indexBytes && etag(r) === before.crc) got = { ix: readIndex(indexBytes), list: before.list };
        const ok = !bad && !!got && !!got.ix && added.every(m => m.key === INDEX_KEY || matches(got.ix.rows.get(m.key), m));
        log.patch = { added: added.map(m => m.key), removed, absent, etag: etag(r) };
        if (ok) {
          const stale = [...files.values()].filter(m => m.key !== INDEX_KEY && !matches(got.ix.rows.get(m.key), m));
          const gone = [...removed, ...stale.map(m => m.key)];
          const replaced = old.filter(m => gone.includes(m.key) || added.some(x => x.key === m.key));
          await commit(name, added, gone, { crc: etag(r), heads: got.ix.heads, mode: 'list', url, list: got.list });
          await dropParts(replaced.map(m => m.id));
          log.result = 'took ' + added.length + ' files, crc ' + etag(r);
        } else {
          await dropParts(added.map(m => m.id));
          await whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
        }
      } else {
        const files = new Map(old.map(m => [m.key, m]));
        const added = [], removed = [];
        let bad = false;
        for (const p of patch.entries) {
          if (!fit(p.key)) continue;
          const flag = p.size > 0 ? inner[p.at + p.size - 1] : -1;
          if (flag === 1 && p.size === 1) { files.delete(p.key); removed.push(p.key); }
          else if (flag === 0) {
            const m = await store(p.key, inner.slice(p.at, p.at + p.size - 1));
            files.set(p.key, m);
            added.push(m);
          } else bad = true;
        }
        const rebuilt = [...files.values()].sort(byKey);
        const crc = archiveSum(rebuilt);
        const im = files.get(INDEX_KEY);
        const ix = im ? readIndex(await bytesOf(im)) : null;
        log.patch = { changed: added.map(m => m.key), removed, crc, etag: etag(r) };
        if (!bad && crc === etag(r) && await checkWhole(rebuilt, ix)) {
          const replaced = old.filter(m => removed.includes(m.key) || added.some(x => x.key === m.key));
          await commit(name, added, removed, { crc, heads: ix ? ix.heads : {}, mode: 'fill', url });
          await dropParts(replaced.map(m => m.id));
          log.result = 'patched, crc ' + crc;
        } else {
          await dropParts(added.map(m => m.id));
          await whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
        }
      }
    } else { step('sync', r, null); log.result = 'status ' + r.status; }
    await note({ kind: 'sync', ...log });
    return log;
  };

  const chains = new Map();
  const enqueue = (name, fn) => {
    const next = (chains.get(name) || Promise.resolve()).catch(() => {}).then(fn);
    chains.set(name, next);
    return next;
  };
  const running = new Map();
  const sync = (name, why, url) => {
    if (!running.has(name)) running.set(name, enqueue(name, () => run(name, why, url)).finally(() => running.delete(name)));
    return running.get(name);
  };

  const listedSize = async (name, key) => {
    const a = await archive(name);
    return a && a.mode === 'list' && a.list && a.list[key] && key !== INDEX_KEY ? a.list[key][1] : null;
  };
  const waiting = new Map();
  const need = (name, key) => {
    let w = waiting.get(name);
    if (!w) {
      w = { keys: new Set() };
      w.done = new Promise(resolve => setTimeout(() => {
        waiting.delete(name);
        resolve(enqueue(name, async () => {
          const a = await archive(name);
          const keys = [];
          for (const k of w.keys) if (a && a.list && a.list[k] && !await file(name, k)) keys.push(k);
          if (keys.length) return run(name, 'want ' + keys.length, a.url, keys);
        }));
      }, 30));
      waiting.set(name, w);
    }
    w.keys.add(key);
    return w.done.catch(() => {}).then(() => file(name, key));
  };
  const remote = async (name, key, start, end) => {
    const a = await archive(name);
    if (!a || a.mode !== 'list' || !a.list || !a.list[key]) return null;
    const at = a.list[key][0];
    const want = 'bytes=' + (at + start) + '-' + (at + end);
    const r = await fetch(a.url, { headers: { Range: want, 'If-Range': '"' + a.crc + '"' }, cache: 'no-store' });
    if (r.status !== 206 || !(r.headers.get('content-range') || '').startsWith('bytes ' + (at + start) + '-' + (at + end) + '/')) {
      try { await r.body.cancel(); } catch {}
      return null;
    }
    return r.body;
  };

  const wantKeys = (name, keys) => enqueue(name, async () => {
    const a = await archive(name);
    const missing = [];
    for (const k of keys) if (a && a.list && a.list[k] && !await file(name, k)) missing.push(k);
    if (missing.length) return run(name, 'want ' + missing.length, a.url, missing);
  });
  const rows = async name => {
    const im = await file(name, INDEX_KEY);
    const ix = im ? readIndex(await bytesOf(im)) : null;
    return ix ? ix.rows : new Map();
  };

  const load = async name => {
    const files = [];
    for (const m of await held(name)) files.push({ key: m.key, bytes: await bytesOf(m) });
    const a = await archive(name);
    return { files, crc: a ? a.crc : null };
  };
  const query = async (name, key, value) => {
    const t = await tx('queries', 'readwrite');
    const s = t.objectStore('queries');
    const old = await request(s.get([name, key]));
    s.put({ name, key, value });
    await done(t);
    return old ? old.value : null;
  };
  const note = async entry => {
    const t = await tx('log', 'readwrite');
    t.objectStore('log').add(entry);
```

```
    await done(t);
  };
  const notes = async () => request((await tx('log', 'readonly')).objectStore('log').getAll());

  return { sync, load, file, blob, slice, need, want: wantKeys, rows, remote, listed: listedSize, held, query, note, notes, heads };
})();
```

10 - File: lib/sha256.js

Size

3526

SHA-256

C25EBBB1551B349ED828C575AB6CF4CD8DB413FCF1DF669D76AF728372A619FF

Code

```
const V31nSha256 = (() => {
    const K = new Uint32Array([
        0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
        0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
        0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
        0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
        0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
        0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
        0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
        0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
    ]);

    const create = () => {
        const h = new Uint32Array([0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19]);
        const w = new Uint32Array(64);
        const block = new Uint8Array(64);
        let fill = 0, total = 0;

        const compress = (b, o) => {
            for (let i = 0; i < 16; i++) w[i] = (b[o + 4 * i] << 24) | (b[o + 4 * i + 1] << 16) | (b[o + 4 * i + 2] << 8) | b[o + 4 * i + 3];
            for (let i = 16; i < 64; i++) {
                const x = w[i - 15], y = w[i - 2];
                const s0 = ((x >>> 7) | (x << 25)) ^ ((x >>> 18) | (x << 14)) ^ (x >>> 3);
                const s1 = ((y >>> 17) | (y << 15)) ^ ((y >>> 19) | (y << 13)) ^ (y >>> 10);
                w[i] = (w[i - 16] + s0 + w[i - 7] + s1) | 0;
            }
            let a = h[0], bb = h[1], c = h[2], d = h[3], e = h[4], f = h[5], g = h[6], hh = h[7];
            for (let i = 0; i < 64; i++) {
                const S1 = ((e >>> 6) | (e << 26)) ^ ((e >>> 11) | (e << 21)) ^ ((e >>> 25) | (e << 7));
                const t1 = (hh + S1 + ((e & f) ^ (~e & g)) + K[i] + w[i]) | 0;
                const S0 = ((a >>> 2) | (a << 30)) ^ ((a >>> 13) | (a << 19)) ^ ((a >>> 22) | (a << 10));
                const t2 = (S0 + ((a & bb) ^ (a & c) ^ (bb & c))) | 0;
                hh = g; g = f; f = e; e = (d + t1) | 0; d = c; c = bb; bb = a; a = (t1 + t2) | 0;
            }
            h[0] += a; h[1] += bb; h[2] += c; h[3] += d; h[4] += e; h[5] += f; h[6] += g; h[7] += hh;
        };

        const update = data => {
            let i = 0;
            total += data.length;
            if (fill) {
                while (fill < 64 && i < data.length) block[fill++] = data[i++];
                if (fill < 64) return;
                compress(block, 0);
                fill = 0;
            }
            for (; i + 64 <= data.length; i += 64) compress(data, i);
            while (i < data.length) block[fill++] = data[i++];
        };

        const hex = () => {
            const bits = total * 8;
            const tail = new Uint8Array(fill < 56 ? 64 : 128);
            tail.set(block.subarray(0, fill));
            tail[fill] = 0x80;
            const view = new DataView(tail.buffer);
            view.setUint32(tail.length - 8, Math.floor(bits / 4294967296));
            view.setUint32(tail.length - 4, bits >>> 0);
            for (let o = 0; o < tail.length; o += 64) compress(tail, o);
            return [...h].map(v => v.toString(16).toUpperCase().padStart(8, '0')).join('');
        };

        return { update, hex };
    };

    return { create };
})();
```

11 - File: lib/show.js

Size *19945*

SHA-256 *67A02048CB30CB62A1F64B645E2B64003F257DAE49EA88D20BB5E43751C5B172*

Code

```
const V31nShow = (() => {
  const INDEX_KEY = 'v31np index.txt';
  const SVG = 'http://www.w3.org/2000/svg';
  const types = {
    html: 'text/html; charset=utf-8', htm: 'text/html; charset=utf-8', css: 'text/css; charset=utf-8', js: 'text/javascript; charset=utf-8',
    mjs: 'text/javascript; charset=utf-8', json: 'application/json', txt: 'text/plain; charset=utf-8', xml: 'application/xml', svg: 'image/svg+xml',
    png: 'image/png', jpg: 'image/jpeg', jpeg: 'image/jpeg', gif: 'image/gif', ico: 'image/x-icon', webp: 'image/webp', avif: 'image/avif',
    bmp: 'image/bmp', woff2: 'font/woff2', woff: 'font/woff', ttf: 'font/ttf', otf: 'font/otf', wasm: 'application/wasm', pdf: 'application/pdf',
    mp3: 'audio/mpeg', wav: 'audio/wav', ogg: 'audio/ogg', oga: 'audio/ogg', opus: 'audio/ogg', m4a: 'audio/mp4', aac: 'audio/aac', flac: 'audio/flac',
    mp4: 'video/mp4', m4v: 'video/mp4', webm: 'video/webm', ogv: 'video/ogg', mov: 'video/quicktime', vtt: 'text/vtt; charset=utf-8', stl: 'model/stl'
  };
  const extOf = key => {
    const dot = key.lastIndexOf('.');
    return dot > key.lastIndexOf('/') && dot >= 0 ? key.slice(dot + 1).toLowerCase() : '';
  };
  const typeOf = key => types[extOf(key)] || 'application/octet-stream';
  const isCss = key => extOf(key) === 'css';
  const skip = /^(?:blob:|data:|about:|javascript:|mailto:|tel:|#)/i;
  const fileUrl = key => location.origin + key.split('/').map(encodeURIComponent).join('/');
  const target = (raw, base) => {
    if (typeof raw !== 'string') return null;
    const t = raw.trim();
    if (t === '' || skip.test(t)) return null;
    let u;
    try { u = new URL(t, base); } catch { return null; }
    if (u.origin !== location.origin) return null;
    let key;
    try { key = decodeURIComponent(u.pathname); } catch { return null; }
    let search = u.search;
    try { search = decodeURIComponent(u.search); } catch {}
    return { key, search, rawSearch: u.search, hash: u.hash, href: u.href };
  };
  const cssPattern = () => /url\(\s*(["']?)([^"')]+?)\1\s*\)|@import\s+(["'])([^"']+)\3/g;
  const cssRefs = text => {
    const out = [];
    for (const m of String(text).matchAll(cssPattern())) out.push(m[2] !== undefined ? m[2] : m[4]);
    return out;
  };
  const srcsetRefs = value => String(value).split(',').map(s => s.trim().split(/\s+/)[0]).filter(Boolean);

  const links = /(?:^|\s)(?:stylesheet|icon|shortcut|apple-touch-icon|mask-icon|preload|modulepreload|prefetch)(?:\s|$)/i;
  const kinds = el => {
    const name = el.namespaceURI === SVG ? el.localName : el.tagName;
    switch (name) {
      case 'IMG': return { src: 'take', srcset: 'set' };
      case 'SCRIPT': return { src: 'take' };
      case 'INPUT': return { src: 'take' };
      case 'EMBED': return { src: 'take' };
      case 'OBJECT': return { data: 'take' };
      case 'LINK': return links.test(el.getAttribute('rel') || '') ? { href: 'take' } : {};
      case 'AUDIO': return { src: 'play' };
      case 'VIDEO': return { src: 'play', poster: 'take' };
      case 'SOURCE': return el.parentNode && /^(AUDIO|VIDEO)$/.test(el.parentNode.tagName || '') ? { src: 'play' } : { src: 'take', srcset: 'set' };
      case 'TRACK': return { src: 'play' };
      case 'image': case 'use': case 'feImage': return { href: 'take', 'xlink:href': 'take' };
      default: return {};
    }
  };

  const report = entry => console.log('v31np', JSON.stringify(entry));
  const leave = async () => {
    if (!navigator.serviceWorker) return;
    for (const r of await navigator.serviceWorker.getRegistrations()) {
      const w = r.active || r.waiting || r.installing;
      if (w && new URL(w.scriptURL).pathname === '/v31np/sw.js') await r.unregister();
    }
  };

  const start = async ({ archive, page, html, started, logs, queries = [] }) => {
    const archiveAbs = new URL(archive, location.href).href;
    const listed = await V31nSync.rows(archive);
    const held = new Map();
    const fill = async () => {
      for (const m of await V31nSync.held(archive)) if (m.key !== INDEX_KEY) held.set(m.key, m);
      if (page) for (const m of await V31nSync.held(page)) if (m.key !== '' && m.key !== INDEX_KEY) held.set(m.key, m);
    };
    await fill();

    const doc = new DOMParser().parseFromString(html, 'text/html');
    const baseEl = doc.querySelector('base[href]');
    const base = baseEl ? new URL(baseEl.getAttribute('href'), location.href).href : location.href;

    const refs = new Set();
    const add = (raw, b) => {
      const t = target(raw, b);
      if (t && listed.has(t.key)) refs.add(t.key);
    };
    const each = fn => {
      for (const el of doc.querySelectorAll('*')) {
        const k = kinds(el);
        for (const a of [...el.attributes]) {
          if (a.name === 'style') fn(el, a, 'style');
          else if (k[a.name]) fn(el, a, k[a.name]);
        }
        if (el.tagName === 'STYLE') fn(el, null, 'sheet');
        if (el.tagName === 'SCRIPT' && (el.getAttribute('type') || '').toLowerCase() === 'importmap') fn(el, null, 'importmap');
      }
    };
    each((el, a, kind) => {
      if (kind === 'take') add(a.value, base);
      else if (kind === 'set') for (const u of srcsetRefs(a.value)) add(u, base);
      else if (kind === 'style') for (const u of cssRefs(a.value)) add(u, base);
      else if (kind === 'sheet') for (const u of cssRefs(el.textContent)) add(u, base);
      else if (kind === 'importmap') {
        try { for (const v of Object.values(JSON.parse(el.textContent).imports || {})) add(String(v), base); } catch {}
      }
    });

    const asked = new Set();
    const rounds = [];
    const round = async keys => {
      const fresh = [...keys].filter(k => !held.has(k) && !asked.has(k));
      if (!fresh.length) return false;
      for (const k of fresh) asked.add(k);
      const got = await V31nSync.want(archive, fresh);
      rounds.push({ wanted: fresh.length, result: got && got.result, steps: got && got.steps });
      await fill();
      return true;
    };
    await round([...refs]);
    const read = new Set();
    for (;;) {
      const more = new Set();
      for (const key of [...refs]) {
        if (!isCss(key) || read.has(key) || !held.has(key)) continue;
        read.add(key);
        const text = await (await V31nSync.blob(held.get(key))).text();
        for (const u of cssRefs(text)) {
          const t = target(u, fileUrl(key));
          if (!t || !listed.has(t.key)) continue;
          refs.add(t.key);
          if (!held.has(t.key)) more.add(t.key);
        }
      }
      if (!more.size || !await round(more)) break;
    }

    const urls = new Map();
    const making = new Set();
    const map = (raw, b, absolute = false) => {
      const t = target(raw, b);
      if (!t) return raw;
```

```
    if (t.search && urls.has(t.key + t.search)) return urls.get(t.key + t.search) + t.hash;
    if (urls.has(t.key)) return urls.get(t.key) + t.hash;
    if (listed.has(t.key)) return archiveAbs + "?key=" + encodeURIComponent(t.key) + t.hash;
    return absolute ? t.href : raw;
  };
  const cssMap = (text, b, absolute = false) => String(text).replace(cssPattern(), (m, q, u, q2, u2) => u !== undefined
    ? 'url(' + q + map(u, b, absolute) + q + ')'
    : '@import ' + q2 + map(u2, b, absolute) + q2);
  const setMap = (value, b) => String(value).split(',').map(part => {
    const s = part.trim();
    const at = s.search(/\s/);
    return at < 0 ? map(s, b) : map(s.slice(0, at), b) + s.slice(at);
  }).join(', ');
  const makeCss = async key => {
    if (urls.has(key) || making.has(key)) return;
    making.add(key);
    const own = fileUrl(key);
    const text = await (await V31nSync.blob(held.get(key))).text();
    for (const u of cssRefs(text)) {
      const t = target(u, own);
      if (t && isCss(t.key) && held.has(t.key)) await makeCss(t.key);
    }
    urls.set(key, URL.createObjectURL(new Blob([cssMap(text, own, true)], { type: typeOf(key) })));
  };
  const make = async () => {
    for (const [key, m] of held) if (!isCss(key) && !urls.has(key)) urls.set(key, URL.createObjectURL(await V31nSync.blob(m, typeOf(key))));
    for (const key of held.keys()) if (isCss(key)) await makeCss(key);
  };
  await make();

  each((el, a, kind) => {
    if (kind === 'take' || kind === 'play') a.value = map(a.value, base);
    else if (kind === 'set') a.value = setMap(a.value, base);
    else if (kind === 'style') a.value = cssMap(a.value, base);
    else if (kind === 'sheet') el.textContent = cssMap(el.textContent, base);
    else if (kind === 'importmap') {
      let o;
      try { o = JSON.parse(el.textContent); } catch { return; }
      o.imports = o.imports || {};
      for (const k of Object.keys(o.imports)) o.imports[k] = map(String(o.imports[k]), base);
      for (const [key, u] of urls) if (/\.m?js$/i.test(key)) o.imports[fileUrl(key)] = o.imports[fileUrl(key)] || u;
      el.textContent = JSON.stringify(o);
    }
  });

  const fixAttr = (el, attr, value) => {
    if (attr === 'style') return cssMap(value, document.baseURI);
    const kind = kinds(el)[attr];
    if (kind === 'set') return setMap(value, document.baseURI);
    return kind ? map(value, document.baseURI) : value;
  };
  const wrap = (proto, prop, fix) => {
    const d = proto && Object.getOwnPropertyDescriptor(proto, prop);
    if (!d || !d.set) return;
    Object.defineProperty(proto, prop, { ...d, set(v) { d.set.call(this, typeof v === 'string' ? fix(this, v) : v); } });
  };
  const byAttr = attr => (el, v) => fixAttr(el, attr, v);
  wrap(HTMLImageElement.prototype, 'src', byAttr('src'));
  wrap(HTMLImageElement.prototype, 'srcset', byAttr('srcset'));
  wrap(HTMLSourceElement.prototype, 'src', byAttr('src'));
  wrap(HTMLSourceElement.prototype, 'srcset', byAttr('srcset'));
  wrap(HTMLMediaElement.prototype, 'src', (el, v) => map(v, document.baseURI));
  wrap(HTMLVideoElement.prototype, 'poster', (el, v) => map(v, document.baseURI));
  wrap(HTMLTrackElement.prototype, 'src', (el, v) => map(v, document.baseURI));
  wrap(HTMLLinkElement.prototype, 'href', (el, v) => map(v, document.baseURI));
  wrap(HTMLScriptElement.prototype, 'src', (el, v) => map(v, document.baseURI));
  wrap(HTMLInputElement.prototype, 'src', (el, v) => map(v, document.baseURI));
  wrap(HTMLEmbedElement.prototype, 'src', (el, v) => map(v, document.baseURI));
  wrap(HTMLObjectElement.prototype, 'data', (el, v) => map(v, document.baseURI));
  for (const prop of ['background', 'backgroundImage', 'cursor', 'content', 'listStyle', 'listStyleImage', 'borderImage', 'borderImageSource', 'maskImage', 'webkitMaskImage', 'cssText']) {
    wrap(CSSStyleDeclaration.prototype, prop, (el, v) => cssMap(v, document.baseURI));
  }
  const setProperty = CSSStyleDeclaration.prototype.setProperty;
  CSSStyleDeclaration.prototype.setProperty = function (prop, value, priority) {
    return setProperty.call(this, prop, typeof value === 'string' ? cssMap(value, document.baseURI) : value, priority);
  };
  const setAttribute = Element.prototype.setAttribute;
  Element.prototype.setAttribute = function (attr, value) {
    return setAttribute.call(this, attr, typeof value === 'string' ? fixAttr(this, String(attr).toLowerCase(), value) : value);
  };
  const realFetch = window.fetch.bind(window);
  const shown = new Map();
  window.fetch = async (input, init) => {
    const isReq = input instanceof Request;
    const raw = typeof input === 'string' || input instanceof URL ? String(input) : isReq ? input.url : null;
    const method = String((init && init.method) || (isReq ? input.method : 'GET')).toUpperCase();
    if (raw === null || method !== 'GET') return realFetch(input, init);
    const accept = String(new Headers((init && init.headers) || (isReq ? input.headers : undefined)).get('accept') || '');
    const t = target(raw, document.baseURI);
    if (t && queries.includes(t.key) && !accept.includes('application/v31np')) {
      const name = new URL(t.href).pathname + t.rawSearch;
      const got = await V31nSync.sync(name, 'query', name);
      const metas = await V31nSync.held(name);
      const own = metas.find(m => m.key === '');
      if (own) {
        for (const m of metas) {
          if (m.key === '' || m.key === INDEX_KEY || (urls.has(m.key) && shown.get(m.key) === m.id)) continue;
          shown.set(m.key, m.id);
          const old = urls.get(m.key);
          urls.set(m.key, URL.createObjectURL(await V31nSync.blob(m, typeOf(m.key.split('?')[0]))));
          if (old) URL.revokeObjectURL(old);
        }
        report({ what: 'query', url: name, result: got && got.result, steps: got && got.steps, files: metas.length - 2 });
        return new Response(await V31nSync.blob(own), { status: 200 });
      }
    }
    if (t && listed.has(t.key) && !urls.has(t.key) && !/^(?:audio|video)\//.test(typeOf(t.key))) {
      await V31nSync.need(archive, t.key);
      await fill();
      await make();
    }
    const mapped = map(raw, document.baseURI);
    if (mapped === raw) return realFetch(input, init);
    return realFetch(mapped, isReq ? { method: 'GET', headers: input.headers, signal: input.signal, ...init } : init);
  };
  const htmlMap = text => String(text)
    .replace(/(\s(?:src|poster)\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => {
      const plain = u.replace(/&amp;/g, '&');
      const m = map(plain, document.baseURI);
      return m === plain ? whole : a + q + m + q;
    })
    .replace(/(\ssrcset\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => a + q + setMap(u.replace(/&amp;/g, '&'), document.baseURI) + q)
    .replace(/(\sstyle\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => a + q + cssMap(u, document.baseURI) + q);
  for (const prop of ['innerHTML', 'outerHTML']) {
    const d = Object.getOwnPropertyDescriptor(Element.prototype, prop);
    if (d && d.set) Object.defineProperty(Element.prototype, prop, { ...d, set(v) { d.set.call(this, typeof v === 'string' ? htmlMap(v) : v); } });
  }
  const adjacent = Element.prototype.insertAdjacentHTML;
  Element.prototype.insertAdjacentHTML = function (where, text) {
    return adjacent.call(this, where, typeof text === 'string' ? htmlMap(text) : text);
  };
  const open = XMLHttpRequest.prototype.open;
  XMLHttpRequest.prototype.open = function (method, url, ...rest) {
    return open.call(this, method, typeof url === 'string' && String(method).toUpperCase() === 'GET' ? map(url, document.baseURI) : url, ...rest);
  };
  window.Audio = new Proxy(window.Audio, { construct: (t, args) => new t(...args.map((a, i) => i === 0 && typeof a === 'string' ? map(a, document.baseURI) : a)) });
  window.FontFace = new Proxy(window.FontFace, { construct: (t, args) => new t(...args.map((a, i) => i === 1 && typeof a === 'string' ? cssMap(a, document.baseURI) : a)) });
  if (window.Worker) window.Worker = new Proxy(window.Worker, { construct: (t, args) => new t(...args.map((a, i) => i === 0 && typeof a === 'string' ? map(a, document.baseURI) : a)) });

  const out = (doc.doctype ? '<!DOCTYPE ' + doc.doctype.name + '>\n' : '') + doc.documentElement.outerHTML;
  report({ what: 'page', ...logs, rounds, refs: refs.size, files: urls.size, listed: listed.size, ms: Math.round(performance.now() - started) });
  document.open();
  document.write(out);
  document.close();

  const second = [...listed].filter(([key, row]) => row.level === 'A' && !held.has(key)).map(([key]) => key);
  if (second.length) {
    const got = await V31nSync.want(archive, second);
    await fill();
    await make();
    report({ what: 'level A', result: got && got.result, steps: got && got.steps, files: urls.size, ms: Math.round(performance.now() - started) });
  }
};

const me = document.currentScript;
const stub = (() => {
  const meta = n => {
    const m = document.querySelector('meta[name="' + n + '"]');
    return m ? String(m.getAttribute('content') || '') : null;
  };
  const name = me && me.dataset.archive !== undefined ? me.dataset.archive : meta('v31np-archive');
  const url = me && me.dataset.url !== undefined ? me.dataset.url : meta('v31np-url');
  return name === null || url === null ? null : { name, url };
```

```
    })();
    if (typeof V31N_PAGE !== 'undefined' && stub && /^[A-Za-z0-9_-]+$/.test(stub.name) && stub.url !== '') {
        const started = performance.now();
        const archive = new URL(stub.url, location.href).href;
        const name = location.pathname + location.search;
        (async () => {
            await leave();
            const [pageLog, listLog] = await Promise.all([V31nSync.sync(name, 'page', name), V31nSync.sync(archive, 'list', archive)]);
            const logs = { page: { result: pageLog.result, steps: pageLog.steps }, list: { result: listLog.result, steps: listLog.steps } };
            const pm = (await V31nSync.held(name)).find(m => m.key === '');
            if (!pm) return report({ what: 'page', shown: false, ...logs });
            await start({ archive, page: name, html: await (await V31nSync.blob(pm)).text(), started, logs });
        })();
    } else if (stub && /^[A-Za-z0-9_-]+$/.test(stub.name) && stub.url !== '') {
        const started = performance.now();
        const archive = new URL(stub.url, location.href).href;
        const queries = String((me && me.dataset.query) || '').split(/\s+/).filter(Boolean).map(q => '/' + q.replace(/^\/+/, ''));
        const listing = leave().then(() => V31nSync.sync(archive, 'list', archive));
        const whole = document.documentElement.outerHTML;
        const mine = me.outerHTML;
        const at = whole.lastIndexOf(mine);
        const prefix = (document.doctype ? '<!DOCTYPE ' + document.doctype.name + '>\n' : '') + (at < 0 ? '' : whole.slice(0, at));
        document.write('<plaintext style="display:none">');
        document.addEventListener('DOMContentLoaded', async () => {
            const rest = document.querySelector('plaintext');
            const listLog = await listing;
            await start({ archive, page: null, html: prefix + (rest ? rest.textContent : ''), started, logs: { list: { result: listLog.result, steps: listLog.steps } }, queries });
        }, { once: true });
    }

    return { start };
})();
```

12 - File: lib/tables.json

Size

6272

SHA-256

876459F60317434C48C825A66543772303964120A71FA383209AC5DCA47C70C5

Code

{"ranges":[[9,13,true],[32,32,true],[33,47,false],[58,64,false],[91,96,false],[123,126,false],[133,133,true],[160,160,true],[161,191,false],[215,215,false],[247,247,false],[894,894,false],[903,903,false],[1370,1375,false],[1417,1418,false],[1470,1470,false],[1472,1472,false],[1475,1475,false],[1478,1478,false],[1523,1524,false],[1545,1546,false],[1548,1549,false],[1563,1563,false],[1566,1567,false],[1642,1645,false],[1748,1748,false],[1792,1805,false],[2039,2041,false],[2404,2405,false],[2416,2416,false],[3572,3572,false],[3663,3663,false],[3674,3675,false],[3844,3858,false],[3898,3901,false],[3973,3973,false],[4048,4052,false],[4170,4175,false],[4347,4347,false],[4961,4968,false],[5741,5742,false],[5760,5760,true],[5787,5788,false],[5867,5869,false],[5941,5942,false],[6100,6102,false],[6104,6106,false],[6144,6154,false],[6158,6158,true],[6468,6469,false],[6622,6623,false],[6686,6687,false],[7002,7008,false],[7227,7231,false],[7294,7295,false],[8192,8202,true],[8208,8231,false],[8232,8233,true],[8239,8239,true],[8240,8259,false],[8261,8273,false],[8275,8286,false],[8287,8287,true],[8317,8318,false],[8333,8334,false],[9001,9002,false],[10088,10101,false],[10181,10182,false],[10214,10223,false],[10627,10648,false],[10712,10715,false],[10748,10749,false],[11513,11516,false],[11518,11519,false],[11776,11822,false],[11824,11824,false],[12288,12288,true],[12289,12291,false],[12296,12305,false],[12308,12319,false],[12336,12336,false],[12349,12349,false],[12448,12448,false],[12539,12539,false],[42509,42511,false],[42611,42611,false],[42622,42622,false],[43124,43127,false],[43214,43215,false],[43310,43311,false],[43359,43359,false],[43612,43615,false],[62827,62828,false],[64830,64831,false],[65040,65049,false],[65072,65106,false],[65108,65121,false],[65123,65123,false],[65128,65128,false],[65130,65131,false],[65279,65279,true],[65281,65283,false],[65285,65290,false],[65292,65295,false],[65306,65307,false],[65311,65312,false],[65339,65341,false],[65343,65343,false],[65371,65371,false],[65373,65373,false],[65375,65381,false]],"slots":[[48,0,0],[49,1,0],[50,2,0],[51,3,0],[52,4,0],[53,5,0],[54,6,0],[55,7,0],[56,8,0],[57,9,0],[97,10,0],[1072,10,1],[98,11,0],[1073,11,1],[250,12,0],[118,12,1],[1074,12,2],[117,13,0],[1091,13,1],[249,14,0],[119,14,1],[103,15,0],[1075,15,1],[100,16,0],[1076,16,1],[254,16,2],[101,17,0],[1077,17,1],[1101,17,2],[380,18,0],[106,18,1],[1078,18,2],[122,19,0],[1079,19,1],[105,20,0],[1080,20,1],[237,21,0],[121,21,1],[1081,21,2],[107,22,0],[1082,22,1],[108,23,0],[1083,23,1],[109,24,0],[1084,24,1],[110,25,0],[1085,25,1],[111,26,0],[1086,26,1],[112,27,0],[1087,27,1],[114,28,0],[1088,28,1],[115,29,0],[1089,29,1],[116,30,0],[1090,30,1],[102,31,0],[1092,31,1],[539,32,0],[1094,32,1],[269,33,0],[1095,33,1],[537,34,0],[1096,34,1],[99,35,0],[113,36,0],[120,37,0],[104,38,0],[1093,38,1],[8216,38,2]],"lower":[[65,90,1,32],[192,214,1,32],[216,222,1,32],[256,302,2,1],[304,304,1,-199],[306,310,2,1],[313,327,2,1],[330,374,2,1],[376,376,1,-121],[377,381,2,1],[385,385,1,210],[386,388,2,1],[390,390,1,206],[391,391,1,1],[393,394,1,205],[395,395,1,1],[398,398,1,79],[399,399,1,202],[400,400,1,203],[401,401,1,1],[403,403,1,205],[404,404,1,207],[406,406,1,211],[407,407,1,209],[408,408,1,1],[412,412,1,211],[413,413,1,213],[415,415,1,214],[416,420,2,1],[422,422,1,218],[423,423,1,1],[425,425,1,218],[428,428,1,1],[430,430,1,218],[431,431,1,1],[433,434,1,217],[435,437,2,1],[439,439,1,219],[440,440,1,1],[444,444,1,1],[452,452,1,2],[453,453,1,1],[455,455,1,2],[456,456,1,1],[458,458,1,2],[459,475,2,1],[478,494,2,1],[497,497,1,2],[498,500,2,1],[502,502,1,-97],[503,503,1,-56],[504,542,2,1],[544,544,1,-130],[546,562,2,1],[570,570,1,10795],[571,571,1,1],[573,573,1,-163],[574,574,1,10792],[577,577,1,1],[579,579,1,-195],[580,580,1,69],[581,581,1,71],[582,590,2,1],[880,882,2,1],[886,886,1,1],[895,895,1,116],[902,902,1,38],[904,906,1,37],[908,908,1,64],[910,911,1,63],[913,929,1,32],[931,939,1,32],[975,975,1,8],[984,1006,2,1],[1012,1012,1,-60],[1015,1015,1,1],[1017,1017,1,-7],[1018,1018,1,1],[1021,1023,1,-130],[1024,1039,1,80],[1040,1071,1,32],[1120,1152,2,1],[1162,1214,2,1],[1216,1216,1,15],[1217,1229,2,1],[1232,1326,2,1],[1329,1366,1,48],[4256,4293,1,7264],[4295,4295,1,7264],[4301,4301,1,7264],[5024,5103,1,38864],[5104,5109,1,8],[7305,7305,1,1],[7312,7354,1,-3008],[7357,7359,1,-3008],[7680,7828,2,1],[7838,7838,1,-7615],[7840,7934,2,1],[7944,7951,1,-8],[7960,7965,1,-8],[7976,7983,1,-8],[7992,7999,1,-8],[8008,8013,1,-8],[8025,8031,2,-8],[8040,8047,1,-8],[8072,8079,1,-8],[8088,8095,1,-8],[8104,8111,1,-8],[8120,8121,1,-8],[8122,8123,1,-74],[8124,8124,1,-9],[8136,8139,1,-86],[8140,8140,1,-9],[8152,8153,1,-8],[8154,8155,1,-100],[8168,8169,1,-8],[8170,8171,1,-112],[8172,8172,1,-7],[8184,8185,1,-128],[8186,8187,1,-126],[8188,8188,1,-9],[8486,8486,1,-7517],[8490,8490,1,-8383],[8491,8491,1,-8262],[8498,8498,1,28],[8544,8559,1,16],[8579,8579,1,1],[9398,9423,1,26],[11264,11311,1,48],[11360,11360,1,1],[11362,11362,1,-10743],[11363,11363,1,-3814],[11364,11364,1,-10727],[11367,11371,2,1],[11373,11373,1,-10780],[11374,11374,1,-10749],[11375,11375,1,-10783],[11376,11376,1,-10782],[11378,11378,1,1],[11381,11381,1,1],[11390,11391,1,-10815],[11392,11490,2,1],[11499,11501,2,1],[11506,11506,1,1],[42560,42604,2,1],[42624,42650,2,1],[42786,42798,2,1],[42802,42862,2,1],[42873,42875,2,1],[42877,42877,1,-35332],[42878,42886,2,1],[42891,42891,1,1],[42893,42893,1,-42280],[42896,42898,2,1],[42902,42920,2,1],[42922,42922,1,-42308],[42923,42923,1,-42319],[42924,42924,1,-42315],[42925,42925,1,-42305],[42926,42926,1,-42308],[42928,42928,1,-42258],[42929,42929,1,-42282],[42930,42930,1,-42261],[42931,42931,1,928],[42932,42946,2,1],[42948,42948,1,-48],[42949,42949,1,-42307],[42950,42950,1,-35384],[42951,42953,2,1],[42955,42955,1,-42343],[42956,42970,2,1],[42972,42972,1,-42561],[42973,42973,1,-42342],[42978,42978,1,-42342],[42997,42997,1,1],[43884,43885,1,-33],[65313,65338,1,32],[66560,66599,1,40],[66736,66771,1,40],[66928,66938,1,39],[66940,66954,1,39],[66956,66962,1,39],[66964,66965,1,39],[68736,68786,1,64],[68944,68965,1,32],[71840,71871,1,32],[93760,93791,1,32],[93856,93880,1,27],[122688,122688,1,1],[122696,122698,2,1],[122701,122701,1,1],[122705,122705,1,1],[122728,122734,2,1],[122738,122750,2,1],[125184,125217,1,34]]}

13 - File: lib/take.php

Size

3275

SHA-256

93C58BA0CBEA1723091C18EC43491574B772C097D760B2877DE01735E0DEF7EC

Code

```
<?php

const V31NP_TYPES = ['html', 'htm', 'css', 'js', 'mjs', 'svg', 'png', 'jpg', 'jpeg', 'gif', 'webp', 'avif', 'ico', 'bmp', 'woff', 'woff2', 'ttf', 'otf',
    'mp3', 'wav', 'ogg', 'oga', 'opus', 'm4a', 'aac', 'flac', 'mp4', 'm4v', 'webm', 'ogv', 'mov', 'vtt'];

function v31np_types(array $settings): array
{
    $extra = array_map(fn($t) => strtolower(ltrim((string)$t, '.')), (array)($settings['types'] ?? []));
    return array_values(array_unique(array_merge(V31NP_TYPES, $extra)));
}

function v31np_rules(string $dir): array
{
    $text = @file_get_contents($dir . '/.htaccess');
    if ($text === false) {
        return [];
    }
    $rules = [];
    $block = null;
    foreach (preg_split('/\r?\n/', $text) as $raw) {
        $line = trim($raw);
        if ($line === '' || $line[0] === '#') {
            continue;
        }
        if (preg_match('/^<FilesMatch\s+"?([^">]+)"?\s*>$/i', $line, $m)) {
            $block = ['match', $m[1]];
            continue;
        }
        if (preg_match('/^<Files\s+"?([^">]+)"?\s*>$/i', $line, $m)) {
            $block = ['files', $m[1]];
            continue;
        }
        if (preg_match('/^<\/Files(Match)?>$/i', $line)) {
            $block = null;
            continue;
        }
        $verdict = null;
        if (preg_match('/^Require\s+all\s+denied$/i', $line) || preg_match('/^Deny\s+from\s+all$/i', $line)) {
            $verdict = true;
        } elseif (preg_match('/^Require\s+all\s+granted$/i', $line)) {
            $verdict = false;
        }
        if ($verdict !== null) {
            $rules[] = [$block === null ? 'all' : $block[0], $block[1] ?? '', $verdict];
            continue;
        }
        if ($block === null && preg_match('/^RedirectMatch\s+(403|404|410|gone)\s+"?([^"\s]+)"?/i', $line, $m)) {
            $rules[] = ['url', $m[2], true];
            continue;
        }
        if ($block === null && preg_match('/^RewriteRule\s+"?([^"\s]+)"?\s+-\s+\[([^\]]*)\]/i', $line, $m) && preg_match('/(^|,)\s*(F|G|forbidden|gone)\s*(,|$)/i', $m[2])) {
            $rules[] = ['rewrite', $m[1], true];
        }
    }
    return $rules;
}

function v31np_regex(string $pattern, string $subject): bool
{
    $r = @preg_match('~' . str_replace('~', '\~', $pattern) . '~u', $subject);
    return $r === 1;
}

function v31np_denied(string $root, string $key, array &$cache): bool
{
    $parts = explode('/', $key);
    $base = array_pop($parts);
    $denied = false;
    $dir = $root;
    $rel = '';
    for ($i = 0; $i <= count($parts); $i++) {
        if (!isset($cache[$dir])) {
            $cache[$dir] = v31np_rules($dir);
        }
        $below = substr($key, strlen($rel));
        foreach ($cache[$dir] as [$kind, $pattern, $verdict]) {
            $hit = match ($kind) {
                'all' => true,
                'match' => v31np_regex($pattern, $base),
                'files' => fnmatch($pattern, $base),
                'url' => v31np_regex($pattern, '/' . $key),
                'rewrite' => v31np_regex($pattern, $below),
            };
            if ($hit) {
                $denied = $verdict;
            }
        }
        if ($i < count($parts)) {
            $dir .= '/' . $parts[$i];
            $rel .= $parts[$i] . '/';
        }
    }
    return $denied;
}
```

14 - File: node/take.js

Size

2974

SHA-256

2038024821A8FE3F28BC90B6B4CA1B8F4866038CE592778DEBE2786CB141402F

Code

```
const fs = require('fs');
const path = require('path');

const TYPES = ['html', 'htm', 'css', 'js', 'mjs', 'svg', 'png', 'jpg', 'jpeg', 'gif', 'webp', 'avif', 'ico', 'bmp', 'woff', 'woff2', 'ttf', 'otf',
    'mp3', 'wav', 'ogg', 'oga', 'opus', 'm4a', 'aac', 'flac', 'mp4', 'm4v', 'webm', 'ogv', 'mov', 'vtt'];

const types = settings => [...new Set([...TYPES, ...[].concat(settings.types || []).map(t => String(t).replace(/^\./, '').toLowerCase())])];

const rulesOf = dir => {
    let text;
    try { text = fs.readFileSync(path.join(dir, '.htaccess'), 'utf8'); } catch { return []; }
    const rules = [];
    let block = null;
    for (const raw of text.split(/\r?\n/)) {
        const line = raw.trim();
        if (line === '' || line[0] === '#') continue;
        let m;
        if ((m = /^<FilesMatch\s+"?([^">]+)"?\s*>$/i.exec(line))) { block = ['match', m[1]]; continue; }
        if ((m = /^<Files\s+"?([^">]+)"?\s*>$/i.exec(line))) { block = ['files', m[1]]; continue; }
        if (/^<\/Files(Match)?>$/i.test(line)) { block = null; continue; }
        let verdict = null;
        if (/^Require\s+all\s+denied$/i.test(line) || /^Deny\s+from\s+all$/i.test(line)) verdict = true;
        else if (/^Require\s+all\s+granted$/i.test(line)) verdict = false;
        if (verdict !== null) { rules.push([block ? block[0] : 'all', block ? block[1] : '', verdict]); continue; }
        if (!block && (m = /^RedirectMatch\s+(403|404|410|gone)\s+"?([^"\s]+)"?/i.exec(line))) { rules.push(['url', m[2], true]); continue; }
        if (!block && (m = /^RewriteRule\s+"?([^"\s]+)"?\s+-\s+\[([^\]]*)\]/i.exec(line)) && /(^|,)\s*(F|G|forbidden|gone)\s*(,|$)/i.test(m[2])) rules.push(['rewrite', m[1], true]);
    }
    return rules;
};

const regex = (pattern, subject) => {
    let p = pattern, flags = 'u';
    if (p.includes('(?i)')) { p = p.split('(?i)').join(''); flags += 'i'; }
    try { return new RegExp(p, flags).test(subject); } catch { return false; }
};
const wild = (pattern, subject) => new RegExp('^' + pattern.replace(/[.+^${}()|\\]/g, '\\$&').replace(/\*/g, '.*').replace(/\?/g, '.') + '$').test(subject);

const denied = (root, key, cache) => {
    const parts = key.split('/');
    const base = parts.pop();
    let out = false, dir = root, rel = '';
    for (let i = 0; i <= parts.length; i++) {
        if (!cache.has(dir)) cache.set(dir, rulesOf(dir));
        const below = key.slice(rel.length);
        for (const [kind, pattern, verdict] of cache.get(dir)) {
            const hit = kind === 'all' ? true
                : kind === 'match' ? regex(pattern, base)
                : kind === 'files' ? wild(pattern, base)
                : kind === 'url' ? regex(pattern, '/' + key)
                : regex(pattern, below);
            if (hit) out = verdict;
        }
        if (i < parts.length) {
            dir = path.join(dir, parts[i]);
            rel += parts[i] + '/';
        }
    }
    return out;
};

module.exports = { types, denied };
```

15 - File: node/core.js

Size *23602*

SHA-256 *64ABE1EEEB954E13084BB5C0D2C097BF692FCE4B41904E499167CAFDEFC757C7*

Code

```
const fs = require('fs');
const fsp = require('fs/promises');
const path = require('path');
const vm = require('vm');
const crypto = require('crypto');

const HOME = path.resolve(__dirname, '..');
const ROOT = path.dirname(HOME);
const LIB = path.join(HOME, 'lib');
const PRE = path.join(HOME, 'pre');
const LIMIT = 64 * 1024 * 1024;
const NAME = /^[A-Za-z0-9_-]+$/;
const INDEX_MAGIC = 'V31NBAIMITIF\0\0\0';
const INDEX_KEY = 'v31np index.txt';
const INDEX = path.join(ROOT, INDEX_KEY);
const HEAD_NAME = /^[!#$%&'*+.^_`|~0-9A-Za-z-]+$/;
const LEVELS = ['S', 'A', 'B', 'Ú', 'U', 'Ü'];
const SHIFT = 11644473600n;
const TICKS = 10000000n;

const readLib = f => fs.readFileSync(path.join(LIB, f), 'utf8');
const { V31nPack: P, V31nOrder: O } = vm.runInThisContext(
  '(function () {\nconst V31N_TABLES = ' + readLib('tables.json') + ';\n' + readLib('order.js') + '\n' + readLib('pack.js') + '\nreturn { V31nPack, V31nOrder };\n}.)'
)();

const types = {
  mp3: 'audio/mpeg', mp4: 'video/mp4', webm: 'video/webm', wav: 'audio/wav', png: 'image/png', jpg: 'image/jpeg', jpeg: 'image/jpeg', gif: 'image/gif',
  svg: 'image/svg+xml', ico: 'image/x-icon', woff: 'font/woff', woff2: 'font/woff2', ttf: 'font/ttf', otf: 'font/otf', css: 'text/css; charset=utf-8',
  js: 'text/javascript; charset=utf-8', json: 'application/json', txt: 'text/plain; charset=utf-8'
};
const typeOf = key => {
  const ext = path.posix.extname(key).slice(1).toLowerCase();
  return types[ext] || 'application/octet-stream';
};

const settings = name => {
  let all = {};
  try { all = JSON.parse(fs.readFileSync(path.join(HOME, 'settings.json'), 'utf8')); } catch {}
  if (!all || typeof all !== 'object') all = {};
  const own = all.archives && typeof all.archives[name] === 'object' && all.archives[name] ? all.archives[name] : {};
  const rest = { ...all };
  delete rest.archives;
  return { empty: '268', behind: '268', send: ['S'], sign: '', ...rest, ...own };
};

const unescapeText = s => {
  let out = '';
  for (let i = 0; i < s.length; i++) {
    if (s[i] !== '\\') { out += s[i]; continue; }
    const e = { '\\': '\\', t: '\t', n: '\n', r: '\r' }[s[++i]];
    if (e === undefined) return null;
    out += e;
  }
  return out;
};
const escapeText = s => s.replace(/[\\\t\n\r]/g, c => ({ '\\': '\\\\', '\t': '\\t', '\n': '\\n', '\r': '\\r' })[c]);
const fileTime = st => (BigInt(Math.floor(st.mtimeMs / 1000)) + SHIFT) * TICKS;

const crcFile = async file => {
  const h = await fsp.open(file, 'r');
  try {
    const chunk = Buffer.alloc(1048576);
    let sum = 0, at = 0;
    for (;;) {
      const { bytesRead } = await h.read(chunk, 0, chunk.length, at);
      if (!bytesRead) break;
      sum = P.crc(chunk.subarray(0, bytesRead), sum);
      at += bytesRead;
    }
    return P.hex(sum);
  } finally {
    await h.close();
  }
};
const shaFile = async file => {
  const h = crypto.createHash('sha256');
  for await (const chunk of fs.createReadStream(file)) h.update(chunk);
  return h.digest('hex').toUpperCase();
};
const shaOf = b => crypto.createHash('sha256').update(b).digest('hex').toUpperCase();

const seedOf = () => {
  const s = settings('').sign || '';
  if (!s) return null;
  const seed = fs.readFileSync(path.join(HOME, s), 'utf8').trim();
  if (!/^[0-9A-F]{64}$/.test(seed)) throw new Error('sign key');
  return seed;
};
const secretKey = () => {
  const seed = seedOf();
  if (!seed) return null;
  return crypto.createPrivateKey({ key: Buffer.concat([Buffer.from('302e020100300506032b657004220420', 'hex'), Buffer.from(seed, 'hex')]), format: 'der', type: 'pkcs8' });
};
const publicHex = secret => secret ? crypto.createPublicKey(secret).export({ format: 'der', type: 'spki' }).subarray(-32).toString('hex').toUpperCase() : '';

const createIndex = (file, rootText) => ({ file, rootText, root: path.join(path.dirname(file), rootText), rows: new Map(), heads: [], dirty: true });
const loadIndex = file => {
  let text;
  try { text = fs.readFileSync(file, 'utf8'); } catch { return null; }
  let index = null, magic = false;
  for (const line of text.split(/\r?\n/)) {
    if (line === '') continue;
    if (!magic) {
      if (line !== INDEX_MAGIC) return null;
      magic = true;
      continue;
    }
    if (!index) {
      const root = unescapeText(line);
      if (!root) return null;
      index = createIndex(file, root);
      index.dirty = false;
      continue;
    }
    const f = line.split('\t');
    if (f[0] === 'F') {
      if (index.heads.length || f.length < 6 || f.length > 8) return null;
      const key = unescapeText(f[1]), p = unescapeText(f[4]);
      if (!key || !p || index.rows.has(key) || !/^\d+$/.test(f[2]) || !/^[0-9A-F]{8}$/.test(f[3]) || !/^\d+$/.test(f[5])) return null;
      let sha = null, level = null;
      for (const extra of f.slice(6)) {
        if (sha === null && level === null && /^[0-9A-F]{64}$/.test(extra)) sha = extra;
        else if (level === null && LEVELS.includes(extra)) level = extra;
        else return null;
      }
      index.rows.set(key, { size: Number(f[2]), crc: f[3], path: p, time: BigInt(f[5]), sha, level });
      continue;
    }
    if (f[0] === 'H' && f.length === 4) {
      const key = unescapeText(f[1]), n = unescapeText(f[2]), v = unescapeText(f[3]);
      if (key === null || !index.rows.has(key) || n === null || !HEAD_NAME.test(n) || v === null || /[\0\r\n]/.test(v)) return null;
      index.heads.push([key, n, v]);
      continue;
    }
    return null;
  }
  return index;
};
const freshRow = async (index, key, sha = false) => {
  const row = index.rows.get(key);
  if (!row) return null;
  const file = path.join(index.root, row.path);
```

```
    let st;
    try { st = await fsp.stat(file); } catch { return null; }
    if (!st.isFile()) return null;
    const time = fileTime(st);
    const changed = st.size !== row.size || time / TICKS !== row.time / TICKS;
    if (changed) {
      row.size = st.size;
      row.time = time;
      row.crc = await crcFile(file);
      index.dirty = true;
    }
    if (sha && (changed || !row.sha)) {
      row.sha = await shaFile(file);
      index.dirty = true;
    }
    return { ...row, file };
  };
  const headLines = (index, keys) => {
    let out = '';
    for (const key of keys) {
      for (const [k, n, v] of index.heads) {
        if (k === key) out += ['H', escapeText(key), escapeText(n), escapeText(v)].join('\t') + '\r\n';
      }
    }
    return out;
  };
  const saveIndex = async index => {
    if (!index.dirty) return true;
    const keys = [...index.rows.keys()].sort(O.key);
    let out = INDEX_MAGIC + '\r\n' + escapeText(index.rootText) + '\r\n';
    for (const key of keys) {
      const r = index.rows.get(key);
      const f = ['I', escapeText(key), r.size, r.crc, escapeText(r.path), r.time];
      if (r.sha) f.push(r.sha);
      if (r.level) f.push(r.level);
      out += f.join('\t') + '\r\n';
    }
    out += headLines(index, keys);
    const temp = index.file + '.' + crypto.randomBytes(16).toString('hex') + '.tmp';
    try {
      await fsp.writeFile(temp, out);
      await fsp.rename(temp, index.file);
    } catch {
      return false;
    }
    index.dirty = false;
    return true;
  };
  const indexEntry = (index, rows, secret) => {
    const keys = [...rows.keys()].sort(O.key);
    let out = INDEX_MAGIC + '\r\n';
    for (const key of keys) {
      const r = rows.get(key);
      const f = ['I', escapeText(key), r.size, r.crc];
      if (secret) f.push(r.sha);
      if (r.level) f.push(r.level);
      out += f.join('\t') + '\r\n';
    }
    out += headLines(index, keys);
    if (secret) out += 'S\t' + crypto.sign(null, Buffer.from(out), secret).toString('hex').toUpperCase() + '\r\n';
    return Buffer.from(out);
  };

  const end = (res, status, headers = {}, body) => {
    res.writeHead(status, headers);
    res.end(body);
  };
  const body = req => new Promise((resolve, reject) => {
    const parts = [];
    let n = 0;
    req.on('data', c => {
      n += c.length;
      if (n > LIMIT) { reject(new Error('large')); req.destroy(); return; }
      parts.push(c);
    });
    req.on('end', () => resolve(Buffer.concat(parts)));
    req.on('error', reject);
  });
  const askOf = buf => {
    const outer = P.parse(P.MAGIC, buf);
    if (!outer || outer.entries.length !== 1 || outer.entries[0].key !== '') return null;
    const e = outer.entries[0];
    const inner = buf.subarray(e.at, e.at + e.size);
    const ask = P.parse(P.ASK, inner);
    if (!ask) return null;
    const client = new Map();
    for (const x of ask.entries) {
      const text = Buffer.from(inner.subarray(x.at, x.at + x.size)).toString('latin1');
      if (x.size !== 8 || !/^[0-9A-F]{8}$/.test(text)) return null;
      client.set(x.key, text);
    }
    return client;
  };
  const view = b => Buffer.from(b.buffer, b.byteOffset, b.length);

  const ranges = (req, res, file, base, total, type, etag, heads = []) => {
    const headers = { 'Cache-Control': 'no-store', ETag: '"' + etag + '"' };
    const extra = heads.map(([n, v]) => [n, v]);
    const write = (status, h, reason) => {
      const all = { ...headers, ...h };
      const list = [...Object.entries(all), ...extra].flat().map(String);
      if (reason) res.writeHead(status, reason, list);
      else res.writeHead(status, list);
    };
    const pipe = (start, stop) => new Promise((ok, fail) => {
      const s = fs.createReadStream(file, { start: base + start, end: base + stop });
      s.on('end', ok);
      s.on('error', fail);
      s.pipe(res, { end: false });
    });
    const head = req.method === 'HEAD';
    const range = String(req.headers.range || '').trim();
    const ifRange = String(req.headers['if-range'] || '').trim();
    if (range === '' || (ifRange !== '' && ifRange !== '"' + etag + '"')) {
      write(200, { 'Content-Type': type, 'Accept-Ranges': 'bytes', 'Content-Length': total });
      if (head || total === 0) return res.end();
      return pipe(0, total - 1).then(() => res.end(), () => res.destroy());
    }
    if (!range.startsWith('bytes=')) { write(400, {}); return res.end(); }
    const parts = [];
    for (const spec of range.slice(6).split(',')) {
      const m = /^(\d*)-(\d*)$/.exec(spec.trim());
      if (!m || (m[1] === '' && m[2] === '')) { write(400, {}); return res.end(); }
      let start, stop;
      if (m[1] === '') { start = Math.max(0, total - Number(m[2])); stop = total - 1; }
      else { start = Number(m[1]); stop = m[2] === '' ? total - 1 : Math.min(Number(m[2]), total - 1); }
      if (start < total && start <= stop && (m[1] !== '' || Number(m[2]) > 0)) parts.push([start, stop]);
    }
    if (!parts.length) { write(416, { 'Content-Range': 'bytes */' + total }); return res.end(); }
    if (parts.length === 1) {
      const [start, stop] = parts[0];
      write(206, { 'Accept-Ranges': 'bytes', 'Content-Type': type, 'Content-Range': 'bytes ' + start + '-' + stop + '/' + total, 'Content-Length': stop - start + 1 });
      if (head) return res.end();
      return pipe(start, stop).then(() => res.end(), () => res.destroy());
    }
    const boundary = 'v31np-' + etag;
    const lines = parts.map(([start, stop]) => '--' + boundary + '\r\nContent-Type: ' + type + '\r\nContent-Range: bytes ' + start + '-' + stop + '/' + total + '\r\n\r\n');
    const closing = '--' + boundary + '--\r\n';
    const length = parts.reduce((n, [start, stop], i) => n + Buffer.byteLength(lines[i]) + (stop - start + 1) + 2, 0) + closing.length;
    write(206, { 'Accept-Ranges': 'bytes', 'Content-Type': 'multipart/byteranges; boundary=' + boundary, 'Content-Length': length });
    if (head) return res.end();
    return (async () => {
      for (let i = 0; i < parts.length; i++) {
        res.write(lines[i]);
        await pipe(parts[i][0], parts[i][1]);
        res.write('\r\n');
      }
      res.end(closing);
    })().catch(() => res.destroy());
  };

  const archive = name => {
    if (!NAME.test(name)) return null;
    const file = path.join(PRE, name + '.v31np');
    const meta = path.join(PRE, name + '.json');
    if (!fs.existsSync(file) || !fs.existsSync(meta)) return null;
    return { file, info: JSON.parse(fs.readFileSync(meta, 'utf8')) };
  };
  const indexBytes = a => {
    const f = a.info.files.find(x => x.key === INDEX_KEY);
    if (!f) return null;
    const fd = fs.openSync(a.file, 'r');
    const bytes = Buffer.alloc(f.size);
```

```
    try { fs.readSync(fd, bytes, 0, f.size, f.at); } finally { fs.closeSync(fd); }
    return bytes;
};

const exchange = async (req, res, name, query) => {
    const a = archive(name);
    const base = { 'Cache-Control': 'no-store' };
    if (!a) return end(res, 404, base);
    base.ETag = "" + a.info.crc + "";
    const set = settings(name);

    const member = query.get('key') || '';
    if (member !== '') {
        const entry = a.info.files.find(f => f.key === member);
        if (!entry || member === INDEX_KEY) return end(res, 404, base);
        if (req.method !== 'GET' && req.method !== 'HEAD') return end(res, 405, { ...base, Allow: 'GET, HEAD' });
        const heads = [];
        const ix = indexBytes(a);
        if (ix) {
            for (const line of ix.toString('utf8').split(/\r?\n/)) {
                const f = line.split('\t');
                if (f[0] === 'H' && f.length === 4 && unescapeText(f[1]) === member) heads.push([unescapeText(f[2]), unescapeText(f[3])]);
            }
        }
        return ranges(req, res, a.file, entry.at, entry.size, typeOf(member), entry.crc, heads);
    }
    if (req.method === 'GET' || req.method === 'HEAD') return ranges(req, res, a.file, 0, fs.statSync(a.file).size, 'application/v31np', a.info.crc);
    if (req.method !== 'POST') return end(res, 405, { ...base, Allow: 'GET, HEAD, POST' });
    if (String(req.headers['content-type'] || '').split(';')[0].trim().toLowerCase() !== 'application/v31np') return end(res, 415, base);
    if (Number(req.headers['content-length'] || 0) > LIMIT) return end(res, 413, { ...base, Connection: 'close' });

    let client;
    try { client = askOf(await body(req)); } catch (e) { if (e && e.message === 'large') return end(res, 413, { ...base, Connection: 'close' }); client = null; }
    if (!client) return end(res, 422, base);

    const full = () => {
        const size = fs.statSync(a.file).size;
        res.writeHead(200, { ...base, 'Content-Type': 'application/v31np', 'Accept-Ranges': 'bytes', 'Content-Length': size });
        fs.createReadStream(a.file).pipe(res);
    };
    const list = () => {
        const bytes = indexBytes(a);
        if (!bytes) return end(res, 500, base);
        const answer = view(P.write(P.MAGIC, [{ key: INDEX_KEY, bytes }]).buf);
        res.writeHead(268, 'List', { ...base, 'Content-Type': 'application/v31np', 'Content-Length': answer.length });
        res.end(answer);
    };
    const server = new Map(a.info.files.map(f => [String(f.key), f]));
    if (client.size === 0) {
        if (set.empty === '268') return list();
        if (set.empty === '303') return end(res, 303, { ...base, Location: '/v31np/pre/' + name + '.v31np' });
        return full();
    }
    if (set.behind === '268') {
        const wanted = [...client.keys()].filter(k => client.get(k) === '00000000');
        if (wanted.length) {
            const own = server.get(INDEX_KEY);
            if (own && client.get(INDEX_KEY) !== own.crc && !wanted.includes(INDEX_KEY)) wanted.push(INDEX_KEY);
            wanted.sort(O.key);
            const items = wanted.map(key => ({ key, size: server.has(key) ? server.get(key).size + 1 : 1 }));
            const innerHead = P.head(P.PATCH, items);
            const innerLength = innerHead.length + items.reduce((n, x) => n + x.size, 0) + P.FOOT;
            const outerHead = P.head(P.MAGIC, [{ key: '', size: innerLength }]);
            res.writeHead(267, 'Re-sync', { ...base, 'Content-Type': 'application/v31np', 'Content-Length': outerHead.length + innerLength + P.FOOT });
            let outer = 0, inner = 0;
            const put = (b, both = true) => new Promise(resolve => {
                outer = P.crc(b, outer);
                if (both) inner = P.crc(b, inner);
                res.write(view(b)) ? resolve() : res.once('drain', resolve);
            });
            const word = v => { const b = Buffer.alloc(4); b.writeUInt32LE(v >>> 0, 0); return b; };
            await put(outerHead, false);
            await put(innerHead);
            const fd = await fsp.open(a.file, 'r');
            try {
                for (const key of wanted) {
                    const f = server.get(key);
                    if (!f) { await put(Buffer.from([1])); continue; }
                    let left = f.size, at = f.at;
                    while (left > 0) {
                        const chunk = Buffer.alloc(Math.min(1048576, left));
                        const { bytesRead } = await fd.read(chunk, 0, chunk.length, at);
                        if (!bytesRead) return res.destroy();
                        await put(chunk.subarray(0, bytesRead));
                        left -= bytesRead;
                        at += bytesRead;
                    }
                    await put(Buffer.from([0]));
                }
            } finally {
                await fd.close();
            }
            await put(P.PI);
            await put(word(inner), false);
            await put(P.PI, false);
            return res.end(word(outer));
        }
        const own = server.get(INDEX_KEY);
        if (own && client.get(INDEX_KEY) === own.crc) return end(res, 304, base);
        return list();
    }
    const changed = [...server.keys()].filter(k => client.get(k) !== server.get(k).crc);
    const deleted = [...client.keys()].filter(k => !server.has(k));
    if (!changed.length && !deleted.length) return end(res, 304, base);
    if (set.behind === '200') return full();
    const keys = [...changed, ...deleted].sort(O.key);
    const fd = await fsp.open(a.file, 'r');
    const entries = [];
    try {
        for (const key of keys) {
            const f = server.get(key);
            if (!f) { entries.push({ key, bytes: new Uint8Array([1]) }); continue; }
            const bytes = Buffer.alloc(f.size + 1);
            if (f.size > 0) await fd.read(bytes, 0, f.size, f.at);
            entries.push({ key, bytes });
        }
    } finally {
        await fd.close();
    }
    const answer = view(P.write(P.MAGIC, [{ key: '', bytes: P.write(P.PATCH, entries).buf }]).buf);
    res.writeHead(267, 'Re-sync', { ...base, 'Content-Type': 'application/v31np', 'Content-Length': answer.length });
    res.end(answer);
};

const sendAnswer = async (req, res, index, keys, page) => {
    const secret = secretKey();
    const rows = new Map();
    for (const key of keys) {
        if (key === '' || key === INDEX_KEY || rows.has(key)) continue;
        const r = await freshRow(index, key, !!secret);
        if (r) rows.set(key, r);
    }
    await saveIndex(index);
    const pageBytes = Buffer.from(page);
    const listed = new Map([['', { size: pageBytes.length, crc: P.hex(P.crc(pageBytes)), sha: shaOf(pageBytes) }], ...rows]);
    const mem = new Map([['', pageBytes], [INDEX_KEY, indexEntry(index, listed, secret)]]);
    const all = [...mem.keys(), ...rows.keys()].sort(O.key);
    const server = new Map(all.map(key => [key, mem.has(key)
        ? { size: mem.get(key).length, crc: P.hex(P.crc(mem.get(key))) }
        : { size: rows.get(key).size, crc: rows.get(key).crc }]));
    const items = all.map(key => ({ key, size: server.get(key).size }));
    const header = P.head(P.MAGIC, items);
    const length = header.length + items.reduce((n, x) => n + x.size, 0) + P.FOOT;
    let whole = P.crc(header);
    for (const key of all) whole = P.combine(whole, parseInt(server.get(key).crc, 16), server.get(key).size);
    whole = P.combine(whole, P.crc(P.PI), P.PI.length);
    const base = { 'Cache-Control': 'no-store', Vary: 'Accept, Content-Type', ETag: "" + P.hex(whole) + "" };

    if (req.method === 'POST') {
        let client;
        try { client = askOf(await body(req)); } catch { client = null; }
        if (!client) return end(res, 422, base);
        if (client.size) {
            const changed = all.filter(k => client.get(k) !== server.get(k).crc);
            const deleted = [...client.keys()].filter(k => !server.has(k));
            if (!changed.length && !deleted.length) return end(res, 304, base);
            const entries = [];
            for (const key of [...changed, ...deleted].sort(O.key)) {
                if (!server.has(key)) { entries.push({ key, bytes: new Uint8Array([1]) }); continue; }
                if (mem.has(key)) { entries.push({ key, bytes: Buffer.concat([mem.get(key), Buffer.from([0])]) }); continue; }
                const size = server.get(key).size;
                const bytes = Buffer.alloc(size + 1);
                const h = await fsp.open(rows.get(key).file, 'r');
```

```
            try {
              const { bytesRead } = size > 0 ? await h.read(bytes, 0, size, 0) : { bytesRead: 0 };
              if (bytesRead !== size) return end(res, 500, base);
            } finally {
              await h.close();
            }
            entries.push({ key, bytes });
          }
          const answer = view(P.write(P.MAGIC, [{ key: '', bytes: P.write(P.PATCH, entries).buf }]).buf);
          res.writeHead(267, 'Re-sync', { ...base, 'Content-Type': 'application/v31np', 'Content-Length': answer.length });
          return res.end(answer);
        }
      } else if (req.method !== 'GET' && req.method !== 'HEAD') {
        return end(res, 405, { ...base, Allow: 'GET, HEAD, POST' });
      }

      res.writeHead(200, { ...base, 'Content-Type': 'application/v31np', 'Content-Length': length });
      if (req.method === 'HEAD') return res.end();
      const put = b => new Promise(resolve => res.write(b) ? resolve() : res.once('drain', resolve));
      let sum = P.crc(header);
      await put(view(header));
      for (const key of all) {
        if (mem.has(key)) {
          sum = P.crc(mem.get(key), sum);
          await put(mem.get(key));
          continue;
        }
        const r = rows.get(key);
        const h = await fsp.open(r.file, 'r');
        try {
          let left = r.size, at = 0;
          while (left > 0) {
            const chunk = Buffer.alloc(Math.min(1048576, left));
            const { bytesRead } = await h.read(chunk, 0, chunk.length, at);
            if (!bytesRead) return res.destroy();
            const part = chunk.subarray(0, bytesRead);
            sum = P.crc(part, sum);
            await put(part);
            left -= bytesRead;
            at += bytesRead;
          }
        } finally {
          await h.close();
        }
      }
      sum = P.crc(P.PI, sum);
      const tail = Buffer.alloc(P.FOOT);
      tail.set(P.PI);
      tail.writeUInt32LE(sum >>> 0, 12);
      res.end(tail);
    };

    module.exports = {
      P, O, HOME, ROOT, LIB, PRE, NAME, INDEX, INDEX_KEY, settings, readLib, secretKey, publicHex, createIndex, loadIndex, freshRow, saveIndex, indexEntry,
      crcFile, shaFile, fileTime, end, ranges, archive, exchange, sendAnswer
    };
```

16 - File: build.js

Size

6051

SHA-256

17FF4C79B97FB825AB5746DAF562FEC93C23CE168A35BCFF08F60C3EBE89F22A

Code

```
const fs = require('fs');
const fsp = require('fs/promises');
const path = require('path');
const crypto = require('crypto');
const core = require('./node/core.js');
const take = require('./node/take.js');
const { P, O, INDEX_KEY } = core;

const glob = pattern => {
  let out = '';
  for (let i = 0; i < pattern.length; i++) {
    const c = pattern[i];
    if (c === '*') out += '[^/]*';
    else if (c === '?') out += '[^/]';
    else if (c === '[') {
      const close = pattern.indexOf(']', i + 2);
      if (close < 0) { out += '\\['; continue; }
      let set = pattern.slice(i + 1, close);
      if (set[0] === '!') set = '^' + set.slice(1);
      out += '[' + set.replace(/\\/g, '\\\\') + ']';
      i = close;
    } else out += c.replace(/[.+^${}()|\\/]/g, '\\$&');
  }
  return new RegExp('^' + out + '$');
};

const walk = async (dir, rel, found) => {
  for (const e of await fsp.readdir(dir, { withFileTypes: true })) {
    const full = path.join(dir, e.name);
    const key = rel ? rel + '/' + e.name : e.name;
    if (e.isDirectory()) await walk(full, key, found);
    else if (e.isFile() || (e.isSymbolicLink() && (await fsp.stat(full).catch(() => null))?.isFile())) found.push(key);
  }
};

(async () => {
  const name = process.argv[2] || '';
  const skip = process.argv.slice(3).map(glob);
  if (!core.NAME.test(name)) {
    process.stderr.write('usage: node v31np/build.js <name> [skip pattern ...]\n');
    process.exit(1);
  }
  const secret = core.secretKey();

  const js = 'const V31N_TABLES = ' + core.readLib('tables.json').trim() + ';\n'
    + "const V31N_KEY = (k => /^[0-9A-Fa-f]{64}$/.test(k) ? k.toUpperCase() : '" + core.publicHex(secret) + "')(String((document.currentScript && document.currentScript.dataset.key) || ''));\n"
    + ['sha256.js', 'order.js', 'pack.js', 'sync.js', 'show.js'].map(core.readLib).join('');
  const jsFile = path.join(core.HOME, 'v31np.js');
  const jsTemp = jsFile + '.' + crypto.randomBytes(8).toString('hex') + '.tmp';
  await fsp.writeFile(jsTemp, js);
  await fsp.rename(jsTemp, jsFile);

  let index = core.loadIndex(core.INDEX);
  if (!index) {
    if (fs.existsSync(core.INDEX)) {
      process.stderr.write('broken index\n');
      process.exit(1);
    }
    index = core.createIndex(core.INDEX, '.');
  }
  const types = take.types(core.settings(name));
  const rules = new Map();
  const ok = key => !(key === INDEX_KEY || key.startsWith('v31np/') || key.split('/').includes('node_modules') || !types.includes(path.posix.extname(key).slice(1).toLowerCase())
    || take.denied(index.root, key, rules) || skip.some(r => r.test(key)));
  const found = [];
  await walk(index.root, '', found);
  const seen = new Set();
  for (const rel of found) {
    if (!ok(rel)) continue;
    const key = '/' + rel;
    seen.add(key);
    if (!index.rows.has(key)) {
      index.rows.set(key, { size: -1, crc: '00000000', path: rel, time: 0n, sha: null, level: null });
      index.dirty = true;
    }
  }
  for (const key of [...index.rows.keys()]) {
    if (!seen.has(key)) {
      index.rows.delete(key);
      index.dirty = true;
    } else await core.freshRow(index, key, !!secret);
  }
  const kept = index.heads.filter(h => index.rows.has(h[0]));
  if (kept.length !== index.heads.length) {
    index.heads = kept;
    index.dirty = true;
  }
  if (!await core.saveIndex(index)) {
    process.stderr.write('index save failed\n');
    process.exit(1);
  }

  const rows = new Map();
  for (const key of index.rows.keys()) rows.set(key, await core.freshRow(index, key, !!secret));
  const entry = core.indexEntry(index, rows, secret);
  const keys = [...index.rows.keys(), INDEX_KEY].sort(O.key);
  const head = P.head(P.MAGIC, keys.map(key => ({ key, size: key === INDEX_KEY ? entry.length : rows.get(key).size })));

  await fsp.mkdir(core.PRE, { recursive: true });
  const target = path.join(core.PRE, name + '.v31np');
  const temp = target + '.' + crypto.randomBytes(8).toString('hex') + '.tmp';
  const out = await fsp.open(temp, 'w');
  let sum = 0;
  const put = async b => {
    sum = P.crc(b, sum);
    await out.write(Buffer.from(b.buffer, b.byteOffset, b.length));
  };
  const info = { files: [] };
  let at = head.length;
  await put(head);
  for (const key of keys) {
    let size, crc;
    if (key === INDEX_KEY) {
      await put(entry);
      size = entry.length;
      crc = P.hex(P.crc(entry));
    } else {
      const r = rows.get(key);
      size = r.size;
      crc = r.crc;
      const h = await fsp.open(r.file, 'r');
      try {
        let left = size, from = 0;
        while (left > 0) {
          const chunk = Buffer.alloc(Math.min(4194304, left));
          const { bytesRead } = await h.read(chunk, 0, chunk.length, from);
          if (!bytesRead) throw new Error('read ' + key);
          await put(chunk.subarray(0, bytesRead));
          left -= bytesRead;
          from += bytesRead;
        }
      } finally {
        await h.close();
      }
    }
    info.files.push({ key, at, size, crc });
    at += size;
  }
  await put(P.PI);
  const tail = Buffer.alloc(4);
  tail.writeUInt32LE(sum >>> 0, 0);
  await out.write(tail);
```

```
  await out.close();
  await fsp.rename(temp, target);
  const crcText = P.hex(sum);
  await fsp.writeFile(path.join(core.PRE, name + '.json'), JSON.stringify({ crc: crcText, ...info }, null, 4));
  console.log(name + '.v31np ' + (at + P.FOOT) + ' bytes, ' + keys.length + ' files, crc ' + crcText + (secret ? ', signed' : ''));
})().catch(e => {
  process.stderr.write(String(e && e.stack || e) + '\n');
  process.exit(1);
});
```

17 - File: build.php

Size

4258

SHA-256

1D25FCB58C2CB60EA58B3B06CFA76879BE5E432D78B995318C59C2C99DF8CC98

Code

```
<?php
if (PHP_SAPI !== 'cli') {
    http_response_code(404);
    exit;
}
require __DIR__ . '/lib/pack.php';
require __DIR__ . '/lib/index.php';
require __DIR__ . '/lib/settings.php';
require __DIR__ . '/lib/take.php';

$name = $argv[1] ?? '';
$skip = array_slice($argv, 2);
if (!preg_match('/^[A-Za-z0-9_-]+$/', $name)) {
    fwrite(STDERR, "usage: php v31np/build.php <name> [skip pattern ...]\n");
    exit(1);
}
$root = dirname(__DIR__);
$secret = V31nIndex::secret();

$lib = __DIR__ . '/lib/';
$key = $secret === null ? '' : strtoupper(bin2hex(sodium_crypto_sign_publickey_from_secretkey($secret)));
$js = 'const V31N_TABLES = ' . trim(file_get_contents($lib . 'tables.json')) . ';\n'
    . 'const V31N_KEY = (k => /^[0-9A-Fa-f]{64}$/.test(k) ? k.toUpperCase() : \'' . $key . '\')(String((document.currentScript && document.currentScript.dataset.key) || \'\'));' . "\n";
foreach (['sha256.js', 'order.js', 'pack.js', 'sync.js', 'show.js'] as $f) {
    $js .= file_get_contents($lib . $f);
}
$temp = __DIR__ . '/v31np.js.' . bin2hex(random_bytes(8)) . '.tmp';
if (file_put_contents($temp, $js) !== strlen($js) || !rename($temp, __DIR__ . '/v31np.js')) {
    fwrite(STDERR, "v31np.js write failed\n");
    exit(1);
}

$file = $root . '/' . V31nIndex::KEY;
$index = V31nIndex::load($file) ?? (is_file($file) ? null : V31nIndex::create($file, '.'));
if ($index === null) {
    fwrite(STDERR, "broken index\n");
    exit(1);
}
$types = v31np_types(v31np_settings($name));
$rules = [];
$take = function (string $key) use ($skip, $types, $root, &$rules): bool {
    if ($key === V31nIndex::KEY || str_starts_with($key, 'v31np/') || in_array('node_modules', explode('/', $key), true) || !in_array(strtolower(pathinfo($key, PATHINFO_EXTENSION)), $types, true)
        || v31np_denied($root, $key, $rules)) {
        return false;
    }
    foreach ($skip as $pattern) {
        if (fnmatch($pattern, $key, FNM_PATHNAME)) {
            return false;
        }
    }
    return true;
};
$index->scan($take, $secret !== null);
if (!$index->save()) {
    fwrite(STDERR, "index save failed\n");
    exit(1);
}
$rows = [];
foreach ($index->keys() as $k) {
    $rows[$k] = $index->fresh($k, $secret !== null);
}
$entry = $index->entry($rows, $secret);
$keys = array_merge($index->keys(), [V31nIndex::KEY]);
usort($keys, [V31nOrder::class, 'key']);
$items = [];
foreach ($keys as $k) {
    $items[] = [$k, $k === V31nIndex::KEY ? strlen($entry) : $rows[$k]['size']];
}
$head = V31nPack::head(V31nPack::MAGIC, $items);

$pre = __DIR__ . '/pre/';
if (!is_dir($pre) && !mkdir($pre, 0777, true)) {
    fwrite(STDERR, "no pre folder\n");
    exit(1);
}
$target = $pre . $name . '.v31np';
$temp = $target . '.' . bin2hex(random_bytes(8)) . '.tmp';
$out = fopen($temp, 'wb');
$ctx = hash_init('crc32b');
$put = function (string $bytes) use ($out, $ctx): void {
    if (fwrite($out, $bytes) !== strlen($bytes)) {
        throw new RuntimeException('write');
    }
    hash_update($ctx, $bytes);
};
$info = ['files' => []];
$at = strlen($head);
$put($head);
foreach ($keys as $k) {
    if ($k === V31nIndex::KEY) {
        $put($entry);
        $size = strlen($entry);
        $crc = V31nPack::hex(crc32($entry));
    } else {
        $size = $rows[$k]['size'];
        $crc = $rows[$k]['crc'];
        $in = fopen($index->path($k), 'rb');
        $left = $size;
        while ($left > 0) {
            $chunk = fread($in, min(4194304, $left));
            if ($chunk === false || $chunk === '') {
                throw new RuntimeException('read ' . $k);
            }
            $put($chunk);
            $left -= strlen($chunk);
        }
        fclose($in);
    }
    $info['files'][] = ['key' => $k, 'at' => $at, 'size' => $size, 'crc' => $crc];
    $at += $size;
}
$put(V31nPack::PI);
$sum = hexdec(hash_final($ctx));
fwrite($out, pack('V', $sum));
fclose($out);
if (!rename($temp, $target)) {
    fwrite(STDERR, "rename failed\n");
    exit(1);
}
$info = ['crc' => V31nPack::hex($sum)] + $info;
file_put_contents($pre . $name . '.json', json_encode($info, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT));
echo $name, ' .v31np ', $at + V31nPack::FOOT, ' bytes, ', count($keys), ' files, crc ', $info['crc'], $secret !== null ? ', signed' : '', "\n";
```

18 - File: v31np.js

Size

68322

SHA-256

D2729ACB71B9E46B855312DFEC66D56DFF69881DDF782D21F255C7ECD90F6FD6

Code

```
const V31N_TABLES =
{"ranges":[[9,13,true],[32,32,true],[33,47,false],[58,64,false],[91,96,false],[123,126,false],[133,133,true],[160,160,true],[161,191,false],[215,215,false],[247,247,false],[894,894,false],[903,903,false],[1370,1375,false],[1417,1418,false],[1470,1470,false],[1472,1472,false],[1475,1475,false],[1478,1478,false],[1523,1524,false],[1545,1546,false],[1548,1549,false],[1563,1563,false],[1566,1567,false],[1642,1645,false],[1748,1748,false],[1792,1805,false],[2039,2041,false],[2404,2405,false],[2416,2416,false],[3572,3572,false],[3663,3663,false],[3674,3675,false],[3844,3858,false],[3898,3901,false],[3973,3973,false],[4048,4052,false],[4170,4175,false],[4347,4347,false],[4961,4968,false],[5741,5742,false],[5760,5760,true],[5787,5788,false],[5867,5869,false],[5941,5942,false],[6100,6102,false],[6104,6106,false],[6144,6154,false],[6158,6158,true],[6468,6469,false],[6622,6623,false],[6686,6687,false],[7002,7008,false],[7227,7231,false],[7294,7295,false],[8192,8202,true],[8208,8231,false],[8232,8233,true],[8239,8239,true],[8240,8259,false],[8261,8273,false],[8275,8286,false],[8287,8287,true],[8317,8318,false],[8333,8334,false],[9001,9002,false],[10088,10101,false],[10181,10182,false],[10214,10223,false],[10627,10648,false],[10712,10715,false],[10748,10749,false],[11513,11516,false],[11518,11519,false],[11776,11822,false],[11824,11824,false],[12288,12288,true],[12289,12291,false],[12296,12305,false],[12308,12319,false],[12336,12336,false],[12349,12349,false],[12448,12448,false],[12539,12539,false],[42509,42511,false],[42611,42611,false],[42622,42622,false],[43124,43127,false],[43214,43215,false],[43310,43311,false],[43359,43359,false],[43612,43615,false],[62827,62828,false],[64830,64831,false],[65040,65049,false],[65072,65106,false],[65108,65121,false],[65123,65123,false],[65128,65128,false],[65130,65131,false],[65279,65279,true],[65281,65283,false],[65285,65290,false],[65292,65295,false],[65306,65307,false],[65311,65312,false],[65339,65341,false],[65343,65343,false],[65371,65371,false],[65373,65373,false],[65375,65381,false]],"slots":[[48,0,0],[49,1,0],[50,2,0],[51,3,0],[52,4,0],[53,5,0],[54,6,0],[55,7,0],[56,8,0],[57,9,0],[97,10,0],[1072,10,1],[98,11,0],[1073,11,1],[250,12,0],[118,12,1],[1074,12,2],[117,13,0],[1091,13,1],[249,14,0],[119,14,1],[103,15,0],[1075,15,1],[100,16,0],[1076,16,1],[254,16,2],[101,17,0],[1077,17,1],[1101,17,2],[380,18,0],[1106,18,1],[1078,18,2],[122,19,0],[1079,19,1],[105,20,0],[1080,20,1],[237,21,0],[121,21,1],[1081,21,2],[107,22,0],[1082,22,1],[108,23,0],[1083,23,1],[109,24,0],[1084,24,1],[110,25,0],[1085,25,1],[111,26,0],[1086,26,1],[112,27,0],[1087,27,1],[114,28,0],[1088,28,1],[115,29,0],[1089,29,1],[116,30,0],[1090,30,1],[102,31,0],[1092,31,1],[539,32,0],[1094,32,1],[269,33,0],[1095,33,1],[537,34,0],[1096,34,1],[99,35,0],[113,36,0],[120,37,0],[104,38,0],[1093,38,1],[8216,38,2]],"lower":[[65,90,1,32],[192,214,1,32],[216,222,1,32],[256,302,2,1],[304,304,1,-199],[306,310,2,1],[313,327,2,1],[330,374,2,1],[376,376,1,-121],[377,381,2,1],[385,385,1,210],[386,388,2,1],[390,390,1,206],[391,391,1,1],[393,394,1,205],[395,395,1,1],[398,398,1,79],[399,399,1,202],[400,400,1,203],[401,401,1,1],[403,403,1,205],[404,404,1,207],[406,406,1,211],[407,407,1,209],[408,408,1,1],[412,412,1,211],[413,413,1,213],[415,415,1,214],[416,420,2,1],[422,422,1,218],[423,423,1,1],[425,425,1,218],[428,428,1,1],[430,430,1,218],[431,431,1,1],[433,434,1,217],[435,437,2,1],[439,439,1,219],[440,440,1,1],[444,444,1,1],[452,452,1,2],[453,453,1,1],[455,455,1,2],[456,456,1,1],[458,458,1,2],[459,475,2,1],[478,494,2,1],[497,497,1,2],[498,500,2,1],[502,502,1,-97],[503,503,1,-56],[504,542,2,1],[544,544,1,-130],[546,562,2,1],[570,570,1,10795],[571,571,1,1],[573,573,1,-163],[574,574,1,10792],[577,577,1,1],[579,579,1,-195],[580,580,1,69],[581,581,1,71],[582,590,2,1],[880,882,2,1],[886,886,1,1],[895,895,1,116],[902,902,1,38],[904,906,1,37],[908,908,1,64],[910,911,1,63],[913,929,1,32],[931,939,1,32],[975,975,1,8],[984,1006,2,1],[1012,1012,1,-60],[1015,1015,1,1],[1017,1017,1,-7],[1018,1018,1,1],[1021,1023,1,-130],[1024,1039,1,80],[1040,1071,1,32],[1120,1152,2,1],[1162,1214,2,1],[1216,1216,1,15],[1217,1229,2,1],[1232,1326,2,1],[1329,1366,1,48],[4256,4293,1,7264],[4295,4295,1,7264],[4301,4301,1,7264],[5024,5103,1,38864],[5104,5109,1,8],[7305,7305,1,1],[7312,7354,1,-3008],[7357,7359,1,-3008],[7680,7828,2,1],[7838,7838,1,-7615],[7840,7934,2,1],[7944,7951,1,-8],[7960,7965,1,-8],[7976,7983,1,-8],[7992,7999,1,-8],[8008,8013,1,-8],[8025,8031,2,-8],[8040,8047,1,-8],[8072,8079,1,-8],[8088,8095,1,-8],[8104,8111,1,-8],[8120,8121,1,-8],[8122,8123,1,-74],[8124,8124,1,-9],[8136,8139,1,-86],[8140,8140,1,-9],[8152,8153,1,-8],[8154,8155,1,-100],[8168,8169,1,-8],[8170,8171,1,-112],[8172,8172,1,-7],[8184,8185,1,-128],[8186,8187,1,-126],[8188,8188,1,-9],[8486,8486,1,-7517],[8490,8490,1,-8383],[8491,8491,1,-8262],[8498,8498,1,28],[8544,8559,1,16],[8579,8579,1,1],[9398,9423,1,26],[11264,11311,1,48],[11360,11360,1,1],[11362,11362,1,-10743],[11363,11363,1,-3814],[11364,11364,1,-10727],[11367,11371,2,1],[11373,11373,1,-10780],[11374,11374,1,-10749],[11375,11375,1,-10783],[11376,11376,1,-10782],[11378,11378,1,1],[11381,11381,1,1],[11390,11391,1,-10815],[11392,11490,2,1],[11499,11501,2,1],[11506,11506,1,1],[42560,42604,2,1],[42624,42650,2,1],[42786,42798,2,1],[42802,42862,2,1],[42873,42875,2,1],[42877,42877,1,-35332],[42878,42886,2,1],[42891,42891,1,1],[42893,42893,1,-42280],[42896,42898,2,1],[42902,42920,2,1],[42922,42922,1,-42308],[42923,42923,1,-42319],[42924,42924,1,-42315],[42925,42925,1,-42305],[42926,42926,1,-42308],[42928,42928,1,-42258],[42929,42929,1,-42282],[42930,42930,1,-42261],[42931,42931,1,928],[42932,42946,2,1],[42948,42948,1,-48],[42949,42949,1,-42307],[42950,42950,1,-35384],[42951,42953,2,1],[42955,42955,1,-42343],[42956,42970,2,1],[42972,42972,1,-42561],[42973,42973,1,-42342],[42978,42978,1,-42342],[42997,42997,1,1],[43884,43885,1,-33],[65313,65338,1,32],[66560,66599,1,40],[66736,66771,1,40],[66928,66938,1,39],[66940,66954,1,39],[66956,66962,1,39],[66964,66965,1,39],[68736,68786,1,64],[68944,68965,1,32],[71840,71871,1,32],[93760,93791,1,32],[93856,93883,1,27],[122688,122688,1,1],[122696,122698,2,1],[122701,122701,1,1],[122705,122705,1,1],[122728,122734,2,1],[122738,122750,2,1],[125184,125217,1,34]]};
const V31N_KEY = (k => /^[0-9A-Fa-f]{64}$/.test(k) ? k.toUpperCase() : '')(String((document.currentScript && document.currentScript.dataset.key) || ''));
const V31nSha256 = (() => {
  const K = new Uint32Array([
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
  ]);

  const create = () => {
    const h = new Uint32Array([0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19]);
    const w = new Uint32Array(64);
    const block = new Uint8Array(64);
    let fill = 0, total = 0;

    const compress = (b, o) => {
      for (let i = 0; i < 16; i++) w[i] = (b[o + 4 * i] << 24) | (b[o + 4 * i + 1] << 16) | (b[o + 4 * i + 2] << 8) | b[o + 4 * i + 3];
      for (let i = 16; i < 64; i++) {
        const x = w[i - 15], y = w[i - 2];
        const s0 = ((x >>> 7) | (x << 25)) ^ ((x >>> 18) | (x << 14)) ^ (x >>> 3);
        const s1 = ((y >>> 17) | (y << 15)) ^ ((y >>> 19) | (y << 13)) ^ (y >>> 10);
        w[i] = (w[i - 16] + s0 + w[i - 7] + s1) | 0;
      }
      let a = h[0], bb = h[1], c = h[2], d = h[3], e = h[4], f = h[5], g = h[6], hh = h[7];
      for (let i = 0; i < 64; i++) {
        const S1 = ((e >>> 6) | (e << 26)) ^ ((e >>> 11) | (e << 21)) ^ ((e >>> 25) | (e << 7));
        const t1 = (hh + S1 + ((e & f) ^ (~e & g)) + K[i] + w[i]) | 0;
        const S0 = ((a >>> 2) | (a << 30)) ^ ((a >>> 13) | (a << 19)) ^ ((a >>> 22) | (a << 10));
        const t2 = (S0 + ((a & bb) ^ (a & c) ^ (bb & c))) | 0;
        hh = g; g = f; f = e; e = (d + t1) | 0; d = c; c = bb; bb = a; a = (t1 + t2) | 0;
      }
      h[0] += a; h[1] += bb; h[2] += c; h[3] += d; h[4] += e; h[5] += f; h[6] += g; h[7] += hh;
    };

    const update = data => {
      let i = 0;
      total += data.length;
      if (fill) {
        while (fill < 64 && i < data.length) block[fill++] = data[i++];
        if (fill < 64) return;
        compress(block, 0);
        fill = 0;
      }
      for (; i + 64 <= data.length; i += 64) compress(data, i);
      while (i < data.length) block[fill++] = data[i++];
    };

    const hex = () => {
      const bits = total * 8;
      const tail = new Uint8Array(fill < 56 ? 64 : 128);
      tail.set(block.subarray(0, fill));
      tail[fill] = 0x80;
      const view = new DataView(tail.buffer);
      view.setUint32(tail.length - 8, Math.floor(bits / 4294967296));
      view.setUint32(tail.length - 4, bits >>> 0);
      for (let o = 0; o < tail.length; o += 64) compress(tail, o);
      return [...h].map(v => v.toString(16).toUpperCase().padStart(8, '0')).join('');
    };

    return { update, hex };
  };

  return { create };
})();
const V31nOrder = (tables => {
  const ranges = tables.ranges;
  const slots = new Map(tables.slots.map(([c, rank, place]) => [c, [rank, place]]));

  const range = c => {
    let lo = 0, hi = ranges.length - 1, hit = -1;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      if (ranges[mid][0] <= c) { hit = mid; lo = mid + 1; } else hi = mid - 1;
    }
    return hit >= 0 && c <= ranges[hit][1] ? ranges[hit] : null;
  };
  const space = c => { const r = range(c); return r !== null && r[2]; };
  const loose = c => range(c) !== null;
  const points = s => Array.from(s, ch => ch.codePointAt(0));
  const text = p => String.fromCodePoint(...p);
  const shifts = tables.lower;
  const lowerCase = c => {
    let lo = 0, hi = shifts.length - 1, hit = -1;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      if (shifts[mid][0] <= c) { hit = mid; lo = mid + 1; } else hi = mid - 1;
    }
    if (hit < 0) return c;
    const [first, last, step, delta] = shifts[hit];
    return c > last || (c - first) % step !== 0 ? c : c + delta;
  };

  const fold = name => {
    let t = points(name);
    let i = 0;
    while (i < t.length && space(t[i])) i++;
    if (i < t.length && t[i] === 0x5B) {
      const close = t.indexOf(0x5D, i + 1);
      if (close !== -1 && close > i + 1) t = t.slice(i + 1, close);
    }
    i = 0;
    while (i < t.length && loose(t[i])) i++;
    t = t.slice(i);

    const d = points(text(t).normalize('NFD'));
    const doubled = [];
```

```
    for (let k = 0; k < d.length; k++) {
      if (d[k] !== 0x0A && k + 1 < d.length && d[k + 1] === 0x308) {
        doubled.push(d[k], d[k]);
        k++;
      } else doubled.push(d[k]);
    }
    t = points(text(doubled).normalize('NFC'));

    const numbered = [];
    for (let k = 0; k < t.length;) {
      if (t[k] === 0x2116) {
        let j = k + 1;
        while (j < t.length && space(t[j])) j++;
        const start = j;
        while (j < t.length && t[j] >= 0x30 && t[j] <= 0x39) j++;
        if (j > start) {
          let lead = start;
          while (lead + 1 < j && t[lead] === 0x30) lead++;
          const digits = t.slice(lead, j);
          numbered.push(0x2116);
          for (let p = digits.length; p < 5; p++) numbered.push(0x30);
          numbered.push(...digits);
          k = j;
          continue;
        }
      }
      numbered.push(t[k]);
      k++;
    }
    return numbered.map(lowerCase);
  };

  const stem = name => {
    const dot = name.lastIndexOf('.');
    return dot === -1 || dot === 0 ? name : name.slice(0, dot);
  };

  const rank = (s1, s2) => {
    const min = Math.min(s1.length, s2.length);
    for (let i = 0; i < min; i++) {
      const c1 = s1[i], c2 = s2[i];
      if (c1 === c2) continue;
      const r1 = slots.get(c1), r2 = slots.get(c2);
      if (r1 && r2) {
        if (r1[0] !== r2[0]) return r1[0] < r2[0] ? -1 : 1;
      } else if (r1) return -1;
      else if (r2) return 1;
      else return c1 < c2 ? -1 : 1;
    }
    return s1.length === s2.length ? 0 : s1.length < s2.length ? -1 : 1;
  };

  const place = (s1, s2) => {
    const min = Math.min(s1.length, s2.length);
    for (let i = 0; i < min; i++) {
      if (s1[i] === s2[i]) continue;
      const r1 = slots.get(s1[i]), r2 = slots.get(s2[i]);
      if (r1 && r2 && r1[1] !== r2[1]) return r1[1] < r2[1] ? -1 : 1;
      return 0;
    }
    return 0;
  };

  const capital = c => lowerCase(c) !== c;

  const cases = (s1, s2) => {
    const min = Math.min(s1.length, s2.length);
    for (let i = 0; i < min; i++) {
      const c1 = capital(s1[i]), c2 = capital(s2[i]);
      if (c1 !== c2) return c1 ? -1 : 1;
    }
    return 0;
  };

  const name = (a, b) => {
    const stemA = fold(stem(a)), stemB = fold(stem(b));
    let c = rank(stemA, stemB);
    if (c !== 0) return c;
    const foldA = fold(a), foldB = fold(b);
    c = rank(foldA, foldB);
    if (c !== 0) return c;
    c = place(stemA, stemB);
    if (c !== 0) return c;
    c = place(foldA, foldB);
    if (c !== 0) return c;
    const pa = points(a), pb = points(b);
    c = cases(pa, pb);
    if (c !== 0) return c;
    if (pa.length !== pb.length) return pa.length < pb.length ? -1 : 1;
    for (let i = 0; i < pa.length; i++) if (pa[i] !== pb[i]) return pa[i] < pb[i] ? -1 : 1;
    return 0;
  };

  const key = (a, b) => {
    const pa = a.split('/'), pb = b.split('/');
    const n = Math.min(pa.length, pb.length);
    for (let i = 0; i < n; i++) {
      const folderA = i < pa.length - 1, folderB = i < pb.length - 1;
      if (folderA !== folderB) return folderA ? -1 : 1;
      const c = name(pa[i], pb[i]);
      if (c !== 0) return c;
    }
    return pa.length === pb.length ? 0 : pa.length < pb.length ? -1 : 1;
  };

  return { name, key };
})(V31N_TABLES);
const V31nPack = (() => {
  const enc = new TextEncoder();
  const dec = new TextDecoder('utf-8', { fatal: true });
  const bytes = s => enc.encode(s);
  const MAGIC = bytes('V31NSPACKFORMAT');
  const ASK = bytes('V31NPIELALAIMIT');
  const PATCH = new Uint8Array([0x56, 0x33, 0x31, 0x4E, 0x32, 0x36, 0x37, 0, 0, 0, 0, 0, 0, 0]);
  const PI = new Uint8Array([0x24, 0x3F, 0x6A, 0x88, 0x85, 0xA3, 0x08, 0xD3, 0x13, 0x19, 0x8A, 0x2E]);
  const PHI = new Uint8Array([0x9E, 0x37, 0x79, 0xB9, 0x7F, 0x4A, 0x7C, 0x15, 0xF3, 0x9C, 0xC0, 0x60]);
  const MARK = 12, LEAD = 27, FOOT = 16, LIMIT = 45035996273704960;

  const table = new Uint32Array(256);
  for (let n = 0; n < 256; n++) {
    let c = n;
    for (let k = 0; k < 8; k++) c = (c & 1) ? (0xEDB88320 ^ (c >>> 1)) : (c >>> 1);
    table[n] = c >>> 0;
  }
  const crc = (data, prev = 0) => {
    let c = (prev ^ 0xFFFFFFFF) >>> 0;
    for (let i = 0; i < data.length; i++) c = table[(c ^ data[i]) & 0xFF] ^ (c >>> 8);
    return (c ^ 0xFFFFFFFF) >>> 0;
  };
  const hex = v => v.toString(16).toUpperCase().padStart(8, '0');
  const times = (mat, vec) => {
    let sum = 0;
    for (let i = 0; vec !== 0; i++, vec >>>= 1) if (vec & 1) sum = (sum ^ mat[i]) >>> 0;
    return sum;
  };
  const square = mat => mat.map(v => times(mat, v));
  const combine = (crc1, crc2, len2) => {
    if (len2 <= 0) return crc1;
    let odd = [0xEDB88320];
    for (let n = 1, row = 1; n < 32; n++, row = (row << 1) >>> 0) odd[n] = row;
    let even = square(odd);
    odd = square(even);
    for (;;) {
      even = square(odd);
      if (len2 % 2) crc1 = times(even, crc1);
      len2 = Math.floor(len2 / 2);
      if (len2 === 0) break;
      odd = square(even);
      if (len2 % 2) crc1 = times(odd, crc1);
      len2 = Math.floor(len2 / 2);
      if (len2 === 0) break;
    }
    return (crc1 ^ crc2) >>> 0;
  };

  const join = parts => {
    let n = 0;
    for (const p of parts) n += p.length;
    const out = new Uint8Array(n);
    let at = 0;
    for (const p of parts) { out.set(p, at); at += p.length; }
    return out;
  };
```

```
  const num = v => {
    const out = [];
    do { out.push(v % 256); v = Math.floor(v / 256); } while (v > 0);
    return new Uint8Array(out);
  };
  const sep = marker => join([new Uint8Array([marker]), PHI]);
  const split = key => {
    const slash = key.lastIndexOf('/');
    const base = slash === -1 ? 0 : slash + 1;
    const dot = key.lastIndexOf('.');
    const ext = dot === -1 || dot < base ? key.length : dot;
    return [key.slice(0, base), key.slice(base, ext), key.slice(ext)];
  };

  const head = (magic, items) => {
    const t2 = [];
    for (const { key, size } of items) {
      t2.push(sep(0), bytes(key), sep(1), num(size));
      for (const part of split(key)) t2.push(sep(1), bytes(part));
    }
    const t2b = join(t2);
    let start = 0, previous, header;
    do {
      previous = start;
      const parts = [magic, PI];
      let address = previous;
      for (const { key, size } of items) {
        parts.push(sep(0), bytes(key), sep(1), num(address));
        address += size;
      }
      parts.push(PI, t2b, PI);
      header = join(parts);
      start = header.length;
    } while (start !== previous);
    return header;
  };

  const archiveCrc = (magic, files) => {
    const header = head(magic, files.map(f => ({ key: f.key, size: f.bytes.length })));
    let sum = crc(header);
    for (const f of files) sum = crc(f.bytes, sum);
    return { header, crc: crc(PI, sum) };
  };

  const write = (magic, files) => {
    const { header, crc: c } = archiveCrc(magic, files);
    const tail = new Uint8Array(FOOT);
    tail.set(PI);
    new DataView(tail.buffer).setUint32(MARK, c, true);
    return { buf: join([header, ...files.map(f => f.bytes), tail]), crc: c };
  };

  const find = (buf, pat, from, to) => {
    if (to > buf.length) to = buf.length;
    outer: for (let i = from; i + pat.length <= to; i++) {
      for (let k = 0; k < pat.length; k++) if (buf[i + k] !== pat[k]) continue outer;
      return i;
    }
    return -1;
  };
  const same = (buf, at, pat) => {
    if (at + pat.length > buf.length) return false;
    for (let k = 0; k < pat.length; k++) if (buf[at + k] !== pat[k]) return false;
    return true;
  };

  const rows = (buf, from, to) => {
    const out = [];
    let open = false, at = from;
    for (;;) {
      const hit = find(buf, PHI, at, to);
      if (hit >= 0 && hit <= at) return null;
      const end = hit < 0 ? to : hit - 1;
      if (open) out[out.length - 1].push(buf.subarray(at, end));
      else if (end !== from) return null;
      if (hit < 0) return out;
      const marker = buf[hit - 1];
      if (marker === 0) { out.push([]); open = true; }
      else if (marker !== 1 || !open) return null;
      at = hit + MARK;
    }
  };
  const value = b => {
    if (b.length === 0 || b.length > 7) return null;
    let v = 0;
    for (let i = b.length - 1; i >= 0; i--) v = v * 256 + b[i];
    return v < LIMIT ? v : null;
  };
  const text = b => { try { return dec.decode(b); } catch { return null; } };

  const parse = (magic, buf) => {
    const total = buf.length;
    if (total < LEAD + 2 * MARK + FOOT) return null;
    if (!same(buf, 0, magic) || !same(buf, 15, PI)) return null;
    const body = total - FOOT;
    const b = find(buf, PI, LEAD, body);
    if (b < 0) return null;
    const c = find(buf, PI, b + MARK, body);
    if (c < 0) return null;
    const t1 = rows(buf, LEAD, b), t2 = rows(buf, b + MARK, c);
    if (!t1 || !t2 || t1.length !== t2.length) return null;
    const sizes = new Map();
    for (const r of t2) {
      if (r.length !== 5) return null;
      const key = text(r[0]), size = value(r[1]);
      if (key === null || size === null || sizes.has(key)) return null;
      const parts = split(key);
      for (let k = 0; k < 3; k++) if (text(r[k + 2]) !== parts[k]) return null;
      sizes.set(key, size);
    }
    let next = c + MARK;
    const entries = [];
    for (const r of t1) {
      if (r.length !== 2) return null;
      const key = text(r[0]), at = value(r[1]);
      if (key === null || at === null || at !== next || !sizes.has(key) || sizes.get(key) > body - next) return null;
      entries.push({ key, at, size: sizes.get(key) });
      next += sizes.get(key);
      sizes.delete(key);
    }
    if (next !== body || !same(buf, body, PI)) return null;
    const want = new DataView(buf.buffer, buf.byteOffset + body + MARK, 4).getUint32(0, true);
    if (crc(buf.subarray(0, body + MARK)) !== want) return null;
    return { entries, crc: want };
  };

  const headOf = (magic, buf) => {
    if (buf.length < LEAD) return 'more';
    if (!same(buf, 0, magic) || !same(buf, 15, PI)) return null;
    const b = find(buf, PI, LEAD, buf.length);
    if (b < 0) return 'more';
    const c = find(buf, PI, b + MARK, buf.length);
    if (c < 0) return 'more';
    const t1 = rows(buf, LEAD, b), t2 = rows(buf, b + MARK, c);
    if (!t1 || !t2 || t1.length !== t2.length) return null;
    const sizes = new Map();
    for (const r of t2) {
      if (r.length !== 5) return null;
      const key = text(r[0]), size = value(r[1]);
      if (key === null || size === null || sizes.has(key)) return null;
      const parts = split(key);
      for (let k = 0; k < 3; k++) if (text(r[k + 2]) !== parts[k]) return null;
      sizes.set(key, size);
    }
    let next = c + MARK;
    const entries = [];
    for (const r of t1) {
      if (r.length !== 2) return null;
      const key = text(r[0]), at = value(r[1]);
      if (key === null || at === null || at !== next || !sizes.has(key)) return null;
      entries.push({ key, at, size: sizes.get(key) });
      next += sizes.get(key);
      sizes.delete(key);
    }
    return { entries, length: c + MARK, end: next };
  };

  return { MAGIC, ASK, PATCH, PI, FOOT, MARK, crc, hex, combine, bytes, join, head, write, archiveCrc, parse, headOf };
})();
const V31nSync = (() => {
  const CHUNK = 4194304;
  const INDEX = 'V31NBAIMITIF\0\0\0';
  const INDEX_KEY = 'v31np index.txt';
```

```
const LEVELS = ['S', 'A', 'B', '\u00DA', 'U', '\u00D9'];
const request = r => new Promise((ok, fail) => { r.onsuccess = () => ok(r.result); r.onerror = () => fail(r.error); });
const done = t => new Promise((ok, fail) => { t.oncomplete = () => ok(); t.onerror = () => fail(t.error); t.onabort = () => fail(t.error); });
let opened = null;
const db = () => opened || (opened = new Promise((ok, fail) => {
  const r = indexedDB.open('v31np', 3);
  r.onupgradeneeded = e => {
    const d = r.result;
    if (e.oldVersion < 3) {
      for (const name of ['files', 'archives']) if (d.objectStoreNames.contains(name)) d.deleteObjectStore(name);
    }
    for (const [name, keyPath] of [['files', ['name', 'key']], ['parts', ['id', 'n']], ['archives', 'name'], ['queries', ['name', 'key']], ['log', 'id']]) {
      if (!d.objectStoreNames.contains(name)) d.createObjectStore(name, name === 'log' ? { keyPath, autoIncrement: true } : { keyPath });
    }
  };
  r.onsuccess = () => ok(r.result);
  r.onerror = () => fail(r.error);
}));
const range = name => IDBKeyRange.bound([name], [name, []]);
const idRange = id => IDBKeyRange.bound([id], [id, []]);
const byKey = (a, b) => V31nOrder.key(a.key, b.key);
const tx = async (stores, mode) => (await db()).transaction(stores, mode);
const want = () => !!V31N_KEY;

const held = async name => (await request((await tx('files', 'readonly')).objectStore('files').getAll(range(name)))).sort(byKey);
const archive = async name => request((await tx('archives', 'readonly')).objectStore('archives').get(name));
const file = async (name, key) => request((await tx('files', 'readonly')).objectStore('files').get([name, key]));
const heads = async (name, key) => {
  const a = await archive(name);
  return a && a.heads && a.heads[key] ? a.heads[key] : [];
};

const newId = () => Date.now().toString(36) + Math.random().toString(36).slice(2);
const putPart = async (id, n, blob) => {
  const t = await tx('parts', 'readwrite');
  t.objectStore('parts').put({ id, n, blob });
  await done(t);
};
const dropParts = async ids => {
  if (!ids.length) return;
  const t = await tx('parts', 'readwrite');
  const store = t.objectStore('parts');
  for (const id of ids) store.delete(idRange(id));
  await done(t);
};
const partsOf = async meta => {
  if (!meta.parts) return [];
  const all = await request((await tx('parts', 'readonly')).objectStore('parts').getAll(idRange(meta.id)));
  return all.sort((a, b) => a.n - b.n).map(p => p.blob);
};
const blob = async (meta, type) => new Blob(await partsOf(meta), type ? { type } : undefined);
const slice = async (meta, start, end, type) => {
  const first = Math.floor(start / CHUNK), last = Math.floor(end / CHUNK);
  const store = (await tx('parts', 'readonly')).objectStore('parts');
  const asks = [];
  for (let n = first; n <= last; n++) asks.push(request(store.get([meta.id, n])));
  const got = await Promise.all(asks);
  const out = got.map((p, i) => {
    const base = (first + i) * CHUNK;
    return p.blob.slice(Math.max(0, start - base), Math.min(p.blob.size, end + 1 - base));
  });
  return new Blob(out, type ? { type } : undefined);
};
const bytesOf = async meta => new Uint8Array(await (await blob(meta)).arrayBuffer());

const sink = () => {
  const id = newId();
  const sha = want() ? V31nSha256.create() : null;
  let n = 0, size = 0, crc = 0, buf = [], fill = 0;
  const flush = async () => {
    if (!fill) return;
    await putPart(id, n++, new Blob(buf));
    buf = [];
    fill = 0;
  };
  return {
    id,
    push: async data => {
      crc = V31nPack.crc(data, crc);
      if (sha) sha.update(data);
      size += data.length;
      let off = 0;
      while (off < data.length) {
        const take = Math.min(CHUNK - fill, data.length - off);
        buf.push(data.slice(off, off + take));
        fill += take;
        off += take;
        if (fill === CHUNK) await flush();
      }
    },
    end: async () => {
      await flush();
      return { id, parts: n, size, crc: V31nPack.hex(crc), sha: sha ? sha.hex() : null };
    }
  };
};
const store = async (key, bytes) => {
  const s = sink();
  await s.push(bytes);
  return { key, ...(await s.end()) };
};

const strict = new TextDecoder('utf-8', { fatal: true });
const isIndex = b => b.length >= 15 && [...INDEX].every((c, i) => b[i] === c.charCodeAt(0));
const unescape = s => {
  let out = '';
  for (let i = 0; i < s.length; i++) {
    if (s[i] !== '\\') { out += s[i]; continue; }
    const e = { '\\': '\\', t: '\t', n: '\n', r: '\r' }[s[++i]];
    if (e === undefined) return null;
    out += e;
  }
  return out;
};
const readIndex = bytes => {
  if (!bytes || !isIndex(bytes)) return null;
  let text;
  try { text = strict.decode(bytes); } catch { return null; }
  const rows = new Map(), heads = {};
  let sig = null, signedTo = 0, at = 0, first = true;
  for (const raw of text.split('\n')) {
    const start = at;
    at += raw.length + 1;
    const line = raw.endsWith('\r') ? raw.slice(0, -1) : raw;
    if (line === '') continue;
    if (first) { if (line !== INDEX) return null; first = false; continue; }
    if (sig !== null) return null;
    const f = line.split('\t');
    if (f[0] === 'I' && f.length >= 4 && f.length <= 6 && !Object.keys(heads).length) {
      const key = unescape(f[1]);
      if (key === null || rows.has(key) || !/^\d+$/.test(f[2]) || !/^[0-9A-F]{8}$/.test(f[3])) return null;
      let sha = null, level = null;
      for (const extra of f.slice(4)) {
        if (sha === null && level === null && /^[0-9A-F]{64}$/.test(extra)) sha = extra;
        else if (level === null && LEVELS.includes(extra)) level = extra;
        else return null;
      }
      rows.set(key, { size: Number(f[2]), crc: f[3], sha, level: level || '\u00D9' });
    } else if (f[0] === 'H' && f.length === 4) {
      const key = unescape(f[1]), n = unescape(f[2]), v = unescape(f[3]);
      if (key === null || !rows.has(key) || !n || !/^[!#$%&'*+.^_`|~0-9A-Za-z-]+$/.test(n) || v === null || /[\0\r\n]/.test(v)) return null;
      (heads[key] = heads[key] || []).push([n, v]);
    } else if (f[0] === 'S' && f.length === 2 && /^[0-9A-F]{128}$/.test(f[1])) {
      sig = f[1];
      signedTo = start;
    } else return null;
  }
  return { rows, heads, sig, signed: sig ? new TextEncoder().encode(text.slice(0, signedTo)) : null };
};
const unhex = s => new Uint8Array(s.match(/../g).map(h => parseInt(h, 16)));
const signedBy = async ix => {
  if (!V31N_KEY) return true;
  if (!ix || !ix.sig) return false;
  try {
    const pub = await crypto.subtle.importKey('raw', unhex(V31N_KEY), { name: 'Ed25519' }, false, ['verify']);
    return await crypto.subtle.verify({ name: 'Ed25519' }, pub, unhex(ix.sig), ix.signed);
  } catch {
    return false;
  }
};
const fit = key => key === '' || key === INDEX_KEY || (key.startsWith('/') && !key.split('/').some(s => s === '.' || s === '..'));
const matches = (row, m) => !!row && row.size === m.size && row.crc === m.crc && (!V31N_KEY || (!!row.sha && row.sha === m.sha));
const checkWhole = async (metas, ix) => {
```

```
    if (!V31N_KEY) return true;
    if (!await signedBy(ix)) return false;
    const others = metas.filter(m => m.key !== INDEX_KEY);
    return others.length === [...ix.rows.keys()].filter(fit).length && others.every(m => matches(ix.rows.get(m.key), m));
};
const archiveSum = metas => {
    const header = V31nPack.head(V31nPack.MAGIC, metas.map(m => ({ key: m.key, size: m.size })));
    let sum = V31nPack.crc(header);
    for (const m of metas) sum = V31nPack.combine(sum, parseInt(m.crc, 16), m.size);
    return V31nPack.hex(V31nPack.combine(sum, V31nPack.crc(V31nPack.PI), V31nPack.MARK));
};

const streamFull = async response => {
    const reader = response.body.getReader();
    let pending = new Uint8Array(0);
    const pull = async () => {
        const r = await reader.read();
        if (r.done) return false;
        pending = pending.length ? V31nPack.join([pending, r.value]) : r.value;
        return true;
    };
    const made = [];
    const fail = async error => {
        try { await reader.cancel(); } catch {}
        await dropParts(made);
        return { error };
    };
    let head;
    for (;;) {
        head = V31nPack.headOf(V31nPack.MAGIC, pending);
        if (head !== 'more') break;
        if (!await pull()) return fail('short header');
    }
    if (!head) return fail('broken header');
    let sum = V31nPack.crc(pending.subarray(0, head.length));
    pending = pending.subarray(head.length);
    const metas = [];
    let indexBytes = null;
    for (const e of head.entries) {
        const s = fit(e.key) ? sink() : null;
        if (s) made.push(s.id);
        const keep = e.key === INDEX_KEY ? [] : null;
        let left = e.size;
        while (left > 0) {
            if (!pending.length && !await pull()) return fail('short');
            const take = pending.subarray(0, Math.min(left, pending.length));
            if (s) await s.push(take);
            sum = V31nPack.crc(take, sum);
            if (keep) keep.push(take.slice());
            pending = pending.subarray(take.length);
            left -= take.length;
        }
        if (s) metas.push({ key: e.key, ...(await s.end()) });
        if (keep) indexBytes = V31nPack.join(keep);
    }
    while (pending.length < V31nPack.FOOT) if (!await pull()) return fail('short footer');
    for (let k = 0; k < V31nPack.MARK; k++) if (pending[k] !== V31nPack.PI[k]) return fail('broken footer');
    sum = V31nPack.crc(pending.subarray(0, V31nPack.MARK), sum);
    const stated = new DataView(pending.buffer, pending.byteOffset + V31nPack.MARK, 4).getUint32(0, true);
    if (pending.length > V31nPack.FOOT || await pull()) return fail('trailing bytes');
    if (sum !== stated) return fail('crc');
    return { metas, ix: readIndex(indexBytes), crc: V31nPack.hex(sum), ids: made };
};

const commit = async (name, put, remove, record) => {
    const t = await tx(['files', 'archives'], 'readwrite');
    const files = t.objectStore('files');
    for (const key of remove) files.delete([name, key]);
    for (const m of put) files.put({ name, ...m });
    if (record) t.objectStore('archives').put({ name, ...record });
    await done(t);
};

const listOf = (ix, indexSize) => {
    const keys = [...ix.rows.keys(), INDEX_KEY].sort(V31nOrder.key);
    const items = keys.map(key => ({ key, size: key === INDEX_KEY ? indexSize : ix.rows.get(key).size }));
    let at = V31nPack.head(V31nPack.MAGIC, items).length;
    const list = {};
    for (const it of items) {
        list[it.key] = [at, it.size];
        at += it.size;
    }
    return list;
};

const etag = r => (r.headers.get('etag') || '').replace(/"/g, '') || null;

const run = async (name, why, url = '/' + name + '.v31np', wanted = []) => {
    const log = { name, why, steps: [] };
    const step = (what, r, bytes) => log.steps.push({ what, status: r.status, redirected: r.redirected, etag: etag(r), bytes });
    const old = await held(name);

    const whole = async (r, again) => {
        const got = await streamFull(r);
        step(again ? 'full after failed check' : 'full', r, got.error ? null : got.metas.reduce((n, m) => n + m.size, 0));
        if (got.error) { log.result = 'broken full file: ' + got.error; return; }
        const tag = etag(r);
        if (tag && !r.redirected && tag !== got.crc) {
            await dropParts(got.ids);
            log.result = 'etag differs from full file';
            return;
        }
        if (!await checkWhole(got.metas, got.ix)) {
            await dropParts(got.ids);
            if (again) { log.result = 'refused, check failed'; return; }
            return whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
        }
        await commit(name, got.metas, old.map(m => m.key), { crc: got.crc, heads: got.ix ? got.ix.heads : {}, mode: 'full', url });
        await dropParts(old.map(m => m.id));
        log.result = 'stored ' + got.metas.length + ' files, crc ' + got.crc;
    };

    const listed = async (bytes, tag) => {
        const ix = readIndex(bytes);
        if (!ix) return null;
        const rows = [...ix.rows.keys(), INDEX_KEY].sort(V31nOrder.key).map(key => key === INDEX_KEY
            ? { key, size: bytes.length, crc: V31nPack.hex(V31nPack.crc(bytes)) }
            : { key, size: ix.rows.get(key).size, crc: ix.rows.get(key).crc });
        if (tag !== archiveSum(rows) || !await signedBy(ix)) return null;
        return { ix, list: listOf(ix, bytes.length) };
    };

    const asked = new Map(old.map(f => [f.key, f.crc]));
    for (const key of wanted) if (!asked.has(key)) asked.set(key, '00000000');
    const ask = V31nPack.write(V31nPack.ASK, [...asked].sort((x, y) => V31nOrder.key(x[0], y[0])).map(([key, crc]) => ({ key, bytes: V31nPack.bytes(crc) })));
    const body = V31nPack.write(V31nPack.MAGIC, [{ key: '', bytes: ask.buf }]);
    log.sent = body.buf.length;
    if (wanted.length) log.wanted = wanted.length;
    const before = await archive(name);
    const r = await fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/v31np' }, body: body.buf, cache: 'no-store' });

    if (r.status === 200) await whole(r, false);
    else if (r.status === 304) { step('sync', r, 0); log.result = 'in sync'; }
    else if (r.status === 268) {
        const buf = new Uint8Array(await r.arrayBuffer());
        step('sync', r, buf.length);
        const outer = V31nPack.parse(V31nPack.MAGIC, buf);
        const e = outer && outer.entries.length === 1 && outer.entries[0].key === INDEX_KEY ? outer.entries[0] : null;
        const bytes = e ? buf.slice(e.at, e.at + e.size) : null;
        const got = bytes ? await listed(bytes, etag(r)) : null;
        if (!got) await whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
        else {
            const im = await store(INDEX_KEY, bytes);
            const stale = old.filter(m => m.key === INDEX_KEY || !matches(got.ix.rows.get(m.key), m));
            await commit(name, [im], stale.map(m => m.key), { crc: etag(r), heads: got.ix.heads, mode: 'list', url, list: got.list });
            await dropParts(stale.map(m => m.id));
            log.result = 'list of ' + got.ix.rows.size + ' files, ' + (old.length - stale.length) + ' kept, crc ' + etag(r);
        }
    } else if (r.status === 267) {
        const buf = new Uint8Array(await r.arrayBuffer());
        step('sync', r, buf.length);
        const outer = V31nPack.parse(V31nPack.MAGIC, buf);
        const e = outer && outer.entries.length === 1 && outer.entries[0].key === '' ? outer.entries[0] : null;
        const inner = e ? buf.slice(e.at, e.at + e.size) : null;
        const patch = inner ? V31nPack.parse(V31nPack.PATCH, inner) : null;
        if (!patch) log.result = 'broken 267 answer';
        else if (before && before.mode === 'list') {
            const files = new Map(old.map(m => [m.key, m]));
            const added = [], removed = [], absent = [];
            let bad = false;
            for (const p of patch.entries) {
                if (!fit(p.key)) continue;
                const flag = p.size > 0 ? inner[p.at + p.size - 1] : -1;
```

```
                if (flag === 1 && p.size === 1) {
                    if (files.has(p.key)) { files.delete(p.key); removed.push(p.key); }
                    else absent.push(p.key);
                } else if (flag === 0) {
                    const m = await store(p.key, inner.slice(p.at, p.at + p.size - 1));
                    files.set(p.key, m);
                    added.push(m);
                } else bad = true;
            }
            const im = files.get(INDEX_KEY);
            const indexBytes = im ? await bytesOf(im) : null;
            let got = null;
            if (added.some(m => m.key === INDEX_KEY)) got = await listed(indexBytes, etag(r));
            else if (indexBytes && etag(r) === before.crc) got = { ix: readIndex(indexBytes), list: before.list };
            const ok = !bad && !!got && !!got.ix && added.every(m => m.key === INDEX_KEY || matches(got.ix.rows.get(m.key), m));
            log.patch = { added: added.map(m => m.key), removed, absent, etag: etag(r) };
            if (ok) {
                const stale = [...files.values()].filter(m => m.key !== INDEX_KEY && !matches(got.ix.rows.get(m.key), m));
                const gone = [...removed, ...stale.map(m => m.key)];
                const replaced = old.filter(m => gone.includes(m.key) || added.some(x => x.key === m.key));
                await commit(name, added, gone, { crc: etag(r), heads: got.ix.heads, mode: 'list', url, list: got.list });
                await dropParts(replaced.map(m => m.id));
                log.result = 'took ' + added.length + ' files, crc ' + etag(r);
            } else {
                await dropParts(added.map(m => m.id));
                await whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
            }
        } else {
            const files = new Map(old.map(m => [m.key, m]));
            const added = [], removed = [];
            let bad = false;
            for (const p of patch.entries) {
                if (!fit(p.key)) continue;
                const flag = p.size > 0 ? inner[p.at + p.size - 1] : -1;
                if (flag === 1 && p.size === 1) { files.delete(p.key); removed.push(p.key); }
                else if (flag === 0) {
                    const m = await store(p.key, inner.slice(p.at, p.at + p.size - 1));
                    files.set(p.key, m);
                    added.push(m);
                } else bad = true;
            }
            const rebuilt = [...files.values()].sort(byKey);
            const crc = archiveSum(rebuilt);
            const im = files.get(INDEX_KEY);
            const ix = im ? readIndex(await bytesOf(im)) : null;
            log.patch = { changed: added.map(m => m.key), removed, crc, etag: etag(r) };
            if (!bad && crc === etag(r) && await checkWhole(rebuilt, ix)) {
                const replaced = old.filter(m => removed.includes(m.key) || added.some(x => x.key === m.key));
                await commit(name, added, removed, { crc, heads: ix ? ix.heads : {}, mode: 'full', url });
                await dropParts(replaced.map(m => m.id));
                log.result = 'patched, crc ' + crc;
            } else {
                await dropParts(added.map(m => m.id));
                await whole(await fetch(url, { headers: { Accept: 'application/v31np' }, cache: 'no-store' }), true);
            }
        }
    } else { step('sync', r, null); log.result = 'status ' + r.status; }
    await note({ kind: 'sync', ...log });
    return log;
};

const chains = new Map();
const enqueue = (name, fn) => {
    const next = (chains.get(name) || Promise.resolve()).catch(() => {}).then(fn);
    chains.set(name, next);
    return next;
};
const running = new Map();
const sync = (name, why, url) => {
    if (!running.has(name)) running.set(name, enqueue(name, () => run(name, why, url)).finally(() => running.delete(name)));
    return running.get(name);
};

const listedSize = async (name, key) => {
    const a = await archive(name);
    return a && a.mode === 'list' && a.list && a.list[key] && key !== INDEX_KEY ? a.list[key][1] : null;
};
const waiting = new Map();
const need = (name, key) => {
    let w = waiting.get(name);
    if (!w) {
        w = { keys: new Set() };
        w.done = new Promise(resolve => setTimeout(() => {
            waiting.delete(name);
            resolve(enqueue(name, async () => {
                const a = await archive(name);
                const keys = [];
                for (const k of w.keys) if (a && a.list && a.list[k] && !await file(name, k)) keys.push(k);
                if (keys.length) return run(name, 'want ' + keys.length, a.url, keys);
            }));
        }, 30));
        waiting.set(name, w);
    }
    w.keys.add(key);
    return w.done.catch(() => {}).then(() => file(name, key));
};
const remote = async (name, key, start, end) => {
    const a = await archive(name);
    if (!a || a.mode !== 'list' || !a.list || !a.list[key]) return null;
    const at = a.list[key][0];
    const want = 'bytes=' + (at + start) + '-' + (at + end);
    const r = await fetch(a.url, { headers: { Range: want, 'If-Range': '"' + a.crc + '"' }, cache: 'no-store' });
    if (r.status !== 206 || !(r.headers.get('content-range') || '').startsWith('bytes ' + (at + start) + '-' + (at + end) + '/')) {
        try { await r.body.cancel(); } catch {}
        return null;
    }
    return r.body;
};

const wantKeys = (name, keys) => enqueue(name, async () => {
    const a = await archive(name);
    const missing = [];
    for (const k of keys) if (a && a.list && a.list[k] && !await file(name, k)) missing.push(k);
    if (missing.length) return run(name, 'want ' + missing.length, a.url, missing);
});
const rows = async name => {
    const im = await file(name, INDEX_KEY);
    const ix = im ? readIndex(await bytesOf(im)) : null;
    return ix ? ix.rows : new Map();
};

const load = async name => {
    const files = [];
    for (const m of await held(name)) files.push({ key: m.key, bytes: await bytesOf(m) });
    const a = await archive(name);
    return { files, crc: a ? a.crc : null };
};
const query = async (name, key, value) => {
    const t = await tx('queries', 'readwrite');
    const s = t.objectStore('queries');
    const old = await request(s.get([name, key]));
    s.put({ name, key, value });
    await done(t);
    return old ? old.value : null;
};
const note = async entry => {
    const t = await tx('log', 'readwrite');
    t.objectStore('log').add(entry);
    await done(t);
};
const notes = async () => request((await tx('log', 'readonly')).objectStore('log').getAll());

return { sync, load, file, blob, slice, need, want: wantKeys, rows, remote, listed: listedSize, held, query, note, notes, heads };
})();
const V31nShow = (() => {
    const INDEX_KEY = 'v31np index.txt';
    const SVG = 'http://www.w3.org/2000/svg';
    const types = {
        html: 'text/html; charset=utf-8', htm: 'text/html; charset=utf-8', css: 'text/css; charset=utf-8', js: 'text/javascript; charset=utf-8',
        mjs: 'text/javascript; charset=utf-8', json: 'application/json', txt: 'text/plain; charset=utf-8', xml: 'application/xml', svg: 'image/svg+xml',
        png: 'image/png', jpg: 'image/jpeg', jpeg: 'image/jpeg', gif: 'image/gif', ico: 'image/x-icon', webp: 'image/webp', avif: 'image/avif',
        bmp: 'image/bmp', woff2: 'font/woff2', woff: 'font/woff', ttf: 'font/ttf', otf: 'font/otf', wasm: 'application/wasm', pdf: 'application/pdf',
        mp3: 'audio/mpeg', wav: 'audio/wav', ogg: 'audio/ogg', oga: 'audio/ogg', opus: 'audio/ogg', m4a: 'audio/mp4', aac: 'audio/aac', flac: 'audio/flac',
        mp4: 'video/mp4', m4v: 'video/mp4', webm: 'video/webm', ogv: 'video/ogg', mov: 'video/quicktime', vtt: 'text/vtt; charset=utf-8', stl: 'model/stl'
    };
    const extOf = key => {
        const dot = key.lastIndexOf('.');
        return dot > key.lastIndexOf('/') && dot >= 0 ? key.slice(dot + 1).toLowerCase() : '';
    };
    const typeOf = key => types[extOf(key)] || 'application/octet-stream';
    const isCss = key => extOf(key) === 'css';
    const skip = /^(?:blob:|data:|about:|javascript:|mailto:|tel:|#)/i;
    const fileUrl = key => location.origin + key.split('/').map(encodeURIComponent).join('/');
```

```
const target = (raw, base) => {
    if (typeof raw !== 'string') return null;
    const t = raw.trim();
    if (t === '' || skip.test(t)) return null;
    let u;
    try { u = new URL(t, base); } catch { return null; }
    if (u.origin !== location.origin) return null;
    let key;
    try { key = decodeURIComponent(u.pathname); } catch { return null; }
    let search = u.search;
    try { search = decodeURIComponent(u.search); } catch {}
    return { key, search, rawSearch: u.search, hash: u.hash, href: u.href };
};
const cssPattern = () => /url\(\s*(["']?)([^"')]+?)\1\s*\)|@import\s+(["'])([^"']+)/3/g;
const cssRefs = text => {
    const out = [];
    for (const m of String(text).matchAll(cssPattern())) out.push(m[2] !== undefined ? m[2] : m[4]);
    return out;
};
const srcsetRefs = value => String(value).split(',').map(s => s.trim().split(/\s+/)[0]).filter(Boolean);

const links = /(?:^|\s)(?:stylesheet|icon|shortcut|apple-touch-icon|mask-icon|preload|modulepreload|prefetch)(?:\s|$)/i;
const kinds = el => {
    const name = el.namespaceURI === SVG ? el.localName : el.tagName;
    switch (name) {
        case 'IMG': return { src: 'take', srcset: 'set' };
        case 'SCRIPT': return { src: 'take' };
        case 'INPUT': return { src: 'take' };
        case 'EMBED': return { src: 'take' };
        case 'OBJECT': return { data: 'take' };
        case 'LINK': return links.test(el.getAttribute('rel') || '') ? { href: 'take' } : {};
        case 'AUDIO': return { src: 'play' };
        case 'VIDEO': return { src: 'play', poster: 'take' };
        case 'SOURCE': return el.parentNode && /^(AUDIO|VIDEO)$/.test(el.parentNode.tagName || '') ? { src: 'play' } : { src: 'take', srcset: 'set' };
        case 'TRACK': return { src: 'play' };
        case 'image': case 'use': case 'feImage': return { href: 'take', 'xlink:href': 'take' };
        default: return {};
    }
};

const report = entry => console.log('v31np', JSON.stringify(entry));
const leave = async () => {
    if (!navigator.serviceWorker) return;
    for (const r of await navigator.serviceWorker.getRegistrations()) {
        const w = r.active || r.waiting || r.installing;
        if (w && new URL(w.scriptURL).pathname === '/v31np/sw.js') await r.unregister();
    }
};

const start = async ({ archive, page, html, started, logs, queries = [] }) => {
    const archiveAbs = new URL(archive, location.href).href;
    const listed = await V31nSync.rows(archive);
    const held = new Map();
    const fill = async () => {
        for (const m of await V31nSync.held(archive)) if (m.key !== INDEX_KEY) held.set(m.key, m);
        if (page) for (const m of await V31nSync.held(page)) if (m.key !== '' && m.key !== INDEX_KEY) held.set(m.key, m);
    };
    await fill();

    const doc = new DOMParser().parseFromString(html, 'text/html');
    const baseEl = doc.querySelector('base[href]');
    const basc = baseEl ? new URL(baseEl.getAttribute('href'), location.href).href : location.href;

    const refs = new Set();
    const add = (raw, b) => {
        const t = target(raw, b);
        if (t && listed.has(t.key)) refs.add(t.key);
    };
    const each = fn => {
        for (const el of doc.querySelectorAll('*')) {
            const k = kinds(el);
            for (const a of [...el.attributes]) {
                if (a.name === 'style') fn(el, a, 'style');
                else if (k[a.name]) fn(el, a, k[a.name]);
            }
            if (el.tagName === 'STYLE') fn(el, null, 'sheet');
            if (el.tagName === 'SCRIPT' && (el.getAttribute('type') || '').toLowerCase() === 'importmap') fn(el, null, 'importmap');
        }
    };
    each((el, a, kind) => {
        if (kind === 'take') add(a.value, base);
        else if (kind === 'set') for (const u of srcsetRefs(a.value)) add(u, base);
        else if (kind === 'style') for (const u of cssRefs(a.value)) add(u, base);
        else if (kind === 'sheet') for (const u of cssRefs(el.textContent)) add(u, base);
        else if (kind === 'importmap') {
            try { for (const v of Object.values(JSON.parse(el.textContent).imports || {})) add(String(v), base); } catch {}
        }
    });

    const asked = new Set();
    const rounds = [];
    const round = async keys => {
        const fresh = [...keys].filter(k => !held.has(k) && !asked.has(k));
        if (!fresh.length) return false;
        for (const k of fresh) asked.add(k);
        const got = await V31nSync.want(archive, fresh);
        rounds.push({ wanted: fresh.length, result: got && got.result, steps: got && got.steps });
        await fill();
        return true;
    };
    await round([...refs]);
    const read = new Set();
    for (;;) {
        const more = new Set();
        for (const key of [...refs]) {
            if (!isCss(key) || read.has(key) || !held.has(key)) continue;
            read.add(key);
            const text = await (await V31nSync.blob(held.get(key))).text();
            for (const u of cssRefs(text)) {
                const t = target(u, fileUrl(key));
                if (!t || !listed.has(t.key)) continue;
                refs.add(t.key);
                if (!held.has(t.key)) more.add(t.key);
            }
        }
        if (!more.size || !await round(more)) break;
    }

    const urls = new Map();
    const making = new Set();
    const map = (raw, b, absolute = false) => {
        const t = target(raw, b);
        if (!t) return raw;
        if (t.search && urls.has(t.key + t.search)) return urls.get(t.key + t.search) + t.hash;
        if (urls.has(t.key)) return urls.get(t.key) + t.hash;
        if (listed.has(t.key)) return archiveAbs + '?key=' + encodeURIComponent(t.key) + t.hash;
        return absolute ? t.href : raw;
    };
    const cssMap = (text, b, absolute = false) => String(text).replace(cssPattern(), (m, q, u, q2, u2) => u !== undefined
        ? 'url(' + q + map(u, b, absolute) + q + ')'
        : '@import ' + q2 + map(u2, b, absolute) + q2);
    const setMap = (value, b) => String(value).split(',').map(part => {
        const s = part.trim();
        const at = s.search(/\s/);
        return at < 0 ? map(s, b) : map(s.slice(0, at), b) + s.slice(at);
    }).join(', ');
    const makeCss = async key => {
        if (urls.has(key) || making.has(key)) return;
        making.add(key);
        const own = fileUrl(key);
        const text = await (await V31nSync.blob(held.get(key))).text();
        for (const u of cssRefs(text)) {
            const t = target(u, own);
            if (t && isCss(t.key) && held.has(t.key)) await makeCss(t.key);
        }
        urls.set(key, URL.createObjectURL(new Blob([cssMap(text, own, true)], { type: typeOf(key) })));
    };
    const make = async () => {
        for (const [key, m] of held) if (!isCss(key) && !urls.has(key)) urls.set(key, URL.createObjectURL(await V31nSync.blob(m, typeOf(key))));
        for (const key of held.keys()) if (isCss(key)) await makeCss(key);
    };
    await make();

    each((el, a, kind) => {
        if (kind === 'take' || kind === 'play') a.value = map(a.value, base);
        else if (kind === 'set') a.value = setMap(a.value, base);
        else if (kind === 'style') a.value = cssMap(a.value, base);
        else if (kind === 'sheet') el.textContent = cssMap(el.textContent, base);
        else if (kind === 'importmap') {
            let o;
            try { o = JSON.parse(el.textContent); } catch { return; }
            o.imports = o.imports || {};
            for (const k of Object.keys(o.imports)) o.imports[k] = map(String(o.imports[k]), base);
```

```
            for (const [key, u] of urls) if (/\.m?js$/i.test(key)) o.imports[fileUrl(key)] = o.imports[fileUrl(key)] || u;
            el.textContent = JSON.stringify(o);
        }
    });

    const fixAttr = (el, attr, value) => {
        if (attr === 'style') return cssMap(value, document.baseURI);
        const kind = kinds(el)[attr];
        if (kind === 'set') return setMap(value, document.baseURI);
        return kind ? map(value, document.baseURI) : value;
    };
    const wrap = (proto, prop, fix) => {
        const d = proto && Object.getOwnPropertyDescriptor(proto, prop);
        if (!d || !d.set) return;
        Object.defineProperty(proto, prop, { ...d, set(v) { d.set.call(this, typeof v === 'string' ? fix(this, v) : v); } });
    };
    const byAttr = attr => (el, v) => fixAttr(el, attr, v);
    wrap(HTMLImageElement.prototype, 'src', byAttr('src'));
    wrap(HTMLImageElement.prototype, 'srcset', byAttr('srcset'));
    wrap(HTMLSourceElement.prototype, 'src', byAttr('src'));
    wrap(HTMLSourceElement.prototype, 'srcset', byAttr('srcset'));
    wrap(HTMLMediaElement.prototype, 'src', (el, v) => map(v, document.baseURI));
    wrap(HTMLVideoElement.prototype, 'poster', (el, v) => map(v, document.baseURI));
    wrap(HTMLTrackElement.prototype, 'src', (el, v) => map(v, document.baseURI));
    wrap(HTMLLinkElement.prototype, 'href', (el, v) => map(v, document.baseURI));
    wrap(HTMLScriptElement.prototype, 'src', (el, v) => map(v, document.baseURI));
    wrap(HTMLInputElement.prototype, 'src', (el, v) => map(v, document.baseURI));
    wrap(HTMLEmbedElement.prototype, 'src', (el, v) => map(v, document.baseURI));
    wrap(HTMLObjectElement.prototype, 'data', (el, v) => map(v, document.baseURI));
    for (const prop of ['background', 'backgroundImage', 'cursor', 'content', 'listStyle', 'listStyleImage', 'borderImage', 'borderImageSource', 'maskImage', 'webkitMaskImage', 'cssText']) {
        wrap(CSSStyleDeclaration.prototype, prop, (el, v) => cssMap(v, document.baseURI));
    }
    const setProperty = CSSStyleDeclaration.prototype.setProperty;
    CSSStyleDeclaration.prototype.setProperty = function (prop, value, priority) {
        return setProperty.call(this, prop, typeof value === 'string' ? cssMap(value, document.baseURI) : value, priority);
    };
    const setAttribute = Element.prototype.setAttribute;
    Element.prototype.setAttribute = function (attr, value) {
        return setAttribute.call(this, attr, typeof value === 'string' ? fixAttr(this, String(attr).toLowerCase(), value) : value);
    };
    const realFetch = window.fetch.bind(window);
    const shown = new Map();
    window.fetch = async (input, init) => {
        const isReq = input instanceof Request;
        const raw = typeof input === 'string' || input instanceof URL ? String(input) : isReq ? input.url : null;
        const method = String((init && init.method) || (isReq ? input.method : 'GET')).toUpperCase();
        if (raw === null || method !== 'GET') return realFetch(input, init);
        const accept = String(new Headers((init && init.headers) || (isReq ? input.headers : undefined)).get('accept') || '');
        const t = target(raw, document.baseURI);
        if (t && queries.includes(t.key) && !accept.includes('application/v31np')) {
            const name = new URL(t.href).pathname + t.rawSearch;
            const got = await V31nSync.sync(name, 'query', name);
            const metas = await V31nSync.held(name);
            const own = metas.find(m => m.key === '');
            if (own) {
                for (const m of metas) {
                    if (m.key === '' || m.key === INDEX_KEY || (urls.has(m.key) && shown.get(m.key) === m.id)) continue;
                    shown.set(m.key, m.id);
                    const old = urls.get(m.key);
                    urls.set(m.key, URL.createObjectURL(await V31nSync.blob(m, typeOf(m.key.split('?')[0]))));
                    if (old) URL.revokeObjectURL(old);
                }
                report({ what: 'query', url: name, result: got && got.result, steps: got && got.steps, files: metas.length - 2 });
                return new Response(await V31nSync.blob(own), { status: 200 });
            }
        }
        if (t && listed.has(t.key) && !urls.has(t.key) && !/^(?:audio|video)\//.test(typeOf(t.key))) {
            await V31nSync.need(archive, t.key);
            await fill();
            await make();
        }
        const mapped = map(raw, document.baseURI);
        if (mapped === raw) return realFetch(input, init);
        return realFetch(mapped, isReq ? { method: 'GET', headers: input.headers, signal: input.signal, ...init } : init);
    };
    const htmlMap = text => String(text)
        .replace(/(\s(?:src|poster)\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => {
            const plain = u.replace(/&amp;/g, '&');
            const m = map(plain, document.baseURI);
            return m === plain ? whole : a + q + m + q;
        })
        .replace(/(\ssrcset\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => a + q + setMap(u.replace(/&amp;/g, '&'), document.baseURI) + q)
        .replace(/(\sstyle\s*=\s*)(["'])([^"']*)\2/gi, (whole, a, q, u) => a + q + cssMap(u, document.baseURI) + q);
    for (const prop of ['innerHTML', 'outerHTML']) {
        const d = Object.getOwnPropertyDescriptor(Element.prototype, prop);
        if (d && d.set) Object.defineProperty(Element.prototype, prop, { ...d, set(v) { d.set.call(this, typeof v === 'string' ? htmlMap(v) : v); } });
    }
    const adjacent = Element.prototype.insertAdjacentHTML;
    Element.prototype.insertAdjacentHTML = function (where, text) {
        return adjacent.call(this, where, typeof text === 'string' ? htmlMap(text) : text);
    };
    const open = XMLHttpRequest.prototype.open;
    XMLHttpRequest.prototype.open = function (method, url, ...rest) {
        return open.call(this, method, typeof url === 'string' && String(method).toUpperCase() === 'GET' ? map(url, document.baseURI) : url, ...rest);
    };
    window.Audio = new Proxy(window.Audio, { construct: (t, args) => new t(...args.map((a, i) => i === 0 && typeof a === 'string' ? map(a, document.baseURI) : a)) });
    window.FontFace = new Proxy(window.FontFace, { construct: (t, args) => new t(...args.map((a, i) => i === 1 && typeof a === 'string' ? cssMap(a, document.baseURI) : a)) });
    if (window.Worker) window.Worker = new Proxy(window.Worker, { construct: (t, args) => new t(...args.map((a, i) => i === 0 && typeof a === 'string' ? map(a, document.baseURI) : a)) });

    const out = (doc.doctype ? '<!DOCTYPE ' + doc.doctype.name + '>\n' : '') + doc.documentElement.outerHTML;
    report({ what: 'page', ...logs, rounds, refs: refs.size, files: urls.size, listed: listed.size, ms: Math.round(performance.now() - started) });
    document.open();
    document.write(out);
    document.close();

    const second = [...listed].filter(([key, row]) => row.level === 'A' && !held.has(key)).map(([key]) => key);
    if (second.length) {
        const got = await V31nSync.want(archive, second);
        await fill();
        await make();
        report({ what: 'level A', result: got && got.result, steps: got && got.steps, files: urls.size, ms: Math.round(performance.now() - started) });
    }
};

const me = document.currentScript;
const stub = (() => {
    const meta = n => {
        const m = document.querySelector('meta[name="' + n + '"]');
        return m ? String(m.getAttribute('content') || '') : null;
    };
    const name = me && me.dataset.archive !== undefined ? me.dataset.archive : meta('v31np-archive');
    const url = me && me.dataset.url !== undefined ? me.dataset.url : meta('v31np-url');
    return name === null || url === null ? null : { name, url };
})();
if (typeof V31N_PAGE !== 'undefined' && stub && /^[A-Za-z0-9_-]+$/.test(stub.name) && stub.url !== '') {
    const started = performance.now();
    const archive = new URL(stub.url, location.href).href;
    const name = location.pathname + location.search;
    (async () => {
        await leave();
        const [pageLog, listLog] = await Promise.all([V31nSync.sync(name, 'page', name), V31nSync.sync(archive, 'list', archive)]);
        const logs = { page: { result: pageLog.result, steps: pageLog.steps }, list: { result: listLog.result, steps: listLog.steps } };
        const pm = (await V31nSync.held(name)).find(m => m.key === '');
        if (!pm) return report({ what: 'page', shown: false, ...logs });
        await start({ archive, page: name, html: await (await V31nSync.blob(pm)).text(), started, logs });
    })();
} else if (stub && /^[A-Za-z0-9_-]+$/.test(stub.name) && stub.url !== '') {
    const started = performance.now();
    const archive = new URL(stub.url, location.href).href;
    const queries = String((me && me.dataset.query) || '').split(/\s+/).filter(Boolean).map(q => '/' + q.replace(/^\/+/, ''));
    const listing = leave().then(() => V31nSync.sync(archive, 'list', archive));
    const whole = document.documentElement.outerHTML;
    const mine = me.outerHTML;
    const at = whole.lastIndexOf(mine);
    const prefix = (document.doctype ? '<!DOCTYPE ' + document.doctype.name + '>\n' : '') + (at < 0 ? '' : whole.slice(0, at));
    document.write('<plaintext style="display:none">');
    document.addEventListener('DOMContentLoaded', async () => {
        const rest = document.querySelector('plaintext');
        const listLog = await listing;
        await start({ archive, page: null, html: prefix + (rest ? rest.textContent : ''), started, logs: { list: { result: listLog.result, steps: listLog.steps } }, queries });
    }, { once: true });
}

return { start };
})();
```

19 - File: v31np.php

Size

8456

SHA-256

3CDCAF73B4F8AE846C9ED1B93185F2399F6B8A2E662DC4D7250E88FD525427BE

Code

```
<?php
require __DIR__ . '/lib/pack.php';
require __DIR__ . '/lib/ranges.php';
require __DIR__ . '/lib/settings.php';

set_time_limit(0);
header('Cache-Control: no-store');
$name = $_GET['name'] ?? '';
$pre = __DIR__ . '/pre/';
if (!preg_match('/^[A-Za-z0-9_-]+$/', $name) || !is_file($pre . $name . '.v31np') || !is_file($pre . $name . '.json')) {
    http_response_code(404);
    exit;
}
$archive = $pre . $name . '.v31np';
$info = json_decode(file_get_contents($pre . $name . '.json'), true);
$settings = v31np_settings($name);
header('ETag: "' . $info['crc'] . '"');

$member = (string)($_GET['key'] ?? '');
if ($member !== '') {
    require_once __DIR__ . '/lib/index.php';
    $entry = null;
    $listing = null;
    foreach ($info['files'] as $f) {
        if ($f['key'] === $member) {
            $entry = $f;
        }
        if ($f['key'] === V31nIndex::KEY) {
            $listing = $f;
        }
    }
    if ($entry === null || $member === V31nIndex::KEY) {
        http_response_code(404);
        exit;
    }
    if (!in_array($_SERVER['REQUEST_METHOD'], ['GET', 'HEAD'], true)) {
        http_response_code(405);
        header('Allow: GET, HEAD');
        exit;
    }
    $heads = [];
    if ($listing !== null) {
        $h = fopen($archive, 'rb');
        fseek($h, $listing['at']);
        $text = fread($h, $listing['size']);
        fclose($h);
        foreach (preg_split('/\r?\n/', $text) as $line) {
            $f = explode("\t", $line);
            if ($f[0] === 'H' && count($f) === 4 && V31nIndex::unescape($f[1]) === $member) {
                $heads[] = [V31nIndex::unescape($f[2]), V31nIndex::unescape($f[3])];
            }
        }
    }
    $types = ['mp3' => 'audio/mpeg', 'mp4' => 'video/mp4', 'webm' => 'video/webm', 'wav' => 'audio/wav', 'png' => 'image/png',
        'jpg' => 'image/jpeg', 'jpeg' => 'image/jpeg', 'gif' => 'image/gif', 'svg' => 'image/svg+xml', 'ico' => 'image/x-icon',
        'woff' => 'font/woff', 'woff2' => 'font/woff2', 'ttf' => 'font/ttf', 'otf' => 'font/otf', 'css' => 'text/css; charset=utf-8',
        'js' => 'text/javascript; charset=utf-8', 'json' => 'application/json', 'txt' => 'text/plain; charset=utf-8'];
    $type = $types[strtolower(pathinfo($member, PATHINFO_EXTENSION))] ?? 'application/octet-stream';
    v31np_ranges($archive, $entry['at'], $entry['size'], $type, $entry['crc'], $heads);
}

$full = function () use ($archive) {
    header('Content-Type: application/v31np');
    header('Accept-Ranges: bytes');
    header('Content-Length: ' . filesize($archive));
    if ($_SERVER['REQUEST_METHOD'] !== 'HEAD') {
        readfile($archive);
    }
    exit;
};

$method = $_SERVER['REQUEST_METHOD'];
if ($method === 'GET' || $method === 'HEAD') {
    v31np_ranges($archive, 0, filesize($archive), 'application/v31np', $info['crc']);
}
if ($method !== 'POST') {
    http_response_code(405);
    header('Allow: GET, HEAD, POST');
    exit;
}

if (strtolower(trim(explode(';', (string)($_SERVER['CONTENT_TYPE'] ?? ''))[0])) !== 'application/v31np') {
    http_response_code(415);
    exit;
}
$most = (function (string $v): int {
    $n = (int)$v;
    return match (strtoupper(substr(trim($v), -1))) { 'G' => $n << 30, 'M' => $n << 20, 'K' => $n << 10, default => $n };
})(ini_get('post_max_size'));
if ($most > 0 && (int)($_SERVER['CONTENT_LENGTH'] ?? 0) > $most) {
    http_response_code(413);
    exit;
}
$broken = function () {
    http_response_code(422);
    exit;
};
$body = file_get_contents('php://input');
$outer = V31nPack::parse(V31nPack::MAGIC, $body);
if ($outer === null || count($outer['entries']) !== 1 || $outer['entries'][0][0] !== '') {
    $broken();
}
[, $at, $size] = $outer['entries'][0];
$inner = substr($body, $at, $size);
$ask = V31nPack::parse(V31nPack::ASK, $inner);
if ($ask === null) {
    $broken();
}
$client = [];
foreach ($ask['entries'] as [$key, $at, $size]) {
    $text = substr($inner, $at, $size);
    if ($size !== 8 || !preg_match('/^[0-9A-F]{8}$/', $text)) {
        $broken();
    }
    $client[$key] = $text;
}

$server = [];
foreach ($info['files'] as $f) {
    $server[$f['key']] = $f;
}
$list = function () use ($archive, $server) {
    $f = $server['v31np index.txt'] ?? null;
    if ($f === null) {
        http_response_code(500);
        exit;
    }
    $h = fopen($archive, 'rb');
    fseek($h, $f['at']);
    $bytes = fread($h, $f['size']);
    fclose($h);
    $answer = V31nPack::write(V31nPack::MAGIC, [['v31np index.txt', $bytes]]);
    header('HTTP/1.1 268 List');
    header('Content-Type: application/v31np');
    header('Content-Length: ' . strlen($answer));
    echo $answer;
    exit;
};

if (!$client) {
    if ($settings['empty'] === '268') {
        $list();
```

```
    }
    if ($settings['empty'] === '303') {
        header('Location: /v31np/pre/' . $name . '.v31np', true, 303);
        exit;
    }
    $full();
}

if ($settings['behind'] === '268') {
    $wanted = [];
    foreach ($client as $key => $crc) {
        if ($crc === '00000000') {
            $wanted[] = (string)$key;
        }
    }
    if ($wanted) {
        $own = $server['v31np index.txt'] ?? null;
        if ($own !== null && ($client['v31np index.txt'] ?? null) !== $own['crc'] && !in_array('v31np index.txt', $wanted, true)) {
            $wanted[] = 'v31np index.txt';
        }
        usort($wanted, [V31nOrder::class, 'key']);
        $items = [];
        $innerLength = 0;
        foreach ($wanted as $key) {
            $size = isset($server[$key]) ? $server[$key]['size'] + 1 : 1;
            $items[] = [$key, $size];
            $innerLength += $size;
        }
        $innerHead = V31nPack::head(V31nPack::PATCH, $items);
        $innerLength += strlen($innerHead) + V31nPack::FOOT;
        $outerHead = V31nPack::head(V31nPack::MAGIC, [['', $innerLength]]);
        header('HTTP/1.1 267 Re-sync');
        header('Content-Type: application/v31np');
        header('Content-Length: ' . (strlen($outerHead) + $innerLength + V31nPack::FOOT));
        $outer = hash_init('crc32b');
        $inner = hash_init('crc32b');
        $put = function (string $bytes, bool $both = true) use ($outer, $inner): void {
            echo $bytes;
            hash_update($outer, $bytes);
            if ($both) {
                hash_update($inner, $bytes);
            }
        };
        $put($outerHead, false);
        $put($innerHead);
        $h = fopen($archive, 'rb');
        foreach ($wanted as $key) {
            if (!isset($server[$key])) {
                $put("\x01");
                continue;
            }
            fseek($h, $server[$key]['at']);
            $left = $server[$key]['size'];
            while ($left > 0) {
                $chunk = fread($h, min(1048576, $left));
                if ($chunk === false || $chunk === '') {
                    exit;
                }
                $put($chunk);
                $left -= strlen($chunk);
            }
            $put("\x00");
            flush();
        }
        fclose($h);
        $put(V31nPack::PI);
        $put(pack('V', hexdec(hash_final($inner))), false);
        $put(V31nPack::PI, false);
        echo pack('V', hexdec(hash_final($outer)));
        exit;
    }
    if (isset($server['v31np index.txt']) && ($client['v31np index.txt'] ?? null) === $server['v31np index.txt']['crc']) {
        http_response_code(304);
        exit;
    }
    $list();
}
$changed = [];
foreach ($server as $key => $f) {
    if (($client[$key] ?? null) !== $f['crc']) {
        $changed[] = $key;
    }
}
$deleted = array_values(array_diff(array_map('strval', array_keys($client)), array_map('strval', array_keys($server))));
if (!$changed && !$deleted) {
    http_response_code(304);
    exit;
}
if ($settings['behind'] === '200') {
    $full();
}

$keys = array_merge($changed, $deleted);
usort($keys, [V31nOrder::class, 'key']);
$h = fopen($archive, 'rb');
$entries = [];
foreach ($keys as $key) {
    if (isset($server[$key])) {
        $f = $server[$key];
        $bytes = '';
        if ($f['size'] > 0) {
            fseek($h, $f['at']);
            $bytes = fread($h, $f['size']);
        }
        $entries[] = [(string)$key, $bytes . "\x00"];
    } else {
        $entries[] = [(string)$key, "\x01"];
    }
}
fclose($h);
$answer = V31nPack::write(V31nPack::MAGIC, [['', V31nPack::write(V31nPack::PATCH, $entries)]]);
header('HTTP/1.1 267 Re-sync');
header('Content-Type: application/v31np');
header('Content-Length: ' . strlen($answer));
echo $answer;
```

20 - File: index.js

Size

3474

SHA-256

39484BD4947DE1CE8F9923D480D247F2C714C59D8F686CFA1C006D2B2F9C9630

Code

```
const fs = require('fs');
const path = require('path');
const core = require('./node/core.js');

const pathOf = req => {
    const url = new URL(req.originalUrl || req.url, 'http://x');
    let pathname;
    try { pathname = decodeURIComponent(url.pathname); } catch { pathname = null; }
    return { pathname, query: url.searchParams };
};

const handle = async (req, res) => {
    const { pathname, query } = pathOf(req);
    if (pathname === null || !pathname.startsWith('/v31np/')) return false;
    if (pathname.includes('\0')) { core.end(res, 400); return true; }
    const rest = pathname.slice('/v31np/'.length);
    const name = /^([^/]+)\.v31np$/.exec(rest);
    if (name) { await core.exchange(req, res, name[1], query); return true; }
    const pre = /^pre\/([A-Za-z0-9_-]+)\.v31np$/.exec(rest);
    if (pre && core.archive(pre[1])) {
        const a = core.archive(pre[1]);
        if (req.method !== 'GET' && req.method !== 'HEAD') { core.end(res, 405, { Allow: 'GET, HEAD' }); return true; }
        core.ranges(req, res, a.file, 0, fs.statSync(a.file).size, 'application/v31np', a.info.crc);
        return true;
    }
    if (rest === 'v31np.js') {
        const file = path.join(core.HOME, 'v31np.js');
        if (!fs.existsSync(file)) { core.end(res, 404); return true; }
        if (req.method !== 'GET' && req.method !== 'HEAD') { core.end(res, 405, { Allow: 'GET, HEAD' }); return true; }
        const b = fs.readFileSync(file);
        core.end(res, 200, { 'Content-Type': 'text/javascript; charset=utf-8', 'Content-Length': b.length }, req.method === 'HEAD' ? undefined : b);
        return true;
    }
    core.end(res, 404);
    return true;
};

function v31np(req, res, next) {
    const done = handle(req, res).catch(() => {
        if (!res.headersSent) core.end(res, 500);
        else res.destroy();
        return true;
    });
    if (typeof next !== 'function') return done;
    done.then(handled => { if (!handled) next(); });
}

v31np.page = async (req, res, name, html, look = '') => {
    if (!core.NAME.test(name)) return core.end(res, 500);
    const method = req.method;
    const type = String(req.headers['content-type'] || '').split(';')[0].trim().toLowerCase();
    const accept = String(req.headers.accept || '');
    if ((method === 'POST' && type === 'application/v31np') || ((method === 'GET' || method === 'HEAD') && accept.includes('application/v31np'))) {
        const index = core.loadIndex(core.INDEX);
        if (!index) return core.end(res, 500);
        const page = typeof html === 'function' ? await html() : html;
        const send = [].concat(core.settings(name).send);
        const keys = [...index.rows.keys()].filter(k => send.includes(index.rows.get(k).level || 'Ú'));
        return core.sendAnswer(req, res, index, keys, page);
    }
    const shell = Buffer.from('<!DOCTYPE html>\n<html>\n<head>\n<meta charset="utf-8">\n<link rel="icon" href="data:,">\n'
        + '<meta name="v31np-archive" content="' + name + '">\n<meta name="v31np-url" content="/v31np/' + name + '.v31np">\n' + look + '<script>\n'
        + 'const V31N_PAGE = {};\n'
        + fs.readFileSync(path.join(core.HOME, 'v31np.js'), 'utf8')
        + '</script>\n</head>\n<body>\n</body>\n</html>\n');
    core.end(res, 200, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'no-store', Vary: 'Accept, Content-Type', 'Content-Length': shell.length }, method === 'HEAD' ? undefined : shell);
};

module.exports = v31np;
```

21 - File: page.php

Size

3278

SHA-256

94D9F6277368409783437 7EE8B0857F841F9D860C272384164183EE00AD3AD28

Code

```
<?php

function v31np_query(string $indexFile, callable $keys): void
{
    header('Vary: Accept, Content-Type');
    $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
    $type = strtolower(trim(explode(';', (string)($_SERVER['CONTENT_TYPE'] ?? ''))[0]));
    $accept = (string)($_SERVER['HTTP_ACCEPT'] ?? '');
    if (!(($method === 'POST' && $type === 'application/v31np') || (($method === 'GET' || $method === 'HEAD') && str_contains($accept, 'application/v31np')))) {
        return;
    }
    require_once __DIR__ . '/lib/answer.php';
    set_time_limit(0);
    v31np_plain();
    $index = V31nIndex::load($indexFile) ?? (is_file($indexFile) ? null : V31nIndex::create($indexFile, '.'));
    if ($index === null) {
        http_response_code(500);
        exit;
    }
    $level = ob_get_level();
    V31nAnswer::start();
    register_shutdown_function(function () use ($index, $level, $keys): void {
        while (ob_get_level() > $level + 1) {
            ob_end_flush();
        }
        V31nAnswer::send($index, array_values(array_filter(array_map(fn($k) => V31nIndex::rooted((string)$k), (array)$keys((string)ob_get_contents(), $index)), fn($k) => $index->has($k))));
    });
}

function v31np_plain(): void
{
    if (function_exists('apache_setenv')) {
        apache_setenv('no-gzip', '1');
        apache_setenv('no-brotli', '1');
    }
    ini_set('zlib.output_compression', '0');
}

function v31np(string $name, string $look = ''): void
{
    require_once __DIR__ . '/lib/settings.php';
    if (!preg_match('/^[A-Za-z0-9_-]+$/', $name)) {
        http_response_code(500);
        exit;
    }
    header('Vary: Accept, Content-Type');
    $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
    $type = strtolower(trim(explode(';', (string)($_SERVER['CONTENT_TYPE'] ?? ''))[0]));
    $accept = (string)($_SERVER['HTTP_ACCEPT'] ?? '');
    if (($method === 'POST' && $type === 'application/v31np') || (($method === 'GET' || $method === 'HEAD') && str_contains($accept, 'application/v31np'))) {
        require_once __DIR__ . '/lib/answer.php';
        set_time_limit(0);
        v31np_plain();
        $file = dirname(__DIR__) . '/' . V31nIndex::KEY;
        if (V31nIndex::load($file) === null) {
            http_response_code(500);
            exit;
        }
        $level = ob_get_level();
        V31nAnswer::start();
        register_shutdown_function(function () use ($file, $level, $name): void {
            while (ob_get_level() > $level + 1) {
                ob_end_flush();
            }
            $index = V31nIndex::load($file);
            $send = (array)v31np_settings($name)['send'];
            V31nAnswer::send($index, array_values(array_filter($index->keys(), fn($k) => in_array($index->level($k), $send, true))));
        });
        return;
    }
    header('Content-Type: text/html; charset=utf-8');
    header('Cache-Control: no-store');
    echo "<!DOCTYPE html>\n<html>\n<head>\n<meta charset=\"utf-8\">\n<link rel=\"icon\" href=\"data:,\">\n";
    echo '<meta name="v31np-archive" content="', $name, '">\n", '<meta name="v31np-url" content="/v31np/", $name, ".v31np\">\n", $look, "<script>\n";
    echo "const V31N_PAGE = {};\n";
    readfile(__DIR__ . '/v31np.js');
    echo "</script>\n</head>\n<body>\n</body>\n</html>\n";
    exit;
}
```

22 - File: .htaccess

Size

249

SHA-256

DA370D49401440C14E3AA50867D50E195BBEB427A5499E8D5111771A9B0096F4

Code

```
RewriteEngine On
RewriteRule ^(lib/|keys/|node/|settings\.json$|pre/.*\.json$|build\.(php|js)$|index\.js$|page\.php$) - [F]
RewriteRule ^([^/]+)\.v31np$ v31np.php?name=$1 [L,QSA]

AddType application/v31np .v31np
SetEnv no-gzip 1
SetEnv no-brotli 1
```



V31NP : Drop-in

Document: PS-N/PF-1032

PDF generated on: 2026 11/05 04:07:16 AM

1 - End point: /v31np/<name>.v31np

Method *POST*

Content-Type *application/v31np*

Body *Request of PS-N/PF-1032, § (Ú)*

Answers *200, 268 List, 267 Re-sync, 303, 304, 405, 413, 415, 422*

2 - End point: /v31np/<name>.v31np

Method *GET, HEAD*

Range *bytes=*

Answers *200, 206, 400, 416*

3 - End point: /v31np/<name>.v31np?key=<key>

Method *GET, HEAD*

ETag *CRC-32 of the file*

Header fields *H rows of the index entry*

Answers *200, 206, 400, 404, 405, 414, 416*

4 - End point: /v31np/pre/<name>.v31np

Method *GET, HEAD*

Answers *200, 206, 400, 416*

5 - End point: /v31np/v31np.js

Method *GET, HEAD*

data-archive *<name>*

data-url *URL of the archive*

data-key *public key, 64 hexadecimal digits*

6 - End point: URL of a page with the call

Method *POST*

Content-Type	*application/v31np*
Answers	*200, 267 Re-sync, 304, 422*
Method	*GET, HEAD*
Accept	*application/v31np*
Answers	*200*
Other	*Shell*

7 - Command

PHP	*php v31np/build.php <name> [skip pattern ...]*
Node.js	*node v31np/build.js <name> [skip pattern ...]*

8 - Node.js

Handler	*if (await v31np(req, res)) return;*
Express	*app.use(v31np)*
Page	*v31np.page(req, res, '<name>', html)*

9 - Settings: v31np/settings.json

empty	*268, 303, 200*
behind	*268, 267, 200*
send	*S, A, B, Ú, U, Ù*
sign	*keys/<file>*
archives	*<name>*